

sa1sbadw5

January 3, 2025

1 CHAPTER 1: DEFINITION OF THE PROBLEM STATEMENT

1.1 INTRODUCTION TO ONLINE TECH-VERSITY

This project focuses on an online tech-versity offering intensive computer science & data science courses through live classes delivered by tech leaders and subject matter experts. The meticulously structured program enhances the skills of software professionals by offering a modern curriculum with exposure to the latest technologies. The platform was founded by industry experts in 2019.

The platform offers online programs and master's programs like Software Development, Data Science & Machine Learning, and DevOps for working professionals and graduates. It also provides on-campus programs like a School of Business and a School of Technology for undergraduate-level students.

1.2 DEFINITION OF THE PROBLEM

The platform endeavors to provide world-class education in computer science & data science domains.

A significant challenge for the platform is understanding the diverse backgrounds of its learners, especially in terms of their current roles, companies, and experience. Clustering similar learners helps in customizing the learning experience, thereby increasing retention and satisfaction.

Analyzing the vast data of learners can uncover patterns in their professional backgrounds and preferences. This allows the platform to make tailored content recommendations and provide specialized mentorship.

1.3 PROBLEM STATEMENT

Assume you are working as a data scientist with the analytics vertical of the platform, focused on profiling the best companies and job positions to work for from the learner database.

You are provided with the information for a segment of learners and are tasked to cluster them on the basis of their job profile, company, and other features. Ideally, these clusters should have similar characteristics.

1.4 IMPORTING THE LIBRARIES AND DATASET

1.4.1 Importing all the required libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import math, warnings, mlflow, time, os, pickle
import regex as re
from sklearn.model_selection import train_test_split
from category_encoders import CatBoostEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import IncrementalPCA
from sklearn.ensemble import IsolationForest
from umap import UMAP
from sklearn.cluster import KMeans
import plotly.express as px
from matplotlib.colors import ListedColormap
from matplotlib.lines import Line2D
from sklearn.metrics import silhouette_score
```

```
[333]: pd.set_option('display.max_colwidth', None)
```

```
[334]: def display_all(df):      # For any Dataframe df
    with pd.option_context('display.max_rows', 100): # Change number of rows
        ↪ accordingly
        with pd.option_context('display.max_columns', 100): # Change number of
            ↪ columns accordingly
            display(df)
```

1.4.2 Importing the Dataset

```
[335]: data = pd.read_csv("clustering.csv")
```

```
[336]: # top 10 rows of the dataset
data.head(10)
```

```
[336]:   Unnamed: 0      company_hash \
0          0      atrgxnnt xzaxv
1          1      qtrxvzwt xzegwgb rxbxnta
2          2      ojzwnvwnxw vx
3          3      ngpgutaxv
4          4      qxen sqghu
5          5      yvuuxrj hzbvqqxta bvqptnxzs ucn rna
6          6      lubgqsvz wyvot wg
7          7      vwwtznht ntwyzgrgsj
```

```

8          8          utqoxontzn ojointbo
9          9          xrbhd

```

```

                                email_hash  orgyear  \
0  6de0a4417d18ab14334c3f43397fc13b30c35149d70c050c0618caea697c87af  2016.0
1  b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c100a9661a92bdcc0407b  2018.0
2  4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e90fd66c9df6b37b9059  2015.0
3  effdede7a2e7c2af664c8a31d9346385016128d66bbc58a44274d5d6876dfec7  2017.0
4  6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520214b31b95211adb095  2017.0
5  18f2c4aa2ac9dd3ae8ff74f32d30413f5165565b90d8f21a44173a1f73fbb0f2  2018.0
6  9bf128ae3f4ea26c7a38b9cdc58cf2acbb8592100c41288e867aac925d40a603  2018.0
7  756d35a7f6bb8ffeaffc8fcca9ddb78e7450fa0de2be0a5e9c8d98eb87f97f9  2019.0
8  e245da546bf50eba09cb7c9976926bd56557d1ac9a17fb019e8de1fdb83fc0d6  2020.0
9  b2dc928f4c22a9860b4a427efb8ab761e1ce0015fba1a5e804e1dc27e305b06b  2019.0

```

```

          ctc          job_position  ctc_updated_year
0  1100000          Other          2020.0
1   449999  FullStack Engineer          2019.0
2  2000000  Backend Engineer          2020.0
3   700000  Backend Engineer          2019.0
4  1400000  FullStack Engineer          2019.0
5   700000  FullStack Engineer          2020.0
6  1500000  FullStack Engineer          2019.0
7   400000  Backend Engineer          2019.0
8   450000          NaN          2019.0
9   360000          NaN          2019.0

```

1.4.3 Description regarding each column of the dataset

Column Name	Description
Unnamed: 0	The index of the dataset
company_hash	An anonymized identifier indicating the current employer of the learner
email_hash	An anonymized identifier representing the email of the learner
orgyear	Represents the year the learner began employment at the current company
ctc	Current Compensation to the Company (CTC) of the learner
job_position	Represents the job profile or role of the learner within their company
ctc_updated_year	The year in which the learner's CTC was most recently updated. This could be due to yearly increments, promotions, or other factors

2 CHAPTER 2: EDA, DATA CLEANING AND PRELIMINARY DATA PREPROCESSING STEPS

2.1 BASIC EDA

2.1.1 Number of rows and columns

```
[337]: print(f"Number of rows in the dataset = {data.shape[0]}")
       print(f"Number of columns in the dataset = {data.shape[1]}")
```

Number of rows in the dataset = 205843

Number of columns in the dataset = 7

2.1.2 Datatypes of all the attributes

```
[338]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205843 entries, 0 to 205842
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Unnamed: 0            205843 non-null int64   
 1   company_hash          205799 non-null object  
 2   email_hash            205843 non-null object  
 3   orgyear               205757 non-null float64  
 4   ctc                   205843 non-null int64   
 5   job_position          153279 non-null object  
 6   ctc_updated_year      205843 non-null float64  
dtypes: float64(2), int64(2), object(3)
memory usage: 11.0+ MB
```

Observation > There are null values in company_hash, orgyear, & job_position

Should drop Unnamed: 0 feature to check if any duplicated rows in the dataset.

2.1.3 Drop Unnamed: 0 Feature to check duplicated rows

```
[339]: data = data.drop(["Unnamed: 0"],axis = 1)
```

```
[340]: data.head()
```

```
[340]:      company_hash \
0      atrgxnnt xzaxv
1  qtrxvzwt xzegwbbb rxbxnta
2      ojzwnvwnxw vx
3      ngpgutaxv
4      qxen sqghu
```

		email_hash	orgyear	\
0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c050c0618caea697c87af		2016.0	
1	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c100a9661a92bdcc0407b		2018.0	
2	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e90fd66c9df6b37b9059		2015.0	
3	effdede7a2e7c2af664c8a31d9346385016128d66bbc58a44274d5d6876dfec7		2017.0	
4	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520214b31b95211adb095		2017.0	

	ctc	job_position	ctc_updated_year
0	1100000	Other	2020.0
1	449999	FullStack Engineer	2019.0
2	2000000	Backend Engineer	2020.0
3	700000	Backend Engineer	2019.0
4	1400000	FullStack Engineer	2019.0

2.1.4 Duplicated rows

```
[341]: data.duplicated().sum()
```

```
[341]: 34
```

```
[342]: data.drop_duplicates(keep = "last" , inplace=True)
```

Note: We can keep last or first. Both are same because there are duplicated rows.

```
[343]: data.duplicated().sum()
```

```
[343]: 0
```

```
[344]: len(data)
```

```
[344]: 205809
```

2.1.5 Missing value or Null Value Detection

```
[345]: # Null value count
data.isnull().sum()
```

```
[345]: company_hash      44
email_hash              0
orgyear                86
ctc                    0
job_position          52548
ctc_updated_year       0
dtype: int64
```

```
[346]: # Percenatage of Null Values
(data.isna().sum() / data.shape[0]) * 100
```

```
[346]: company_hash      0.021379
       email_hash       0.000000
       orgyear          0.041786
       ctc              0.000000
       job_position     25.532411
       ctc_updated_year  0.000000
       dtype: float64
```

```
[347]: # Filtering the columns which have zero null values and sorting in descending
       ↪order
def missing_data(data):
    total_missing_data = data.isna().sum().sort_values(ascending = False)
    percentage_missing_data = ((data.isna().sum()/len(data)*100)).
    ↪sort_values(ascending = False)
    missingdata = pd.concat([total_missing_data, percentage_missing_data],axis=
    ↪= 1, keys=['Total', 'Percent'])
    return missingdata

missing_data = missing_data(data)
missing_data[missing_data["Total"]>0]
```

```
[347]:
```

	Total	Percent
job_position	52548	25.532411
orgyear	86	0.041786
company_hash	44	0.021379

Observation > There are 25.53% percent null values in job_position. We cannot drop them as it is huge number of null values.

Should check company_hash null values. Should impute based on email_hash

2.2 NULL VALUE IMPUTATION & DATA CLEANING

2.2.1 Trying to impute company_hash null value using latest valid company of respective email_hash

```
[348]: data["company_hash"].isna().sum()
```

```
[348]: 44
```

```
[349]: data[data["company_hash"].isna()]
```

```
[349]:
```

	company_hash	\
1115	NaN	
2400	NaN	
3277	NaN	
4205	NaN	

4596	NaN
11753	NaN
14739	NaN
18853	NaN
19466	NaN
22846	NaN
31489	NaN
40272	NaN
45630	NaN
48583	NaN
62974	NaN
68706	NaN
71138	NaN
76614	NaN
79347	NaN
80516	NaN
82680	NaN
85037	NaN
93712	NaN
103107	NaN
103299	NaN
108479	NaN
117021	NaN
117915	NaN
125509	NaN
130031	NaN
141214	NaN
141355	NaN
142613	NaN
142955	NaN
146191	NaN
156511	NaN
162073	NaN
171527	NaN
175270	NaN
176727	NaN
178114	NaN
178330	NaN
194796	NaN
202282	NaN

	email_hash \
1115	8fe09b732fe2e5b66c14904fd02ff89fb54f458465ac1e5b04468e11db641e01
2400	1074b55f02e6fc88596db85854e057c98cb53c038e0d7f3d5e353e0c1d1a977b
3277	66263f4942b046c67ae6e2570e7825c03792631a0b13f1b5fed1fe3eafc396db
4205	6eb55d779699a2ea94f340ab7a58c8ec505e38bbb41214278e72b5f03d2af064
4596	18813fe2a50a45cc02c5b3871c676bd147c80ff0327ee9e7bd8d9c121d9fa6d3

11753 ea4f735b9357e8086a42bacc1f64b18e98c3dd1ad81f60140c0f4e6be8616830
14739 b4a56d1199bc569aabd30cba8ea7a86fbddc85211453bac05e92b44a8e82e090
18853 07a60d6e853852471b0963b78a0a3074532572a258086431b4a14a2a1e2aaecc
19466 07a60d6e853852471b0963b78a0a3074532572a258086431b4a14a2a1e2aaecc
22846 bdce6736cc1d55a909a46aed9e0bfdcd7cd523bfcf9b63ffa17c618a9a6424e5
31489 8e70184e76f9a29078e8ddd928d24582e096f5b1a63197a6e3803966b40bff31
40272 4baf80fe2b9513f2f1d17d90f26071bd21f4a89d865fa1d16a18143c7b94e972
45630 b17c74b195c1fa8038bf82c674716ae81b41b995a3b4349fb7ae7d07e0873599
48583 d9d7be8e4e4e5b6eb1092772d366c6bb21c8502e0e8253871fd6605da409e721
62974 8420dc8fe52b5acaf629914b3917fbc37111924b9654042f659f342abb9ea48f
68706 8fe09b732fe2e5b66c14904fd02ff89fb54f458465ac1e5b04468e11db641e01
71138 8ce727669517d613c973b752e211e50f0bad3cab50d7cb1e03517b100bdb0473
76614 c83f98e2b2fb365515f48002f40db363a9de3319069f383a64f16d8b94d6bcc9
79347 1a5f329f97cdac513d7e33b5f8705e46053595ef6254c90ff68ba3d711588542
80516 cd6bc6ddf180c00306ad009a187cc5eb2a4b62af4ad58427f058a459a1ed8287
82680 5ef6071f5c390f317dfa60f7aad9ee7a1abc92aaa02fc68c4b479578b52eca8
85037 a80b0711a63a65c0b70c3ed3f825043a8fc2b6871c3b0e53aad5dff78929c197
93712 50f183667fd8a115dda5aa345988b314e1d98a3d937ea047ad82db9148caddbc
103107 98177023d0d95876047a39ed525d4c7eb44af739502aca75a9b78373ef88eb29
103299 87f640fc89281c082d94d1ee7fd6ee7391a8e30ad6182ba292ddd14aa502ba59
108479 50f183667fd8a115dda5aa345988b314e1d98a3d937ea047ad82db9148caddbc
117021 1606fcb8a2b3e4b242df4ee71190194fc556cf0d54861633038209d41757b52a
117915 f4bb477609fc559301e37c73a5b17428afe18c53c0fa27f4993cba5efb3f425b
125509 9f9ba6d4e58f4f175f348d6d188be8ee8d5aa3537a0187ae5c6b7b067f01e051
130031 d7f39bfaa3be4957fa36a97fab5ab7a39f6d55f377f334ece6bd1b3c8f6558a
141214 cd281f18ef3d9042fab48860b4a7f80ec1d559c3fc2f857c266fc367b05a4e4d
141355 a51106f9193e46db561f27b02db413396e651be5394ca5e008ce4bdc1fc9ae75
142613 ce803383ed3dda0838959afb466eb1cffff964c94cb1ea7cc5dec8e6293e61d4
142955 f47fbe35140825c07caf830b18058e737a3c6f18c50f503c535e5efdfbe5ac50
146191 f47fbe35140825c07caf830b18058e737a3c6f18c50f503c535e5efdfbe5ac50
156511 c9af26980cf32f393089c1b33d3e138450e506b2d044a359a23d117e36362ff5
162073 d000a77f0045504e2ee51a667ac0ad2671795b3f70ce33f16f9c06ba4ca20aa1
171527 824c00340acc623b57c75ca41535bf9d52fdee81d006bf7991990f47b1a62d99
175270 aae19078f349d6403856abf2c28ee731a32c0dbeb8de76587828b494a777454a
176727 a75da322109f201148da6b4a1ab785518e6229c1379a09a2501fa05a5ee19252
178114 3fd7b50dc84e8b2493f097c6fc33c8abed19107ed0b1d3f7330668f89f0bcfcd
178330 0e781c3797c031c6aad2fa3d97c82773624a5da9a35de93195eb67c77e0199e3
194796 c1ef4ed5eeb40dcacbb5f7d0fc345fb77c6176024eb2391f124989b73f76f4fc
202282 c9af26980cf32f393089c1b33d3e138450e506b2d044a359a23d117e36362ff5

	orgyear	ctc	job_position	ctc_updated_year
1115	2022.0	66600000	NaN	2020.0
2400	2018.0	250000	Other	2019.0
3277	2018.0	500000	Other	2019.0
4205	2018.0	600000	NaN	2020.0
4596	2020.0	300000	NaN	2021.0
11753	2018.0	300000	NaN	2021.0

14739	2013.0	1600000	NaN	2021.0
18853	2017.0	700000	FullStack Engineer	2021.0
19466	2017.0	700000	NaN	2021.0
22846	2010.0	2000000	NaN	2020.0
31489	2018.0	229999	NaN	2021.0
40272	2012.0	2000000	NaN	2021.0
45630	2020.0	600000	NaN	2020.0
48583	2011.0	910000	Data Scientist	2019.0
62974	2018.0	300000	NaN	2018.0
68706	2022.0	66600000	Database Administrator	2020.0
71138	2019.0	200000	NaN	2020.0
76614	2014.0	1000000	NaN	2021.0
79347	2015.0	2200000	NaN	2021.0
80516	2017.0	300000	NaN	2021.0
82680	2014.0	600000	NaN	2019.0
85037	2019.0	1200000	FullStack Engineer	2018.0
93712	2019.0	500000	Database Administrator	2019.0
103107	2011.0	1200000	NaN	2021.0
103299	2013.0	400000	NaN	2021.0
108479	2019.0	500000	NaN	2019.0
117021	2016.0	700000	NaN	2016.0
117915	2015.0	600000	NaN	2021.0
125509	2017.0	400000	NaN	2021.0
130031	2015.0	800000	Backend Engineer	2019.0
141214	2006.0	1000000	NaN	2019.0
141355	2015.0	110000	NaN	2016.0
142613	2018.0	2000000	Data Analyst	2018.0
142955	2019.0	2700000	FullStack Engineer	2019.0
146191	2019.0	2700000	NaN	2019.0
156511	2021.0	900000	NaN	2020.0
162073	2020.0	800000	NaN	2019.0
171527	2012.0	2100000	Other	2021.0
175270	2018.0	956000	NaN	2017.0
176727	2017.0	1200000	NaN	2018.0
178114	2007.0	132000	NaN	2017.0
178330	2018.0	1800000	NaN	2016.0
194796	2019.0	2000000	Data Scientist	2019.0
202282	2021.0	900000	Other	2020.0

```
[350]: # procedure to impute the company_hash null values with latest company of
↳respective email_hash
# Step 1: Filter out rows where `company_hash` is NaN
filtered_data = data[~data['company_hash'].isna()]

# Step 2: Sort by `email_hash` and `ctc_updated_year` (descending to get the
↳latest year at the top)
```

```
sorted_data = filtered_data.sort_values(by=['email_hash', 'ctc_updated_year'],
    ↪ascending=[True, False])

# Step 3: Group by `email_hash` and select the latest `company_hash`
latest_company_dict = sorted_data.groupby('email_hash').first()['company_hash'].
    ↪to_dict()

# `latest_company_dict` now has `email_hash` as keys and the latest
    ↪`company_hash` as values

# Replace NaN values in `company_hash` using the dictionary
data['company_hash'] = data.apply(
    lambda row: latest_company_dict[row['email_hash']] if pd.
    ↪isna(row['company_hash']) and row['email_hash'] in latest_company_dict else
    ↪row['company_hash'],
    axis=1
)
```

```
[351]: data["company_hash"].isna().sum()
```

```
[351]: 37
```

Observation > Before imputation 44 , After imputation 37 Null values are present. Only 7 Rows are imputed with this method

So remove or drop the remaining 37 null values rows of company_hash

2.2.2 Drop the remaining company_hash_null rows

```
[352]: data = data.dropna(subset=['company_hash'])
```

```
[353]: data.isna().sum()
```

```
[353]: company_hash      0
email_hash      0
orgyear      86
ctc      0
job_position    52523
ctc_updated_year      0
dtype: int64
```

2.2.3 Descriptive Statistics regarding each column of dataset

```
[354]: data.describe()
```

```
[354]:
```

	orgyear	ctc	ctc_updated_year
count	205686.000000	2.057720e+05	205772.000000

mean	2014.882014	2.271453e+06	2019.628307
std	63.582047	1.180121e+07	1.325141
min	0.000000	2.000000e+00	2015.000000
25%	2013.000000	5.300000e+05	2019.000000
50%	2016.000000	9.500000e+05	2020.000000
75%	2018.000000	1.700000e+06	2021.000000
max	20165.000000	1.000150e+09	2021.000000

Observation > Dataset contains 0 to 20165 in orgyear. Clearly it is human_typing error. Should handle it.

ctc_updated_year is in range of 2015 to 2021. No human typing errors in it.

According to common sense, ctc_updated_year should be Future or Present year to the orgyear. We can use this while eliminating the unwanted rows. Should verify these rows.

ctc is in range of Rs 2/- to Rs 1,00,01,50,000/- (100 crores) (assuming currency in Indian Rupees). There is a high chance of outliers because of this ctc feature. Should handle them.

```
[355]: data.describe(include = "object")
```

```
[355]:
```

	company_hash \
count	205772
unique	37299
top	nvnnv wgzohrnrvzwj otqcxwto
freq	8337

	email_hash \
count	205772
unique	153411
top	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
freq	10

	job_position
count	153249
unique	1016
top	Backend Engineer
freq	43546

```
[356]: len(data)
```

```
[356]: 205772
```

Observation > Unique values in email_hash != 205772, So email_hash cannot be primary key of the dataset. That means there are repetitions of same learner multiple times in the dataset.

There are 37299 unique companies and nvnnv wgzohrnrvzwj otqcxwto company has repeated most number of times in the dataset

There are 1016 unique job_positions. Backend Engineer is high frequency job_position

2.2.4 Number of unique values in each column of given dataset

```
[357]: for i in data.columns:
        print(i,":",data[i].nunique())
```

```
company_hash : 37299
email_hash   : 153411
orgyear      : 77
ctc          : 3359
job_position : 1016
ctc_updated_year : 7
```

2.2.5 Unique values of columns whose nunique < 78

```
[358]: for i in data.columns:
        if data[i].nunique() < 78:
            print(i,sorted(data[i].unique()),"",sep = "\n")
```

```
orgyear
[1973.0, 1981.0, 1990.0, 1991.0, 1992.0, 1994.0, 1995.0, 1996.0, 1997.0, 1998.0,
1999.0, 2000.0, 2001.0, 2002.0, 2003.0, 2004.0, 2005.0, 2006.0, 2007.0, 2008.0,
2009.0, 2010.0, 2011.0, 2012.0, 2013.0, 2014.0, 2015.0, 2016.0, 2017.0, 2018.0,
2019.0, 2020.0, 2021.0, 2022.0, 2023.0, 2031.0, nan, 0.0, 1.0, 2.0, 3.0, 4.0,
5.0, 6.0, 38.0, 83.0, 91.0, 200.0, 201.0, 206.0, 208.0, 209.0, 1900.0, 1970.0,
1971.0, 1972.0, 1976.0, 1977.0, 1979.0, 1982.0, 1984.0, 1985.0, 1986.0, 1987.0,
1988.0, 1989.0, 1993.0, 2024.0, 2025.0, 2026.0, 2027.0, 2028.0, 2029.0, 2101.0,
2106.0, 2107.0, 2204.0, 20165.0]
```

```
ctc_updated_year
[2015.0, 2016.0, 2017.0, 2018.0, 2019.0, 2020.0, 2021.0]
```

Observation > We can clearly observe there are bunch of orgyear values which are nan, 0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 38.0, 83.0, 91.0, 200.0, 201.0, 206.0, 208.0, 209.0, 1900.0, 2025.0, 2026.0, 2027.0, 2028.0, 2029.0, 2031.0, 2101.0, 2106.0, 2107.0, 2204.0, 20165.0 (Meaning-less)

2.2.6 Datatypes

```
[359]: data.dtypes
```

```
[359]: company_hash      object
email_hash             object
orgyear               float64
ctc                   int64
job_position          object
```

```
ctc_updated_year    float64
dtype: object
```

Observation > company_hash, email_hash and job_position are string datatypes. Remaining are numerical

2.2.7 Range of values of all numerical

```
[360]: for i in data.columns:
        if not(isinstance(data[i][0],str)):
            print(f"Maximum of {i}",data[i].max())
            print(f"Minimum of {i}",data[i].min())
            print()
```

```
Maximum of orgyear 20165.0
Minimum of orgyear 0.0
```

```
Maximum of ctc 1000150000
Minimum of ctc 2
```

```
Maximum of ctc_updated_year 2021.0
Minimum of ctc_updated_year 2015.0
```

Observation > Range of orgyear has human typing error.

2.2.8 Value counts of all features

```
[361]: for i in data.columns:
        print("Value Counts of {}".format(i),end="\n\n")
        print(data[i].value_counts(dropna= False),end="\n\n")
```

Value Counts of company_hash

```
company_hash
nvnv wgzohrnrvzwj otqcxwto    8337
xzegojo                      5382
vbvkgz                       3481
zgn vuurxwvmrt vwwghzn       3410
wgszxkvzn                    3240
...
ihxwprgsxw ogenfvqt          1
onvqmhwp                      1
bvsxw ogenfvqt uqxcvnt rxbxnta 1
agsbv ojointbo                1
bvptbjnqxu td vbvkgz          1
Name: count, Length: 37299, dtype: int64
```

Value Counts of email_hash

```
email_hash
bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b 10
6842660273f70e9aa239026ba33bfe82275d6ab0d20124021b952b5bc3d07e6c 9
3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378 9
298528ce3160cc761e4dc37a07337ee2e0589df251d73645aae209b010210eee 9
d15041f58bb01c8ee29f72e33b136e26bc32f3169a40b53d75fe7ae9cbb9a551 8
..
09102244a24ed7ae3beff197e3bfa8402f2d72dc69e1fa5bee18d9fc5d6aeae 1
bc51538e8265ba631baf120d53a17ac4aac9232127285e5c9fa09117af2c68f 1
ff9cc70bdbe86b3929ed54254034eb4508a62745d106664b5356e2aea4e73a8b 1
9148cf557e1df4894984d8c8e5384b16d9d023959c0d58ec677612700d0ae068 1
0bcfc1d05f2e8dc4147743a1313aa70a119b41b30d4a1f7e738a6a87d3712c31 1
Name: count, Length: 153411, dtype: int64
```

Value Counts of orgyear

```
orgyear
2018.0    25245
2019.0    23413
2017.0    23229
2016.0    23037
2015.0    20606
...
2107.0      1
1972.0      1
2101.0      1
208.0        1
200.0        1
Name: count, Length: 78, dtype: int64
```

Value Counts of ctc

```
ctc
600000    7829
400000    7596
1000000    7576
500000    7238
800000    6750
...
1916000      1
5340000      1
2305000      1
4225000      1
3327000      1
Name: count, Length: 3359, dtype: int64
```

Value Counts of job_position

```
job_position
NaN                    52523
Backend Engineer      43546
FullStack Engineer    24708
Other                 18067
Frontend Engineer     10417
...
Compliance auditor    1
91                    1
Senior Software Development Engineer (Backend) 1
Messenger come driver 1
Android Application developer 1
Name: count, Length: 1017, dtype: int64
```

Value Counts of ctc_updated_year

```
ctc_updated_year
2019.0    68655
2021.0    64961
2020.0    49429
2017.0     7559
2018.0     6743
2016.0     5498
2015.0     2927
Name: count, dtype: int64
```

Observation > Same email has repeated many times in the dataset. Should verify it.

Should Clean and apply preprocessing steps on the dataset properly before applying clustering techniques.

2.2.9 Why there are repetitions in email_hash

```
[362]: sum(data["email_hash"].value_counts() > 1)
```

```
[362]: 41186
```

Observation > Around 41186 emails have repeated more than once

```
[363]: data["email_hash"].value_counts()
```

```
[363]: email_hash
bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b    10
6842660273f70e9aa239026ba33bfe82275d6ab0d20124021b952b5bc3d07e6c    9
3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378    9
298528ce3160cc761e4dc37a07337ee2e0589df251d73645aae209b010210eee    9
```

```

d15041f58bb01c8ee29f72e33b136e26bc32f3169a40b53d75fe7ae9cbb9a551      8
..
09102244a24ed7ae3beff197e3bfa8402f2d72dc69e1fa5bee18d9fc5d6aeae      1
bc51538e8265ba631bafe120d53a17ac4aac9232127285e5c9fa09117af2c68f      1
ff9cc70bdbbe86b3929ed54254034eb4508a62745d106664b5356e2aea4e73a8b      1
9148cf557e1df4894984d8c8e5384b16d9d023959c0d58ec677612700d0ae068      1
0bcfc1d05f2e8dc4147743a1313aa70a119b41b30d4a1f7e738a6a87d3712c31      1
Name: count, Length: 153411, dtype: int64

```

```

[364]: data[data["email_hash"] ==
↳ "bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b"]

```

```

[364]:
      company_hash \
24109  oxej ntwyzgrgsxto rxbxnta
45984  oxej ntwyzgrgsxto rxbxnta
72315  oxej ntwyzgrgsxto rxbxnta
102915 oxej ntwyzgrgsxto rxbxnta
117764 oxej ntwyzgrgsxto rxbxnta
121483 oxej ntwyzgrgsxto rxbxnta
124476 oxej ntwyzgrgsxto rxbxnta
144479 oxej ntwyzgrgsxto rxbxnta
152801 oxej ntwyzgrgsxto rxbxnta
159835 oxej ntwyzgrgsxto rxbxnta

      email_hash \
24109  bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
45984  bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
72315  bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
102915 bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
117764 bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
121483 bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
124476 bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
144479 bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
152801 bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b
159835 bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b

      orgyear      ctc      job_position      ctc_updated_year
24109    2018.0  720000             NaN             2020.0
45984    2018.0  720000    Support Engineer             2020.0
72315    2018.0  720000             Other             2020.0
102915    2018.0  720000  FullStack Engineer             2020.0
117764    2018.0  720000    Data Analyst             2020.0
121483    2018.0  660000             Other             2019.0
124476    2018.0  660000    Support Engineer             2019.0
144479    2018.0  660000  FullStack Engineer             2019.0
152801    2018.0  660000    Devops Engineer             2019.0
159835    2018.0  660000             NaN             2019.0

```


Observation > Same email address, Same company_hash but there are different etc, job_position and etc_updated_year.

So The learner might have been promoted, switched roles, or had salary changes within the same company. But it is not possible to work simultaneously with same salary in different job position. We have to handle by using priority_job_positions and mode values.

2.2.10 Impute orgyear null values

```
[365]: # Step 1: Calculate the minimum 'orgyear' for each 'email_hash' and
        ↪ 'company_hash' group
        # Taking min because we need to give importance to starting date of a person in
        ↪ that company.
min_orgyear = data.groupby(['email_hash', 'company_hash'])['orgyear'].
        ↪ transform('min')

# Step 2: Fill NaN values in 'orgyear' with the minimum value for each group
data['orgyear'] = data['orgyear'].fillna(min_orgyear)
```

```
[366]: data.isna().sum()
```

```
[366]: company_hash      0
email_hash      0
orgyear        81
ctc             0
job_position    52523
ctc_updated_year 0
dtype: int64
```

Observation > Only 5 rows of orgyear got imputed with its min value wrt email & company hash

```
[367]: data[data['orgyear'].isna()]['email_hash'].value_counts()
```

```
[367]: email_hash
c311ade7f3789388138e6bf387276c2f948ee673c2cefbafc6b11c90ee982aa1    2
9f1a8d9d82350279cca75af13cbf6395e665c6d186d9c1148f3ce46b060a6cf2    2
1d1217f79134358ad5ea2e0c6ac9e0685a18c4d24eab512fa9588df081ff180c    1
bb09aebf33a8d0918a49fb30e8728b3153b3298b98d77e08dcff7e3975e6cfb1    1
ae06a986b3e4ad7ffcb8b1ac54faa13a302f724719c37bd34d6d8ba29082d636    1
..
eea42d67722298c9635828cc1aa015d58772477f849f6c0e33a5980aeb397f78    1
a879de3dd54d6ac3dbc886f0b92e5685f8f710fb1ba9aa905c9becc8caf93bc2    1
060f40a7586f40c79211d1d6ba97b928d16991dfb9f4cb8cddb2a4e19c5ced7e    1
6f02761728a8992ad57fe702c81277bed881786575f85f80426fe6bd6f355148    1
638ffe59e3f96d6c2e182979680d4642f947c74b6cef7afee101d416b9f3925a    1
Name: count, Length: 79, dtype: int64
```

```
[368]: data[data['email_hash']=='9f1a8d9d82350279cca75af13cbf6395e665c6d186d9c1148f3ce46b060a6cf2']
```

```
[368]:
```

		company_hash	\
172408		vbuygqv	ogenfvqt uqxcvnt rxbxnta
173365		vbuygqv	ogenfvqt uqxcvnt rxbxnta

		email_hash	\
172408		9f1a8d9d82350279cca75af13cbf6395e665c6d186d9c1148f3ce46b060a6cf2	
173365		9f1a8d9d82350279cca75af13cbf6395e665c6d186d9c1148f3ce46b060a6cf2	

	orgyear	ctc	job_position	ctc_updated_year
172408	NaN	1200000	NaN	2019.0
173365	NaN	1200000	Backend Engineer	2019.0

Observation > We can use ctc_updated_year to impute the orgyear.

```
[369]: data["orgyear"].fillna(data["ctc_updated_year"], inplace=True)
```

```
[370]: data.isna().sum()
```

```
[370]:
```

company_hash	0
email_hash	0
orgyear	0
ctc	0
job_position	52523
ctc_updated_year	0
dtype: int64	

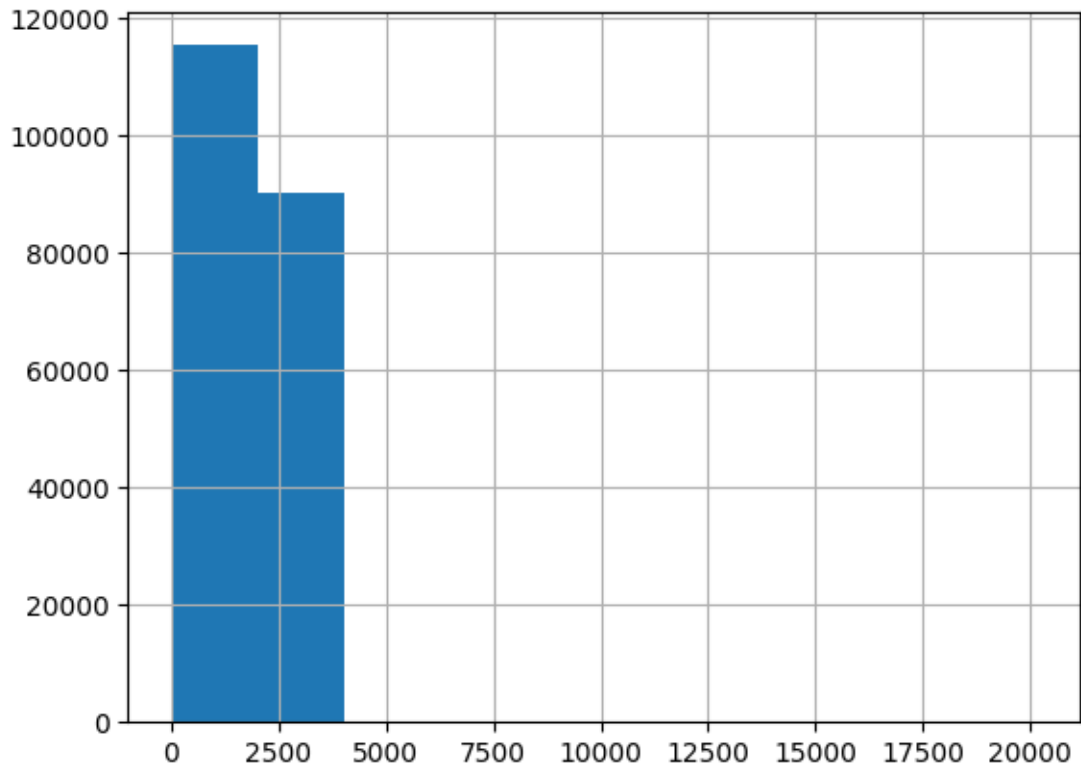
2.2.11 Drop Wrong Value of “orgyear”

```
[371]: len(data)
```

```
[371]: 205772
```

```
[372]: data['orgyear'].hist(bins = 10)
```

```
[372]: <Axes: >
```



```
[373]: display_all(data['orgyear'].value_counts().sort_index())
```

orgyear	
0.0	17
1.0	2
2.0	3
3.0	6
4.0	1
5.0	2
6.0	2
38.0	1
83.0	1
91.0	3
200.0	1
201.0	1
206.0	1
208.0	1
209.0	1
1900.0	1
1970.0	2
1971.0	1
1972.0	1

1973.0	1
1976.0	1
1977.0	1
1979.0	1
1981.0	1
1982.0	4
1984.0	3
1985.0	5
1986.0	8
1987.0	6
1988.0	10
1989.0	22
1990.0	38
1991.0	79
1992.0	47
1993.0	74
1994.0	65
1995.0	94
1996.0	134
1997.0	234
1998.0	279
1999.0	340
2000.0	495
2001.0	713
2002.0	685
2003.0	1018
2004.0	1455
2005.0	1873
2006.0	2074
2007.0	2256
2008.0	2728
2009.0	3777
2010.0	5750
2011.0	7968
2012.0	10491
2013.0	12351
2014.0	16691
2015.0	20607
2016.0	23038
2017.0	23235
2018.0	25256
2019.0	23436
2020.0	13458
2021.0	3675
2022.0	909
2023.0	252
2024.0	43
2025.0	13

```

2026.0      9
2027.0      1
2028.0      4
2029.0      5
2031.0      5
2101.0      1
2106.0      1
2107.0      1
2204.0      1
20165.0     2
Name: count, dtype: int64

```

```
[374]: # considering lowest orgyear = 1970 and highest orgyear = 2024
len(data[~((data["orgyear"] >= 1970) & (data["orgyear"] <= 2024))])
```

```
[374]: 87
```

```
[375]: data = data[((data["orgyear"] >= 1970) & (data["orgyear"] <= 2024))]
data.head()
```

```
[375]:
      company_hash \
0      atrgxmnt xzaxv
1  qtrxvzwt xzegwgb rxbxnta
2      ojzwnvwnxw vx
3      ngpgutaxv
4      qxen sqghu

      email_hash  orgyear \
0  6de0a4417d18ab14334c3f43397fc13b30c35149d70c050c0618caea697c87af  2016.0
1  b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c100a9661a92bdcc0407b  2018.0
2  4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e90fd66c9df6b37b9059  2015.0
3  effdede7a2e7c2af664c8a31d9346385016128d66bbc58a44274d5d6876dfec7  2017.0
4  6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520214b31b95211adb095  2017.0

      ctc      job_position  ctc_updated_year
0  1100000      Other      2020.0
1   449999  FullStack Engineer      2019.0
2  2000000  Backend Engineer      2020.0
3   700000  Backend Engineer      2019.0
4  1400000  FullStack Engineer      2019.0

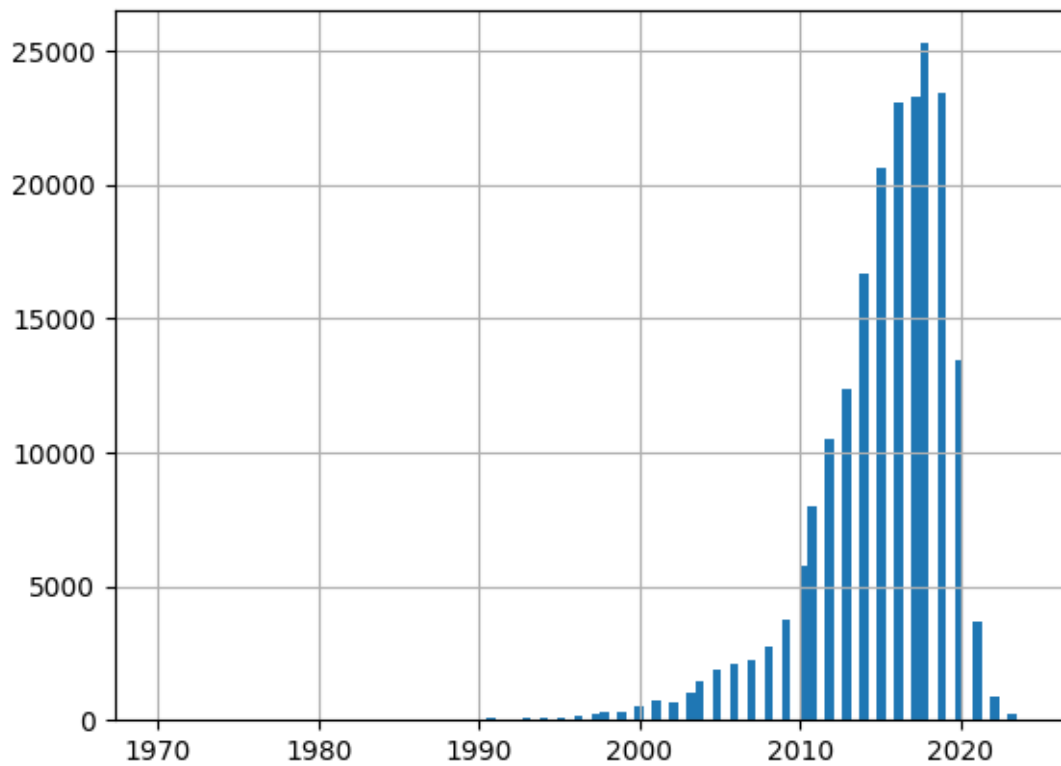
```

```
[376]: len(data)
```

```
[376]: 205685
```

```
[377]: data['orgyear'].hist(bins = 100)
```

[377]: <Axes: >



```
[378]: display_all(data['orgyear'].value_counts().sort_index())
```

orgyear	
1970.0	2
1971.0	1
1972.0	1
1973.0	1
1976.0	1
1977.0	1
1979.0	1
1981.0	1
1982.0	4
1984.0	3
1985.0	5
1986.0	8
1987.0	6
1988.0	10
1989.0	22
1990.0	38
1991.0	79

1992.0	47
1993.0	74
1994.0	65
1995.0	94
1996.0	134
1997.0	234
1998.0	279
1999.0	340
2000.0	495
2001.0	713
2002.0	685
2003.0	1018
2004.0	1455
2005.0	1873
2006.0	2074
2007.0	2256
2008.0	2728
2009.0	3777
2010.0	5750
2011.0	7968
2012.0	10491
2013.0	12351
2014.0	16691
2015.0	20607
2016.0	23038
2017.0	23235
2018.0	25256
2019.0	23436
2020.0	13458
2021.0	3675
2022.0	909
2023.0	252
2024.0	43

Name: count, dtype: int64

2.2.12 Checking the rows which have `ctc_updated_year < orgyear`

```
[379]: len(data[data['ctc_updated_year'] < data['orgyear']])
```

```
[379]: 8790
```

```
[380]: data[data['ctc_updated_year'] < data['orgyear']]['orgyear'].value_counts()
```

```
[380]: orgyear
2020.0    3099
2021.0    1256
2018.0    1164
```

```

2022.0    909
2017.0    820
2016.0    709
2019.0    538
2023.0    252
2024.0     43
Name: count, dtype: int64

```

```
[381]: data[data['ctc_updated_year'] < data['orgyear']]['email_hash'].head()
```

```

[381]: 8      e245da546bf50eba09cb7c9976926bd56557d1ac9a17fb019e8de1fdb83fc0d6
      17      26b502eb6439ac80bd618a6f7c2b1c640b84c1e64c472cf0510b0b36c2d3c247
      40      96b1eb6b3ba24764239d502e3e5d9353af9dbd2059453439b9741526ed038774
      48      7ce201f4f032c2af65b5d11f549de91ea3e62920834e72c20eee306fd240b5c5
      56      3a8a6f0924d0d337eb63d8f6c6b6a3608913b7e4c9535322066c6b3574c2628e
Name: email_hash, dtype: object

```

```
[382]: data[data['email_hash'] ==
↳ 'e245da546bf50eba09cb7c9976926bd56557d1ac9a17fb019e8de1fdb83fc0d6']
```

```

[382]:          company_hash \
8      utqoxontzn ojointbo
156556 utqoxontzn ojointbo

          email_hash \
8      e245da546bf50eba09cb7c9976926bd56557d1ac9a17fb019e8de1fdb83fc0d6
156556 e245da546bf50eba09cb7c9976926bd56557d1ac9a17fb019e8de1fdb83fc0d6

      orgyear      ctc job_position      ctc_updated_year
8      2020.0  450000          NaN      2019.0
156556  2020.0  450000          NaN      2020.0

```

Observation > There 8790 rows which have ctc_updated_year < orgyear. It is not possible to update the ctc without joining the company. So modify the ctc_updated_year.

```
[383]: data['ctc_updated_year'] = data[['ctc_updated_year', 'orgyear']].max(axis=1)
```

```
[384]: data.isna().sum()
```

```

[384]: company_hash      0
      email_hash      0
      orgyear      0
      ctc      0
      job_position    52492
      ctc_updated_year      0
      dtype: int64

```

Observation > There are many null values remained in Job position, So these will be converted

as “unknown” Job Position and Some Regex cleaning also can be applied using preprocess string function below.

2.2.13 Correcting orgyear by maintaining unique orgyear for same person & company combo

```
[385]: a = data.groupby(["email_hash", "company_hash"])["orgyear"].apply(lambda x: x.  
↳nunique())
```

```
[386]: len(a[a>1])
```

```
[386]: 261
```

```
[387]: a[a>1]
```

```
[387]: email_hash                                company_hash  
00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc  ugftqowyggr  
2  
012c5e9ac559c733509bbb9cd3e9d04f8883f4b0f4c623bf08f1c0f4711bee95  xb  v onhatzn  
2  
01dfab5f96789321113c3eb920394751121644947d7f92eae6b7727483eb5577  bgmxrtxqgz  
2  
021ea9c97b6b287336e9345f39f9308c33ccbd15ac366d5f26f60032097ec26c  ouxqg ogrhnxgz  
uqxcvnt rxbxnta      2  
02289b4a3f17a28c90db1c4e2c27f7385367305fd16b87fb37bc718f0e31509b  qtaxeewgb  
xzaxv rna              2  
..  
fbf21d0bb708e3a8cbc99b9bebf6353730e28bd44429b8ebeadeb34fc7155f87  egjq  
2  
fcb6f441a589fcc16f99e95b996d166eeb79f9276e4cebf5d404ca4726c41e1e  buyqd  
2  
fcf87e40de5c05cc449344553f03f65a493f38be62516dc82f690c0df4a5431c  ywr  
ntwyzgrgsxto              2  
fd16d99d5e78aa98b62e10abe4547d7ef8bad305c1a497b11cfbfe860160458d  wvqtonvwp  
2  
fd59193f44b2df0b451db26ad0a2a3e4297c46837e42431ac8d444cc4ba9e2bf  ytva axsxnv  
fgqpo              2  
Name: orgyear, Length: 261, dtype: int64
```

```
[388]: data[data["email_hash"] ==  
↳"00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc"]
```

```
[388]:      company_hash \  
36843    ugftqowyggr  
144759    ugftqowyggr  
180747    ugftqowyggr  
184639    ugftqowyggr
```

	email_hash \
36843	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc
144759	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc
180747	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc
184639	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc

	orgyear	ctc	job_position	ctc_updated_year
36843	2017.0	600000	QA Engineer	2020.0
144759	2017.0	600000	SDET	2020.0
180747	2021.0	750000	QA Engineer	2021.0
184639	2021.0	750000	SDET	2021.0

```
[389]: data["orgyear"] = data.groupby(["email_hash", "company_hash"])["orgyear"].
      ↪transform("min")
```

```
[390]: data[data["email_hash"] ==
      ↪"00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc"]
```

```
[390]:      company_hash \
36843    ugftqowyggr
144759    ugftqowyggr
180747    ugftqowyggr
184639    ugftqowyggr
```

	email_hash \
36843	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc
144759	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc
180747	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc
184639	00a1a8c6f324f6a768bbd5ee566c6f598c9fd548019af6d7742ac8b173cba0fc

	orgyear	ctc	job_position	ctc_updated_year
36843	2017.0	600000	QA Engineer	2020.0
144759	2017.0	600000	SDET	2020.0
180747	2017.0	750000	QA Engineer	2021.0
184639	2017.0	750000	SDET	2021.0

```
[391]: data.isna().sum()
```

```
[391]: company_hash      0
email_hash              0
orgyear                 0
ctc                     0
job_position            52492
ctc_updated_year        0
dtype: int64
```

2.2.14 Filling the job_position null values with “mode_value” of the group or “unknown” (If there is no mode)

```
[392]: def fill_job_position(group):
        non_nan_positions = group['job_position'].dropna()
        if not non_nan_positions.empty: # Check if there are non-NaN values
            mode_value = non_nan_positions.mode().iloc[0] # Get the mode of
            ↪non-NaN values
            return group['job_position'].fillna(mode_value)
        else:
            return group['job_position'].fillna("unknown")

data['job_position'] = data.groupby(["email_hash", "company_hash", "orgyear",
            ↪"ctc_updated_year"], group_keys=False).apply(fill_job_position)
```

```
[393]: data.isna().sum()
```

```
[393]: company_hash      0
email_hash            0
orgyear              0
ctc                  0
job_position          0
ctc_updated_year     0
dtype: int64
```

Observation > All null values are handled properly.

2.3 DATA CLEANING USING REGEX

```
[394]: data["job_position"].value_counts()
```

```
[394]: job_position
Backend Engineer      52219
FullStack Engineer    30366
unknown               24897
Other                 21742
Frontend Engineer     12549
...
SDET 3                1
senior software engineer-L2  1
Big data Developer    1
Associate Technology  1
Android Application developer  1
Name: count, Length: 1017, dtype: int64
```

Observation > Unknown job_position is in 3rd position. Backend Engineer is in 1st position.

2.3.1 Identifying and Removing the Special characters from Dataset using REGEX

Identifying the unwanted charaters in the string features

```
[395]: # Define regex for unwanted characters
# All the characters except A to Z, a to z, 0 to 9 and space are treated as
↳ unwanted characters
regex = r'[^A-Za-z0-9 ]+'

# Loop through string columns to identify unwanted characters
for column in data.select_dtypes(include='object').columns:
    # Find rows with unwanted characters in the current column
    unwanted_values = data[column][data[column].str.contains(regex, na=False)]

    # If there are unwanted characters, print the column name and the values
    if not unwanted_values.empty:
        print(f"Unwanted characters found in column: {column}")
        print()
        print(np.unique(unwanted_values))
        print()
    else:
        print(f"Unwanted characters found in column: {column}")
        print()
        print(None)
        print()
```

Unwanted characters found in column: company_hash

None

Unwanted characters found in column: email_hash

None

Unwanted characters found in column: job_position

```
['.' '...' '.7' 'Advisory Consultant - UI/UX Expert'
'Analyst / Software Engineer' 'Application Developer (Frontend)'
'Asp.Net developer' 'Assistant System Engineer - Trainee'
'Assistant manager @ capital market '
'Associate Director, Online Products'
'Associate System Engineer - Trainee' 'Associate System Engineer-Trainee'
'Associate consultant - Developer' 'Associate- Futures Engineering Team'
'Asst. Manager' 'Business Analyst / Product Owner' "Can't reveal"
'Co founder/CTO' 'Co-Founder' 'Co-founder' 'Co-founderg'
'Core Tech Engineer / Researcher' 'Data/Product Engineer'
'DevOps/Cloud Engineer' 'Developer & Production Support'
'Engineering Lead - AI' 'F&b service' "Founders' Office"
'Front-End Developer' 'Full-stack web developer'
'Info Technology Sr. Associate' 'Intern - Software developer'
```

'Intern-data analyst' 'Jr. Prod Specialist' 'Jr. Software Engineer'
 'Junior Associate, Technology' 'Lead - MIS'
 'Lead Engineer - Software Development' 'MTS - II' 'MTS-2' 'MTS-3'
 'Manager-CX' 'Member Technical Staff -1'
 'Member of Technical Staff (JAVA)' 'Member of Technical Staff (MTS)'
 'Member of Technical Staff - 2'
 'Mobile Application Engineer(Android & iOS)'
 'Now iam not working, waiting for job' 'PhD. Student'
 'Principal Engineer - 1' 'Product Engineer (Backend)' 'QA/QC Engineer'
 'QAE-Intern' 'R & D Engineer-2' 'R & D engineer' 'R&D Engineer'
 'Research Assistant / Software Engineer'
 'Research Intern - Machine Learning' 'S/w Engineer 2' 'SDE - 3 '
 'SDE - II' 'SDE, Lead' 'SDE-1' 'SDE-2' 'SDE-3' 'SDE-I' 'SDE-II'
 'SDET - Intern' 'SDET-2' 'SE(R&D)' 'SQL, PLSQL Developer' 'Scientist - C'
 'Senior Associate - (Index Technology - Production Support)'
 'Senior Engineer AI/ML' 'Senior Manager, Online Products'
 'Senior Member Of Technical Staff - R&D - Machine Learning'
 'Senior Mobile Applications Developer (Android/iOS)'
 'Senior Product Development Engineer (Senior Developer)'
 'Senior Software Development Engineer (Backend)'
 'Senior Software Engineer (.Net Backend)'
 'Senior Software Engineer (Backend)'
 'Senior Software Engineer (front end)'
 'Senior Software Engineer - Infrastructure'
 'Senior Software Engineer, QA' 'Snr Analyst Programmer/Java Developer'
 'Software / web developer' 'Software Dev. Engineer'
 'Software Developer (SDE-3)' 'Software Developer - Android'
 'Software Developer - PL10' 'Software Developer - iOS'
 'Software Developer grade-1' 'Software Development Engineer - 2'
 'Software Development Engineer - I' 'Software Development Engineer - II'
 'Software Development Engineer - III'
 'Software Development Engineer - Intern'
 'Software Development Engineer -2 ' 'Software Development Engineer ||'
 'Software Engineer - Programmer Analyst' 'Software Engineer (Android)'
 'Software Engineer (Backend)' 'Software Engineer (Full Stack)'
 'Software Engineer (Full stack)' 'Software Engineer (Intern)'
 'Software Engineer - Frontend' 'Software Engineer - I'
 'Software Engineer - II' 'Software Engineer - Operations'
 'Software Engineer - Testing' 'Software Engineer 2 (Backend)'
 'Software Engineer 2 (Full Stack)' 'Software Engineer R&D'
 'Software Engineer Trainee(ML)' 'Software Engineer(Advanced Analytics)'
 'Software Engineer(R&D) Intern ' 'Software Engineer, Associate'
 'Software Engineering - Engineer 2' 'Software Engineering Co-Op'
 'Software Prod & Plat Eng Analyst' 'Software developer (Data engineer)'
 'Software developer (automation)'
 'Software developer/Application Developer' 'Software engineer - 2'
 'Software engineer III (sde2) ' 'Solution Architect - Apprentice'
 "Some data entry operator like some copy's write.type and upload"

```
'Specialist Programmer (PP)' 'Sr Software Engg - Android' 'Sr.'
'Sr. Associate Platform L2' 'Sr. EXECUTIVE ' 'Sr. MIS Executive'
'Sr. Operation Executive' 'Sr. Product Engineer' 'Sr. Software Engineer'
'Sr. Software Engineer(Android)' 'Sr. Technical Architect'
'Sr. iOS Engineer' 'Sr. iOS developer' 'Sr. technical manager'
'Staff engineer/mgr' 'Sub-Inspector'
'Summer Technology Analyst (Software Developer Intern)'
'System Engineer,FullStack Developer' 'Team Lead - Android'
'Tech lead /Team Lead ' 'Test Technician, Specialist'
'Tool room Engineer, Mechanical Engineer '
'Trainee Engineer (Java Developer)' 'Trainee(Intern)' 'Web / UI Designer'
'Web Designer/Developer Intern' 'Web Developer (Full Stack)'
'iOS / Swift Developer' 'senior software engineer-L2'
'software developer - UI' 'sr. developer']
```

Observation > Only Job_position has unwanted characters. (unwanted characters are all the characters except A to Z, a to z, 0 to 9, space” ”)

Same job position but different names are present if we observe carefully

For eg: Senior Software Engineer, Sr. Software Engineer.

It is difficult to clean this type of data without NLP techniques. Right now we are not using NLP Techniques.

Basic cleaning of this feature can be applied by REGEX 1. All unwanted characters will be empty string 2. Sr -> Senior 3. Jr -> Junior 4. Asst. -> Assistant

So many other abbreviations can be corrected. But For now, we are ignoring them.

2.3.2 Removing the unwanted characters in job_position feature using Regex and also applying basic cleaning (no NLP)

```
[396]: def preprocess_string(string):
        new_string= re.sub('[^A-Za-z0-9 ]+', '', string).lower().strip()
        new_string = re.sub('sr',"senior",new_string)
        new_string = re.sub('jr',"senior",new_string)
        new_string = re.sub('asst',"assistant",new_string)
        return new_string

mystring='\tSr. Software Engineer\\\\\\\\&&**() (Android)'
preprocess_string(mystring)
```

```
[396]: 'senior software engineer android'
```

```
[397]: data.job_position.nunique()
```

```
[397]: 1017
```

```
[398]: data.job_position=data.job_position.apply(lambda x: preprocess_string(str(x)))
data.job_position.nunique()
```

```
[398]: 895
```

```
[399]: data[data["job_position"] == ""]
```

```
[399]:
```

	company_hash \	email_hash \	orgyear	ctc	job_position	ctc_updated_year
5796	seo gxr svo axcxoxgz		2011.0	700000		2021.0
139142	otre tburgjta		2019.0	600000		2021.0
146638	otre tburgjta		2019.0	600000		2021.0

```
[400]: data[data["job_position"]=="null"]
```

```
[400]:
```

	company_hash \	email_hash \	orgyear	ctc	job_position	ctc_updated_year
28156	zgn vuurxwvmrt vwwghzn		2019.0	200000	null	2021.0
30972	zgn vuurxwvmrt vwwghzn		2019.0	200000	null	2021.0

```
[401]: data['job_position'].replace('', 'unknown', inplace=True)
data['job_position'].replace('null', 'unknown', inplace=True)
```

```
[402]: data[data["job_position"] == ""]
```

```
[402]: Empty DataFrame
Columns: [company_hash, email_hash, orgyear, ctc, job_position,
ctc_updated_year]
Index: []
```

Observation > Some of the job postions decreased because of regex cleaning

```
[403]: data.shape
```

```
[403]: (205685, 6)
```

```
[404]: data.duplicated().sum()
```

```
[404]: 27396
```

```
[405]: data.drop_duplicates(inplace=True)
data.shape
```

```
[405]: (178289, 6)
```

```
[406]: data.duplicated().sum()
```

```
[406]: 0
```

Observation > Number of duplicates may increase by cleaning the job position feature by using regex. (We are not applying NLP techniques now)

2.4 HANDLING MULTIPLE JOB POSITIONS PROBLEM

```
[407]: data["email_hash"].value_counts()
```

```
[407]: email_hash
3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378    9
6842660273f70e9aa239026ba33bfe82275d6ab0d20124021b952b5bc3d07e6c    9
c0eb129061675da412b0deb15871dd06ef0d7cd86eb5f7e8cc6a20b0d1938183    8
bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b    8
298528ce3160cc761e4dc37a07337ee2e0589df251d73645aae209b010210eee    8
..
1bd7b5d6541ace89f145068a7ba1988468d52f0f5f3dc555e5c7ed8c04066a7b    1
0c5281f204102cd5d53e655e202ae924afe95050088bd80dbb98a8d1b7ed0e44    1
c0b0d3335b26356d97cea57ee38c265d352a4d5a6cca410cf85cc05d3e2cff86    1
46fd2ff0349f678f79f178290d94e30f139c3a4fd4ef4c44ce79a3e10a155f3e    1
0bcfc1d05f2e8dc4147743a1313aa70a119b41b30d4a1f7e738a6a87d3712c31    1
Name: count, Length: 153343, dtype: int64
```

```
[408]: data[data["email_hash"] == "3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378"]
```

```
[408]:          company_hash \
10214    wgcxvb ntwyzgrgsxto
10734    wgcxvb ntwyzgrgsxto
16651    wgcxvb ntwyzgrgsxto
18165    wgcxvb ntwyzgrgsxto
20794    wgcxvb ntwyzgrgsxto
```



```

137990  wgcxvb ntwyzgrgsxto
164929  wgcxvb ntwyzgrgsxto
168354  wgcxvb ntwyzgrgsxto
172176  wgcxvb ntwyzgrgsxto

```

```

                                                    email_hash \
10214    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
10734    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
16651    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
18165    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
20794    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
137990    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
164929    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
168354    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378
172176    3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378

```

	orgyear	ctc	job_position	ctc_updated_year
10214	2018.0	1130000	engineering intern	2021.0
10734	2018.0	1130000	other	2021.0
16651	2018.0	1130000	backend engineer	2021.0
18165	2018.0	1130000	data scientist	2021.0
20794	2018.0	1130000	data analyst	2021.0
137990	2018.0	1085000	data scientist	2020.0
164929	2018.0	1085000	data analyst	2020.0
168354	2018.0	1085000	backend engineer	2020.0
172176	2018.0	1085000	engineering intern	2020.0

```

[409]: data[data["email_hash"] == "3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378"]["job_position"].
        mode()

```

```

[409]: 0    backend engineer
      1    data analyst
      2    data scientist
      3    engineering intern
      Name: job_position, dtype: object

```

Observation > Same person(email_hash) has mulitple job positions, But remaining all features are unique.

This is practical not possible. So Take mode value of job position for each group.

```

[410]: data["job_position"].value_counts()

```

```

[410]: job_position
      backend engineer    43701
      fullstack engineer    25956
      unknown            24830

```

```

other                                18097
frontend engineer                    10440
...
senior engineer software              1
software engineering specialist        1
android lead                          1
senior analysts                      1
azure data factory                    1
Name: count, Length: 893, dtype: int64

```

```

[411]: # # Define priority order based on job position frequency
# priority_order = scaler['job_position'].value_counts().index.tolist()

# def choose_priority_mode(modes):
#     # If there's only one mode, return it directly
#     if len(modes) == 1:
#         return modes.iloc[0]

#     # Otherwise, choose based on priority
#     for job in priority_order:
#         if job in modes:
#             return job
#     return modes.iloc[0] # Fallback in case no priority match

# scaler = (
#     scaler.groupby(['email_hash', 'company_hash', 'ctc', 'ctc_updated_year'])
#     .apply(lambda x: x.assign(
#         job_position=choose_priority_mode(x['job_position'].mode())
#     ))
#     .drop_duplicates(subset=['email_hash', 'company_hash', 'ctc', 'ctc_updated_year'])
#     .reset_index(drop=True)
# )
# Taking too much time to execute

```

2.4.1 Replacing the multiple job positions with mode value or based on value counts priority order

```

[412]: # Step 1: Calculate priority order from most frequent job positions
priority_order = data['job_position'].value_counts().index.tolist()
priority_map = {job: i for i, job in enumerate(priority_order)}

# Step 2: Find modes and assign priorities
job_mode_df = (
    data.groupby(['email_hash', 'company_hash', 'ctc', 'ctc_updated_year'])['job_position']
    .agg(lambda x: x.mode().tolist()) # Get mode(s) as a list

```

```

        .reset_index()
    )

    # Step 3: Assign priority to each mode list and select the highest-priority
    ↪ position
    job_mode_df['job_position'] = job_mode_df['job_position'].apply(
        lambda modes: modes[0] if len(modes) == 1 else min(modes, key=lambda job:
    ↪ priority_map.get(job, float('inf')))
    )

    # Step 4: Merge the prioritized positions back into the original DataFrame
    data= data.drop(columns=['job_position']).merge(job_mode_df, on=['email_hash',
    ↪ 'company_hash', 'ctc', 'ctc_updated_year'])

```

```
[413]: data["job_position"].value_counts()
```

```
[413]: job_position
backend engineer          50587
fullstack engineer       25089
unknown                  24830
other                    17828
frontend engineer        9744
...
pat                       1
leisure manager          1
front end engineer        1
software web developer     1
analyst software engineer  1
Name: count, Length: 620, dtype: int64
```

Observation > We can observe that value counts have modified before and after. backend engineer value count was increased.

```
[414]: data["job_position"].nunique()
```

```
[414]: 620
```

```
[415]: data.duplicated().sum()
```

```
[415]: 11707
```

```
[416]: data.drop_duplicates(inplace=True)
data.shape
```

```
[416]: (166582, 6)
```

2.5 REPLACE ORGYEAR & CTC_UPDATED_YEAR WITH ITS MINIMUM VALUE FOR SAME EMAIL, SAME COMPANY, SAME CTC, SAME JOB POSITION

```
[417]: data = (
        data.groupby(['email_hash', 'company_hash', 'ctc', 'job_position'])
        .agg({
            'orgyear': 'min',
            'ctc_updated_year': 'min'
        })
        .reset_index()
    )
```

```
[418]: data["email_hash"].value_counts()
```

```
[418]: email_hash
db84980ad197f8eff08b14a3442ff57f6374ea780f2587b310aac54b6c32ee3a    3
80302c80b1c62765a93a98d0228cad4b3e0a57bf4d6e3a01c75d06fa9fa74cea    2
cc0fad02b3a8d0a35666c57f13e6f4f68eb8becbe78c2057015c028df11f39db    2
38bfe24091475110021b573d22c04878d6a97bb74d06c4e379dd9d1d8341492e    2
cbf6b0fa61a1ac23942e085ac364c5416eb1e47ea288b921b0f490fbf6b26f21    2
..
59216d0595d84ec308c8ab6107a8e071af0aaa11bff208ae4a6f7734758a1992    1
5920eeb4a9faf7bc94c21ee02c3750e820f5042a3143aac2cc8644305a30a4e6    1
5920516f4576e1aad29295b27a26d70f0b602f9c299fa9852288a309d7b2c0d76    1
591fffc0bf54dccac8480cccbe81b2c76b0dc2b44d2d836215d89aab6d51e727    1
fffffa3eb3575f43b86d986911463dce7bcadcea227e5a4b0f1756744efdb7477    1
Name: count, Length: 153343, dtype: int64
```

```
[419]: data.duplicated().sum()
```

```
[419]: 0
```

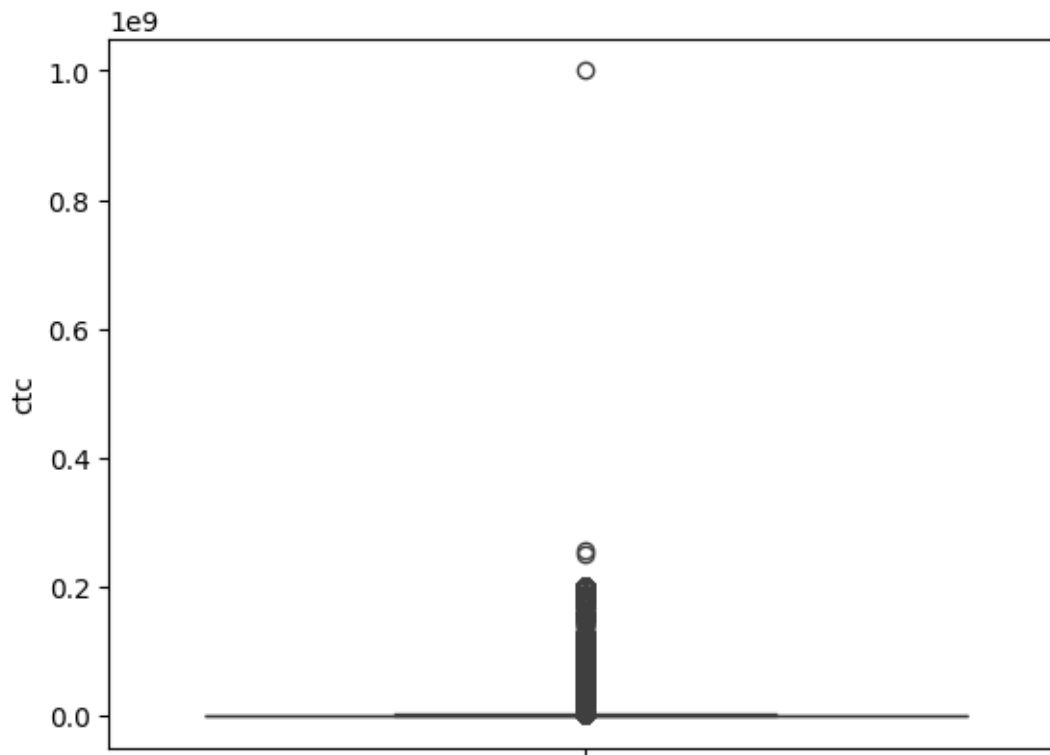
```
[420]: data.isna().sum()
```

```
[420]: email_hash    0
company_hash    0
ctc              0
job_position    0
orgyear         0
ctc_updated_year 0
dtype: int64
```

2.6 OUTLIER DETECTION

```
[421]: sns.boxplot(y="ctc" , data= data)
```

```
[421]: <Axes: ylabel='ctc'>
```



```
[422]: for i in [0.9,0.99,0.999,0.9999,0.99999,0.999999]:  
        print(f"Quantile {i} of ctc is {data['ctc'].quantile(i)}")
```

```
Quantile 0.9 of ctc is 2900000.0  
Quantile 0.99 of ctc is 19998840.000000026  
Quantile 0.999 of ctc is 200000000.0  
Quantile 0.9999 of ctc is 200000000.0  
Quantile 0.99999 of ctc is 251885666.47812158  
Quantile 0.999999 of ctc is 876504135.6503121
```

```
[423]: a = np.percentile(data["ctc"] , 25)  
b = np.percentile(data["ctc"] , 50)  
c = np.percentile(data["ctc"] , 75)  
  
iqr = c-a  
  
lower = max((a-1.5*iqr) , 0)  
upper = (c + 1.5*iqr)  
  
outliers = data[(data["ctc"] < lower) | (data["ctc"] > upper)]
```

```

no_outliers = data[(data["ctc"] >= lower) & (data["ctc"] <= upper)]

print("outliers =" ,len(outliers))
print("no_outliers =" ,len(no_outliers))
print()
print("percent_outliers =" ,(len(outliers)/len(data))*100)

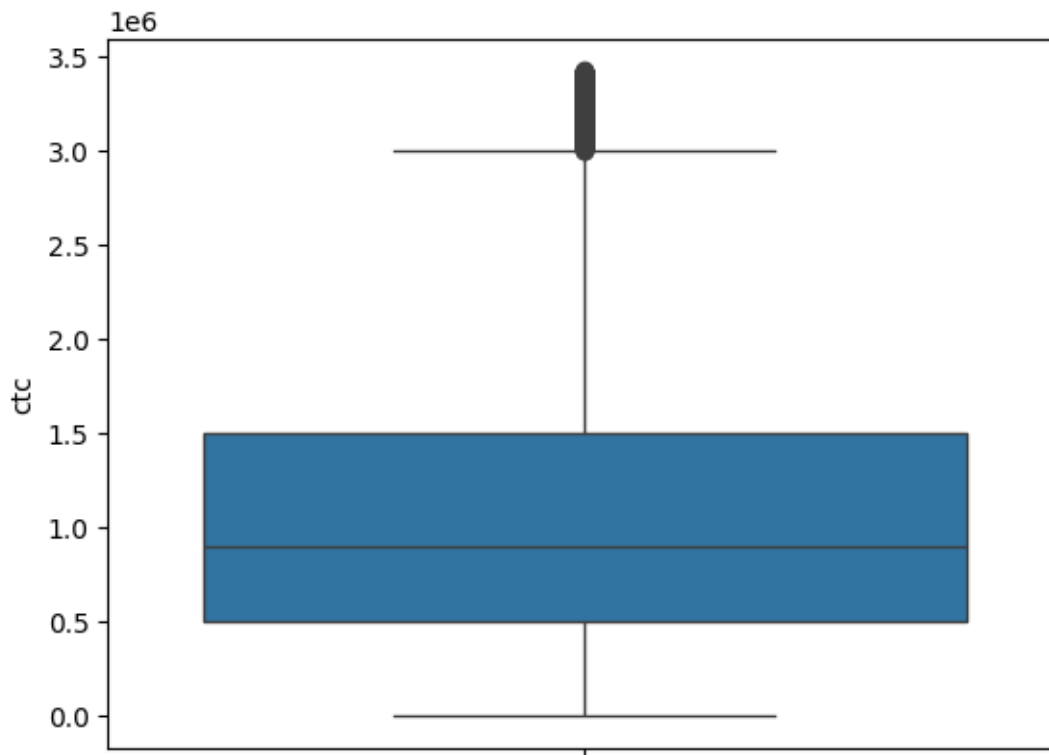
sns.boxplot(data = no_outliers , y = "ctc")
plt.show()

```

outliers = 11146

no_outliers = 154913

percent_outliers = 6.712072215296973



Observation > Not removing the outliers because some persons may get very good salary or hike because of various reasons like skill updation, hard work etc.,. Treating them as valid outliers.

But should provide 'is_outlier' tags using anomaly detection algorithms like LOF, Isolation forest etc.,

2.7 FEATURE ENGINEERING

2.7.1 Total_Years_of_experience (considering 2024 as present year)

```
[424]: # Current year for experience calculation
current_year = 2024

# Calculate total years of experience by finding the minimum orgyear per
↳email_hash
data["total_experience"] = data.groupby('email_hash')['orgyear'].
↳transform(lambda x: current_year - x.min())
```

2.7.2 Hike

```
[425]: # Create lagged CTC for overall hike calculation
data['previous_ctc_all'] = data.groupby('email_hash')['ctc'].shift(1)

# Calculate Overall Hike
data['Hike'] = data['ctc'] - data['previous_ctc_all']
data['Hike'].fillna(0, inplace=True) # Fill NaN values with 0 for the first
↳record

# Drop previous_ctc_all
data = data.drop(["previous_ctc_all"],axis = 1)
```

2.7.3 Hike From Same Company

```
[426]: # Sort by email_hash and ctc_updated_year
data.sort_values(by=['email_hash', 'ctc_updated_year'], inplace=True)

# Group by email_hash and company_hash, then use shift to create lagged CTC
data['previous_ctc'] = data.groupby(['email_hash', 'company_hash'])['ctc'].
↳shift(1)

# Calculate ctc_changed
data['Hike_From_Same_Company'] = data['ctc'] - data['previous_ctc']

# If there's no previous CTC, set ctc_changed to 0
data['Hike_From_Same_Company'].fillna(0, inplace=True)

# Remove previous_ctc
data = data.drop(["previous_ctc"],axis = 1)
```

```
[427]: data[(data["Hike"]==0)&(data["Hike_From_Same_Company"]==0)]["email_hash"].
↳nunique()
```

```
[427]: 151733
```

Observation > There are 151733 emails with no Hike

```
[428]: data[(data["Hike"]>0)]["email_hash"].nunique()
```

[428]: 7768

```
[429]: data[(data["Hike"]<0)]["email_hash"].nunique()
```

[429]: 2541

Observation > There are 7768 emails with some Hike

There are 2541 emails with demotion in CTC.

```
[430]: data[(data["Hike"]>0)&(data["Hike_From_Same_Company"]>0)]["email_hash"].  
        ↪nunique()
```

[430]: 3778

```
[431]: data[(data["Hike"]<0)&(data["Hike_From_Same_Company"]<0)]["email_hash"].  
        ↪nunique()
```

[431]: 0

Observation > There are 3778 emails who has got Hike From same company

There are 0 emails who has got demotion in Hike from Same company (Negative values)

```
[432]: data[(data["Hike"]>0)&(data["Hike_From_Same_Company"]==0)]["email_hash"].  
        ↪nunique()
```

[432]: 3990

```
[433]: data[(data["Hike"]<0)&(data["Hike_From_Same_Company"]==0)]["email_hash"].  
        ↪nunique()
```

[433]: 2541

Observation > There are 3990 emails who has got Hike due to change in company

There are 2541 emails who has demoted and joined in another company

```
[434]: data[(data["Hike"]==0)&(data["Hike_From_Same_Company"]>0)]["email_hash"].  
        ↪nunique()
```

[434]: 0

Observation > Cross checking: there should be no email where Hike == 0 But Hike From Same company > 0.

2.7.4 Creating “Is_Job_Position_Changed” & “Is_CTC_Changed” Features

```
[435]: # Sort by email_hash, orgyear, and ctc_updated_year for ordered comparison
data.sort_values(by=['email_hash', 'orgyear', 'ctc_updated_year'], inplace=True)

# Calculate `Is_Job_Position_Changed`
# Group by email_hash and check if job_position has changed from previous row
data['previous_job_position'] = data.groupby('email_hash')['job_position'].
    ↪shift(1)
data['Is_Job_Position_Changed'] = data.apply(
    lambda row: 0 if pd.isna(row['previous_job_position']) or
    ↪row['job_position'] == row['previous_job_position'] else 1,
    axis=1
)

# Calculate `Is_CTC_Changed`
# Group by email_hash and check if ctc has changed from previous row
data['previous_ctc'] = data.groupby('email_hash')['ctc'].shift(1)
data['Is_CTC_Changed'] = data.apply(
    lambda row: 0 if pd.isna(row['previous_ctc']) else (1 if row['ctc'] >
    ↪row['previous_ctc'] else -1 if row['ctc'] < row['previous_ctc'] else 0),
    axis=1
)

# Drop temporary columns for clarity
data.drop(columns=['previous_job_position', 'previous_ctc'], inplace=True)
```

2.7.5 Working Simultaneously in Multiple Companies (Same year multiple companies)

```
[436]: # Function to calculate number of unique companies per (email_hash, orgyear)
    ↪group
def count_companies_in_same_year(group):
    # Count unique companies per orgyear within each email_hash
    group['Working_in_multiple_companies'] = group.
    ↪groupby(['orgyear'])['company_hash'].transform('nunique')
    return group

# Apply the function to each email_hash group
data = data.groupby('email_hash').apply(count_companies_in_same_year).
    ↪reset_index(drop=True)
```

```
[437]: data["Working_in_multiple_companies"].unique()
```

```
[437]: array([1, 2, 3], dtype=int64)
```

```
[438]: data["Working_in_multiple_companies"].value_counts()
```

```
[438]: Working_in_multiple_companies
      1    153094
      2     12962
      3         3
      Name: count, dtype: int64
```

```
[439]: data[data["Working_in_multiple_companies"] == 3]["email_hash"].nunique()
```

```
[439]: 1
```

```
[440]: data[data["Working_in_multiple_companies"] == 2]["email_hash"].nunique()
```

```
[440]: 6481
```

```
[441]: len(data[data["Working_in_multiple_companies"] == 1])
```

```
[441]: 153094
```

```
[442]: data[data["Working_in_multiple_companies"] == 1]["email_hash"].nunique()
```

```
[442]: 146861
```

```
[443]: 153094-146861
```

```
[443]: 6233
```

Observation > There is only email , who works simultaneously in 3 companies

There are 6481 emails, who works simultaneously in 2 companies. So Total $2 \times 6481 = 12962$ Rows.

But there are 146861 emails who works in one company at a time. But the number of rows with “Working_in_multiple_companies” == 1 are 153094 rows.

This is Because 6233 emails, works in different job_positions or different ctc but one company at a time.

2.7.6 Working Tenure in a company

```
[444]: # Current year for tenure calculation
      current_year = 2024

      def calculate_tenure(group):
          # Check if orgyear has only one unique year across all companies
          if group['orgyear'].nunique() == 1:
              tenure = current_year - group['orgyear'].iloc[0]
              group['tenure'] = tenure
          else:
```

```

        # Calculate tenure normally based on the difference with the next
        ↪orgyear or the current year
        sorted_orgyears = sorted(group['orgyear'].unique())
        tenure_mapping = {}

        # For each orgyear, calculate tenure to the next unique orgyear or to
        ↪the current year
        for i, orgyear in enumerate(sorted_orgyears):
            if i < len(sorted_orgyears) - 1:
                next_orgyear = sorted_orgyears[i + 1]
                tenure_mapping[orgyear] = next_orgyear - orgyear
            else:
                tenure_mapping[orgyear] = current_year - orgyear

        group['tenure'] = group['orgyear'].map(tenure_mapping)

    return group

# Apply the function grouped by email_hash
data= data.groupby('email_hash').apply(calculate_tenure).reset_index(drop=True)

```

```
[445]: data.isna().sum()
```

```

[445]: email_hash          0
       company_hash      0
       ctc               0
       job_position      0
       orgyear           0
       ctc_updated_year  0
       total_experience   0
       Hike              0
       Hike_From_Same_Company  0
       Is_Job_Position_Changed  0
       Is_CTC_Changed    0
       Working_in_multiple_companies  0
       tenure            0
       dtype: int64

```

```
[446]: data['job_position'] = data['job_position'].str.strip()
```

```
[447]: data['job_position'].fillna('unknown', inplace=True) # Example
```

```

[448]: data[data["email_hash"] ==
        ↪"00e5dc302f374d63a0436b95aa98eef22e9cadade3e5cb6da3f3a92b2e1ca4c8"]

```

```

[448]:                                     email_hash \
576   00e5dc302f374d63a0436b95aa98eef22e9cadade3e5cb6da3f3a92b2e1ca4c8

```

```

577 00e5dc302f374d63a0436b95aa98eef22e9cadade3e5cb6da3f3a92b2e1ca4c8

      company_hash      ctc      job_position  orgyear  \
576 uurej vzvrjnxwt ucn rna    940000  fullstack engineer    2016.0
577      zgn ohqt    1100000      unknown    2017.0

      ctc_updated_year  total_experience      Hike  Hike_From_Same_Company  \
576          2020.0          8.0          0.0          0.0
577          2021.0          8.0    160000.0          0.0

      Is_Job_Position_Changed  Is_CTC_Changed  Working_in_multiple_companies  \
576              0              0              1
577              1              1              1

      tenure
576      1.0
577      7.0

```

2.7.7 Saving Final Cleaned “Scaler_Clean” Dataframe into csv for Future use

```

[449]: # Save the DataFrame as a CSV file
data.to_csv("data_cleaned_data.csv", index=False)

```

3 CHAPTER 3: DATA VISUALISATION

```

[450]: data = pd.read_csv("data_cleaned_data.csv")

```

```

[451]: data.head()

```

```

[451]:                                     email_hash  \
0  00003288036a44374976948c327f246fdbdf077854690475243a9c91d3b1cf9f
1  0000aaa0e6b61f7636af1954b43d294484cd151c9b3cf6a761391159aa77f3fa
2  0000d58fbc18012bf6fa2605a7b0357d126ee69bc41032ace6b6223c48468887
3  000120d0c8aa304fcf12ab4b85e21feb80a342cfea03d461a50a3e16626a4177
4  00014d71a389170e668ba96ae8e1f9d991591acc8990253cc62a371eea92155b

      company_hash      ctc      job_position  orgyear  \
0      bxwqgogen    3500000      backend engineer    2012.0
1      nqsn axsnvr     250000      backend engineer    2013.0
2      gunhb    1300000  fullstack engineer    2021.0
3  bxwqgotbx wgqugqvnxgz    2000000  fullstack engineer    2004.0
4      fvrbvqn rvmo    3400000      unknown    2009.0

      ctc_updated_year  total_experience  Hike  Hike_From_Same_Company  \
0          2019.0          12.0          0.0          0.0
1          2020.0          11.0          0.0          0.0

```

2	2021.0	3.0	0.0	0.0
3	2021.0	20.0	0.0	0.0
4	2018.0	15.0	0.0	0.0

	Is_Job_Position_Changed	Is_CTC_Changed	Working_in_multiple_companies	\
0	0	0	1	
1	0	0	1	
2	0	0	1	
3	0	0	1	
4	0	0	1	

	tenure
0	12.0
1	11.0
2	3.0
3	20.0
4	15.0

```
[452]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 166059 entries, 0 to 166058
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   email_hash                            166059 non-null object
1   company_hash                          166059 non-null object
2   ctc                                    166059 non-null int64
3   job_position                          166059 non-null object
4   orgyear                              166059 non-null float64
5   ctc_updated_year                      166059 non-null float64
6   total_experience                       166059 non-null float64
7   Hike                                  166059 non-null float64
8   Hike_From_Same_Company                166059 non-null float64
9   Is_Job_Position_Changed               166059 non-null int64
10  Is_CTC_Changed                        166059 non-null int64
11  Working_in_multiple_companies          166059 non-null int64
12  tenure                                166059 non-null float64
dtypes: float64(6), int64(4), object(3)
memory usage: 16.5+ MB
```

```
[453]: data.isna().sum()
```

```
[453]: email_hash          0
       company_hash       0
       ctc                 0
       job_position        0
```

```

orgyear          0
ctc_updated_year 0
total_experience  0
Hike             0
Hike_From_Same_Company 0
Is_Job_Position_Changed 0
Is_CTC_Changed   0
Working_in_multiple_companies 0
tenure           0
dtype: int64

```

```
[454]: data.shape
```

```
[454]: (166059, 13)
```

3.1 UNIVARIATE ANALYSIS

```
[455]: cont_cols = ['ctc','total_experience','Hike','Hike_From_Same_Company','tenure']

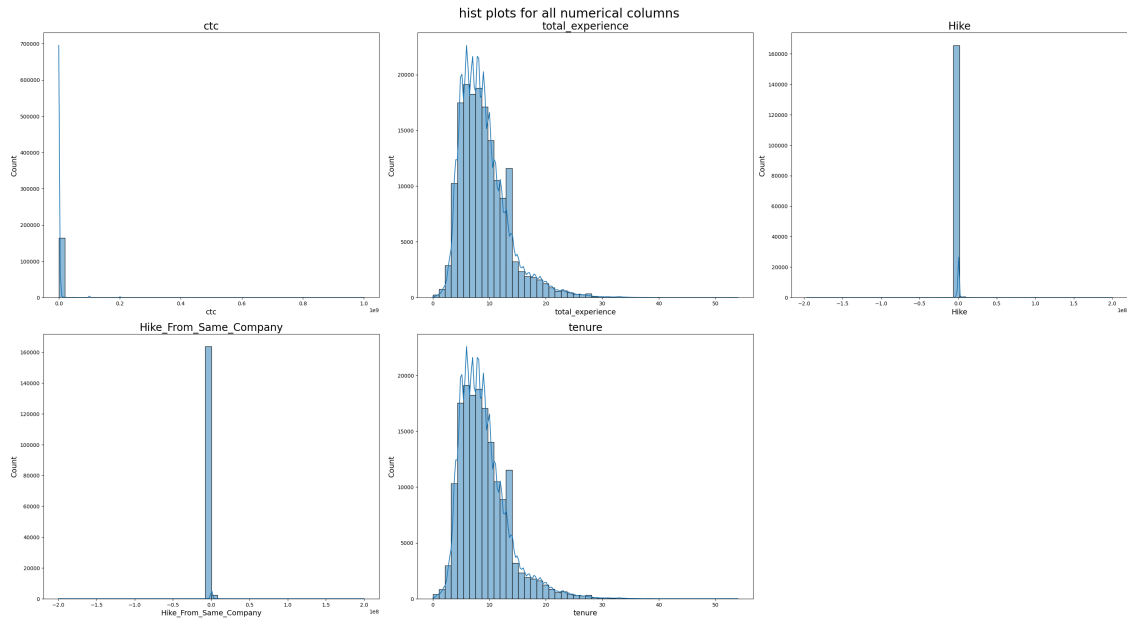
cat_cols = [
    ↪['email_hash','company_hash','job_position','orgyear','ctc_updated_year','Is_Job_Position_C
```

3.1.1 Distribution plots of all Numerical Columns

```
[456]: fig = plt.figure(figsize = (3*10,len(cont_cols)*10/3))
plt.suptitle("hist plots for all numerical columns\n",fontsize=24)
k = 1
for i in cont_cols:
    plt.subplot(math.ceil(len(cont_cols)/3),3,k)
    plt.title("{}".format(i),fontsize = 20)
    k += 1
    plot = sns.histplot(data=data,x = i,kde = True,bins = 50)

    # Increase label and legend font sizes
    plot.set_xlabel(plot.get_xlabel(), fontsize=14)
    plot.set_ylabel(plot.get_ylabel(), fontsize=14)

    warnings.filterwarnings('ignore')
plt.tight_layout()
plt.subplots_adjust(top=0.94)
plt.show()
warnings.filterwarnings('ignore')
```

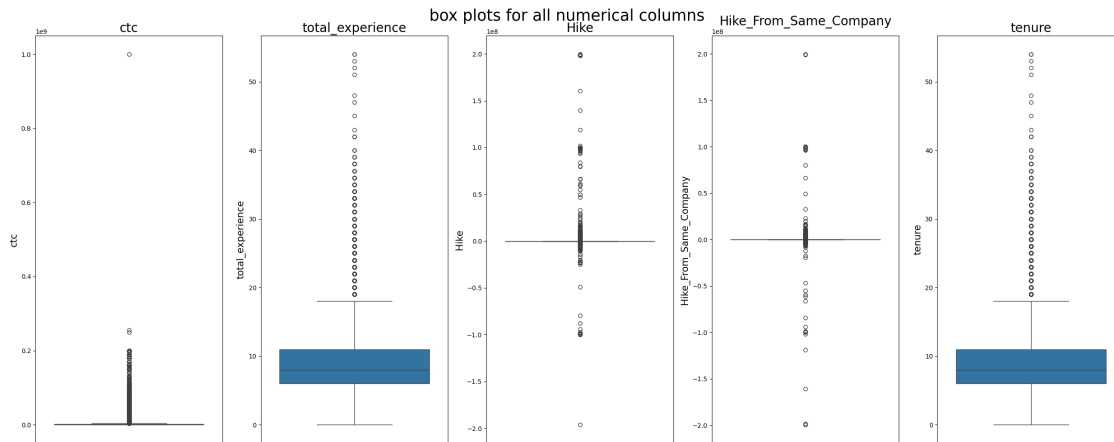


3.1.2 Box plots of all Numerical columns

```
[457]: fig = plt.figure(figsize = (5*5,len(cont_cols)*5/(5/2)))
plt.suptitle("box plots for all numerical columns\n",fontsize=25)
k = 1
for i in cont_cols:
    plt.subplot(math.ceil(len(cont_cols)/5),5,k)
    plt.title("{}".format(i),fontsize = 20)
    k += 1
    plot = sns.boxplot(data=data,y = i)

    # Increase label and legend font sizes
    plot.set_xlabel(plot.get_xlabel(), fontsize=15)
    plot.set_ylabel(plot.get_ylabel(), fontsize=15)

    warnings.filterwarnings('ignore')
plt.tight_layout()
plt.subplots_adjust(top=0.92)
plt.show()
warnings.filterwarnings('ignore')
```



Observation: > High skewness in ctc, hike and hike from same company.

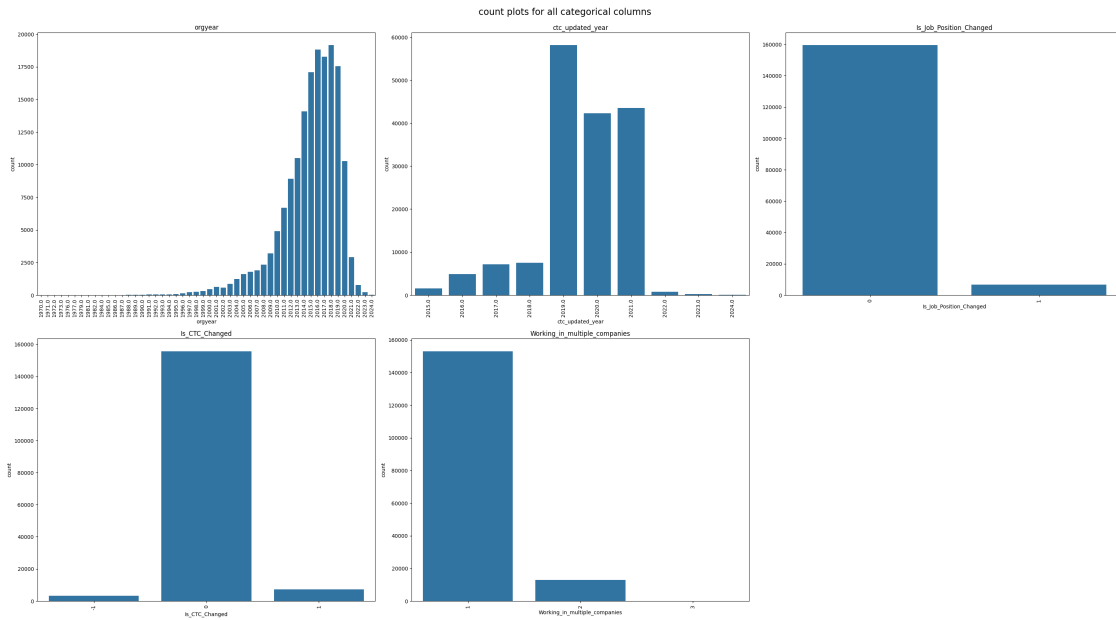
Because we have not removed the outliers in ctc and Hike and Hike_From_Same_Company Features are derived from ctc.

We will not remove the outliers but provide `is_outlier` tags using Anomaly Detection Algorithms

Above plots indicates that most of the members have hikes are salary less 1×10^7

3.1.3 Count plots of all categorical columns

```
[458]: cat_cols2 =
    ↳ ['orgyear', 'ctc_updated_year', 'Is_Job_Position_Changed', 'Is_CTC_Changed', 'Working_in_multip
fig = plt.figure(figsize = (3*10, len(cat_cols2)*10/3))
fig.suptitle("count plots for all categorical columns\n", fontsize = "xx-large" )
k = 1
for i in cat_cols2:
    plt.subplot(math.ceil(len(cat_cols2)/3), 3, k)
    plt.title("{}".format(i))
    sns.countplot(data=data, x = i)
    plt.xticks(rotation = 90)
    k = k+1
plt.tight_layout()
plt.subplots_adjust(top=0.94)
plt.show()
warnings.filterwarnings('ignore')
```

3.2 BIVARIATE ANALYSIS

3.2.1 Top 10 Companies Offering Most Unique Job Positions

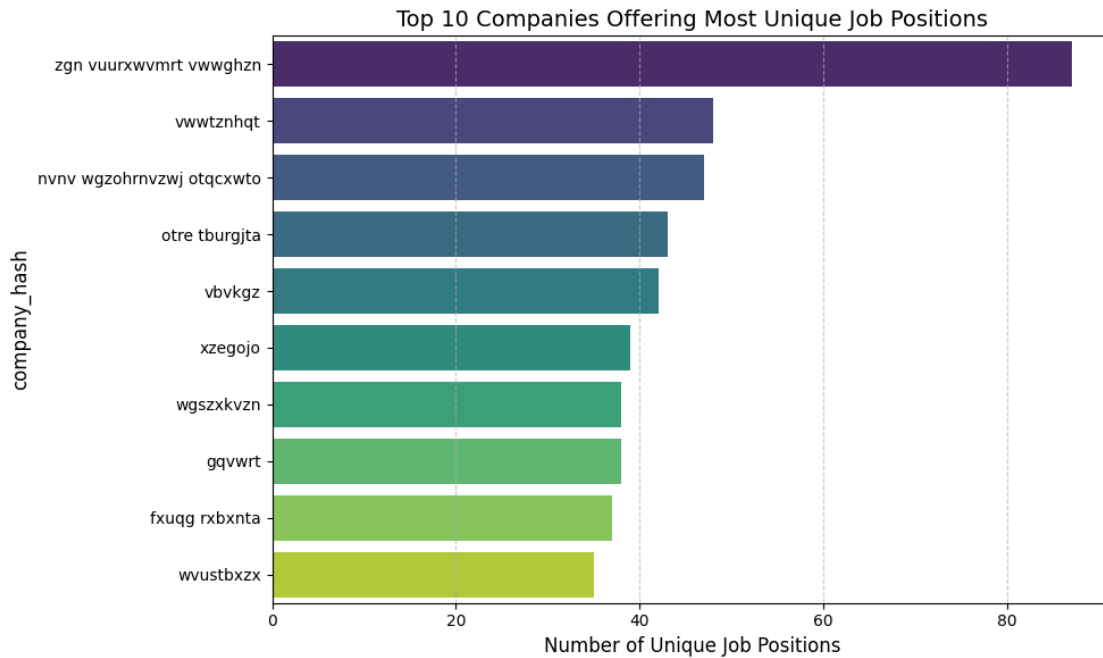
```
[459]: # Step 1: Aggregate the data
top_companies = (
    data.groupby("company_hash")["job_position"]
    .nunique()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_companies,
    x="job_position",
    y="company_hash",
    palette="viridis"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies Offering Most Unique Job Positions", fontsize=14)
plt.xlabel("Number of Unique Job Positions", fontsize=12)
plt.ylabel("company_hash", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
```

```
plt.tight_layout()

# Step 4: Display the plot
plt.show()
```



3.2.2 Top 10 Job Positions Offered in Most Companies

```
[460]: # Step 1: Aggregate the data
top_positions = (
    data.groupby("job_position")["company_hash"]
        .nunique()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# Rename columns for clarity
top_positions.columns = ["job_position", "num_companies"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_positions,
    x="num_companies",
    y="job_position",
```

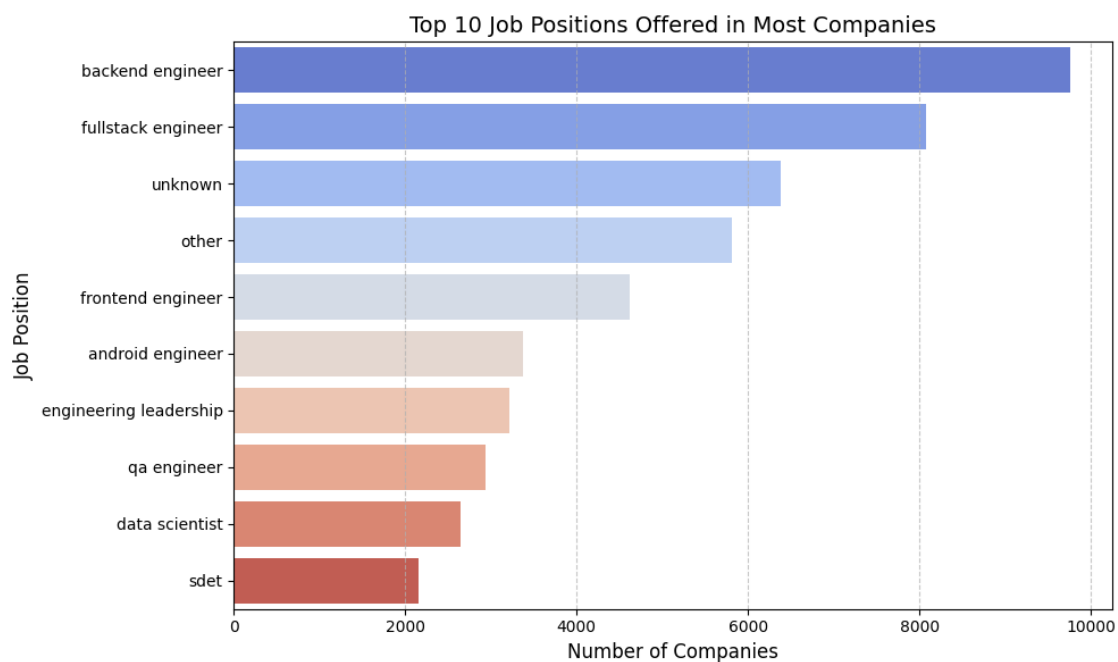
```

    palette="coolwarm"
)

# Step 3: Customize the plot
plt.title("Top 10 Job Positions Offered in Most Companies", fontsize=14)
plt.xlabel("Number of Companies", fontsize=12)
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



3.2.3 Top 10 Companies with Highest Median CTC

```

[461]: # Step 1: Aggregate the data
top_median_ctc = (
    data.groupby("company_hash")["ctc"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# Rename columns for clarity

```

```

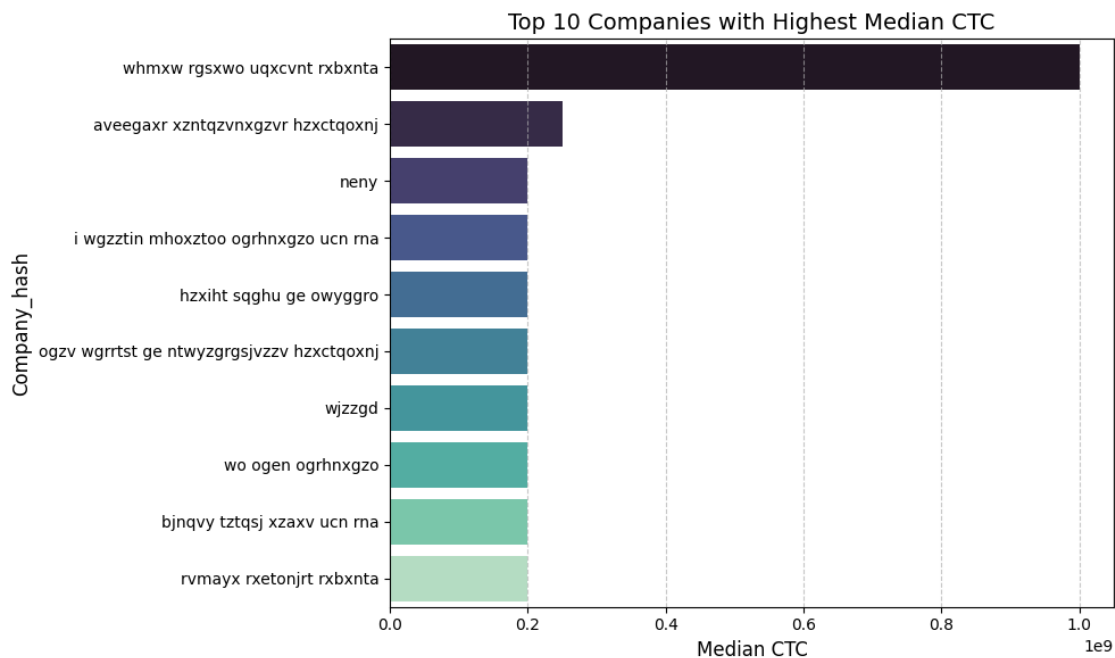
top_median_ctc.columns = ["company_hash", "median_ctc"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_median_ctc,
    x="median_ctc",
    y="company_hash",
    palette="mako"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies with Highest Median CTC", fontsize=14)
plt.xlabel("Median CTC", fontsize=12)
plt.ylabel("Company_hash", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



```
[462]: data[data["company_hash"] == "whmxw rgxwo uqxcvnt rxbxnta"]
```

```

[462]:
27199  29a71dd13adf6d2d497571a565bb3096cf66cb46cd1eceeefa83817ecd4f1371  email_hash \

```

```

                company_hash          ctc job_position  orgyear  \
27199  whmxw rgsxwo uqxcvnt rxbxnta  1000150000      unknown  2015.0

        ctc_updated_year  total_experience  Hike  Hike_From_Same_Company  \
27199          2020.0          9.0    0.0          0.0

        Is_Job_Position_Changed  Is_CTC_Changed  Working_in_multiple_companies  \
27199              0              0              1

        tenure
27199      9.0

```

```
[463]: data[data["company_hash"] == "aveegaxr xzntqzvnxyzvr hzxctqoxnj"]
```

```

[463]:
                                email_hash  \
4531  06d231f167701592a69cdd7d5c825a0f5b30f0347a40782e114434c5e3b5d9d6

                company_hash          ctc job_position  orgyear  \
4531  aveegaxr xzntqzvnxyzvr hzxctqoxnj  2500000000      unknown  2021.0

        ctc_updated_year  total_experience  Hike  Hike_From_Same_Company  \
4531          2021.0          3.0    0.0          0.0

        Is_Job_Position_Changed  Is_CTC_Changed  Working_in_multiple_companies  \
4531              0              0              1

        tenure
4531      3.0

```

Observation: > Lets remove these two outliers because, only these companies have outlier medians. Just two rows, so no problem to remove these outliers.

```
[464]: data = data[~((data["company_hash"] == "whmxw rgsxwo uqxcvnt rxbxnta") |
↪(data["company_hash"] == "aveegaxr xzntqzvnxyzvr hzxctqoxnj"))]
```

```

[465]: # Step 1: Aggregate the data
top_median_ctc = (
    data.groupby("company_hash")["ctc"]
    .median()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

# Rename columns for clarity
top_median_ctc.columns = ["company_hash", "median_ctc"]

```

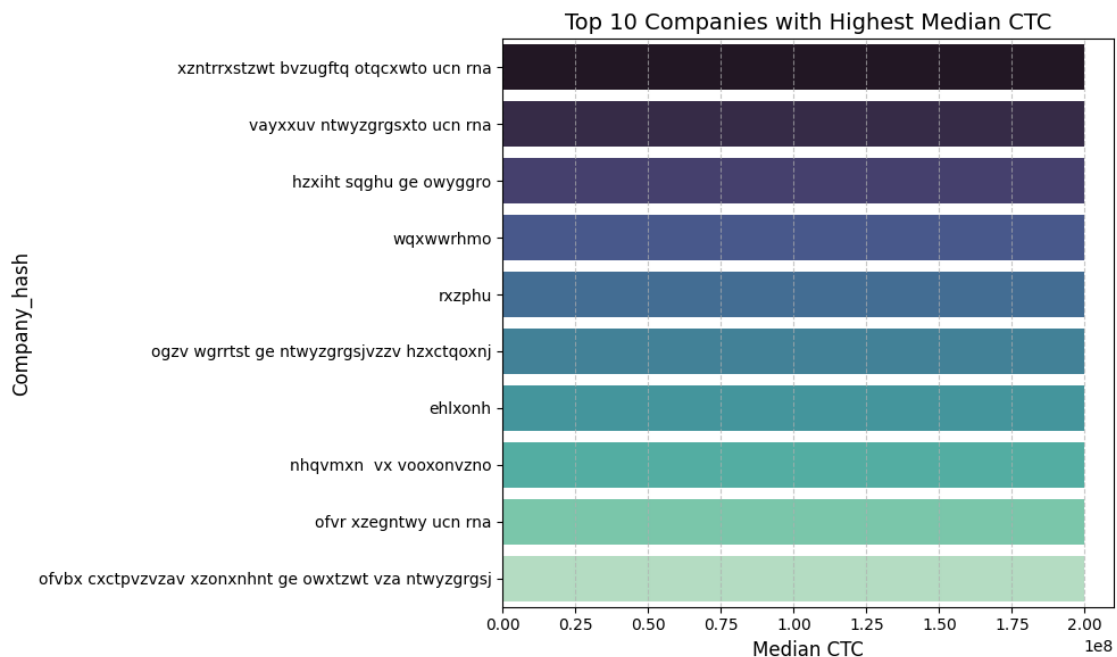
```

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_median_ctc,
    x="median_ctc",
    y="company_hash",
    palette="mako"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies with Highest Median CTC", fontsize=14)
plt.xlabel("Median CTC", fontsize=12)
plt.ylabel("Company_hash", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



3.2.4 Top 10 Job Positions with Highest Median CTC

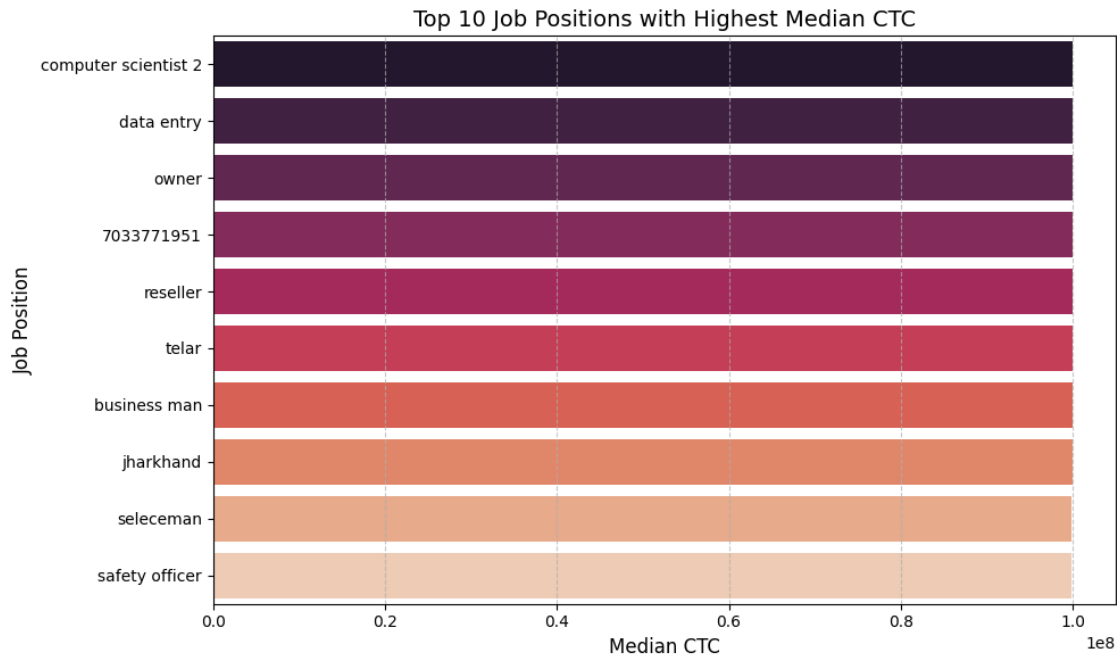
```
[466]: # Step 1: Aggregate the data
top_positions_ctc = (
    data.groupby("job_position")["ctc"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()

# Rename columns for clarity
top_positions_ctc.columns = ["job_position", "median_ctc"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_positions_ctc,
    x="median_ctc",
    y="job_position",
    palette="rocket"
)

# Step 3: Customize the plot
plt.title("Top 10 Job Positions with Highest Median CTC", fontsize=14)
plt.xlabel("Median CTC", fontsize=12)
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()
```



Observation > Data has irrelevant job position names. Generally it is preferred to NLP techniques to clean this feature. It is Future scope of our project.

3.2.5 Top 10 Companies with Highest Net CTC Changes

```
[467]: # Step 1: Aggregate the data
net_ctc_changes = (
    data.groupby("company_hash")["Is_CTC_Changed"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

# Rename columns for clarity
net_ctc_changes.columns = ["company_hash", "net_ctc_change_score"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=net_ctc_changes,
    x="net_ctc_change_score",
    y="company_hash",
    palette="coolwarm"
)
```

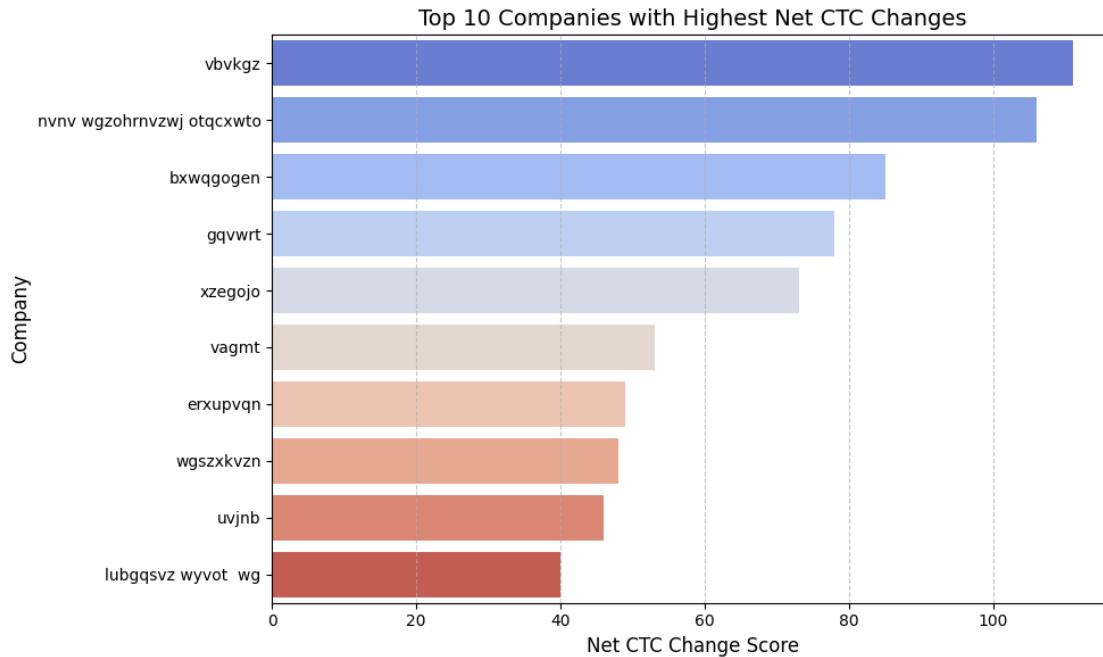


```

# Step 3: Customize the plot
plt.title("Top 10 Companies with Highest Net CTC Changes", fontsize=14)
plt.xlabel("Net CTC Change Score", fontsize=12)
plt.ylabel("Company", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



3.2.6 Top 10 Job Positions with Highest Net CTC Changes

```

[468]: # Step 1: Aggregate the data
net_ctc_changes = (
    data.groupby("job_position")["Is_CTC_Changed"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# Rename columns for clarity
net_ctc_changes.columns = ["job_position", "net_ctc_change_score"]

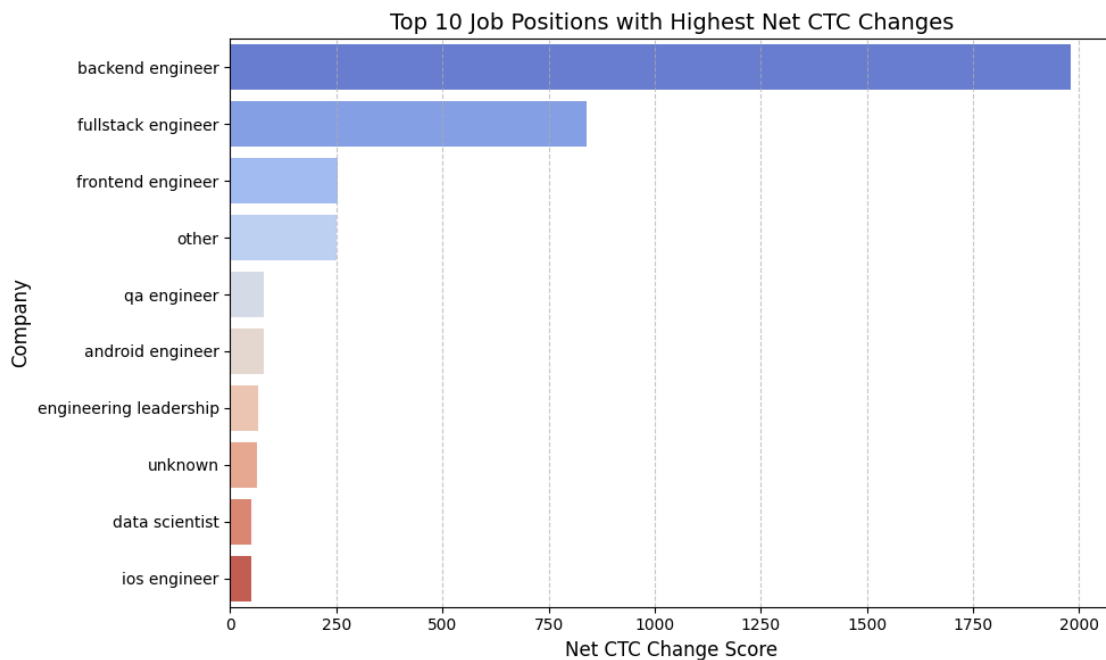
# Step 2: Create a bar plot

```

```
plt.figure(figsize=(10, 6))
sns.barplot(
    data=net_ctc_changes,
    x="net_ctc_change_score",
    y="job_position",
    palette="coolwarm"
)

# Step 3: Customize the plot
plt.title("Top 10 Job Positions with Highest Net CTC Changes", fontsize=14)
plt.xlabel("Net CTC Change Score", fontsize=12)
plt.ylabel("Company", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()
```



3.2.7 Top 10 Companies with Highest number of job changes

```
[469]: # Step 1: Aggregate the data
net_job_changes = (
    data.groupby("company_hash")["Is_Job_Position_Changed"]
    .sum()
    .sort_values(ascending=False)
```

```

        .head(10)
        .reset_index()
    )

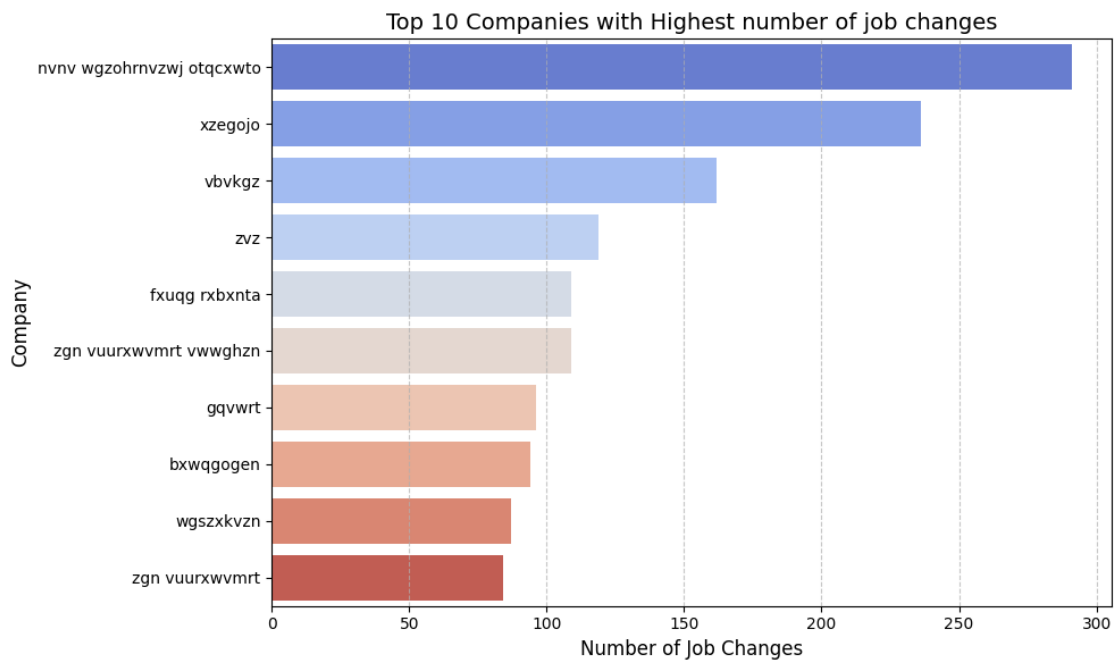
    # Rename columns for clarity
    net_job_changes.columns = ["company_hash", "Job_change_score"]

    # Step 2: Create a bar plot
    plt.figure(figsize=(10, 6))
    sns.barplot(
        data=net_job_changes,
        x="Job_change_score",
        y="company_hash",
        palette="coolwarm"
    )

    # Step 3: Customize the plot
    plt.title("Top 10 Companies with Highest number of job changes", fontsize=14)
    plt.xlabel("Number of Job Changes", fontsize=12)
    plt.ylabel("Company", fontsize=12)
    plt.grid(axis="x", linestyle="--", alpha=0.7)
    plt.tight_layout()

    # Step 4: Display the plot
    plt.show()

```



3.2.8 Top 10 Job Positions with Highest number of job changes

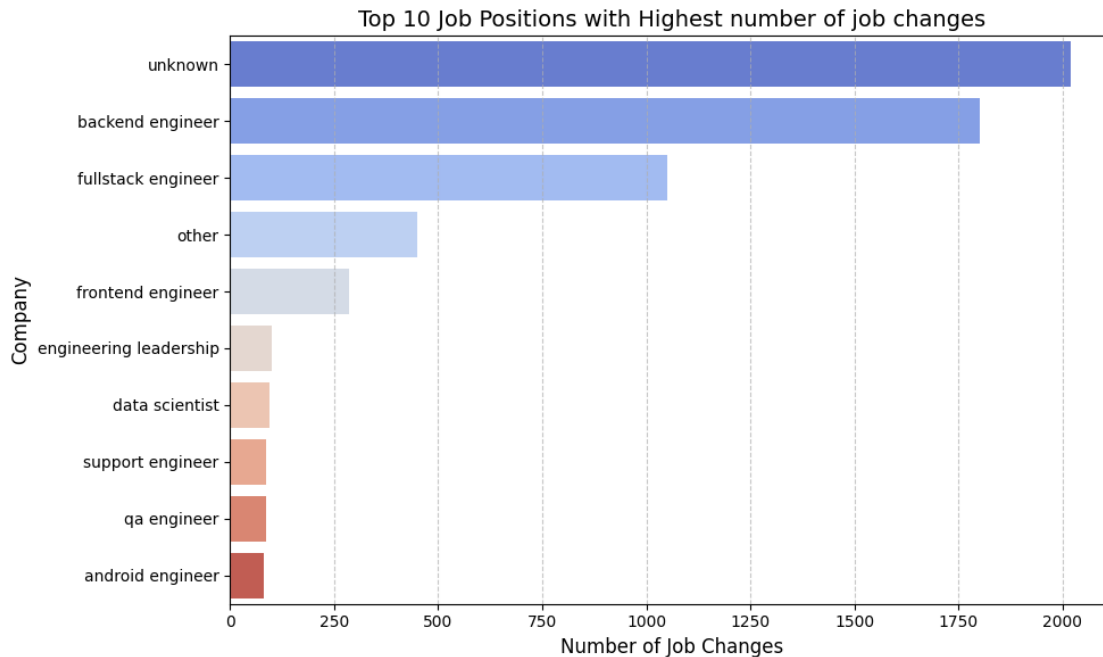
```
[470]: # Step 1: Aggregate the data
net_job_changes = (
    data.groupby("job_position")["Is_Job_Position_Changed"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# Rename columns for clarity
net_job_changes.columns = ["job_position", "Job_change_score"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=net_job_changes,
    x="Job_change_score",
    y="job_position",
    palette="coolwarm"
)

# Step 3: Customize the plot
plt.title("Top 10 Job Positions with Highest number of job changes",
    ↪fontsize=14)
plt.xlabel("Number of Job Changes", fontsize=12)
plt.ylabel("Company", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()
```



3.2.9 Top 10 Job Positions with Highest Median Hike

```
[471]: # Step 1: Aggregate the data
top_positions_Hike = (
    data.groupby("job_position")["Hike"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()

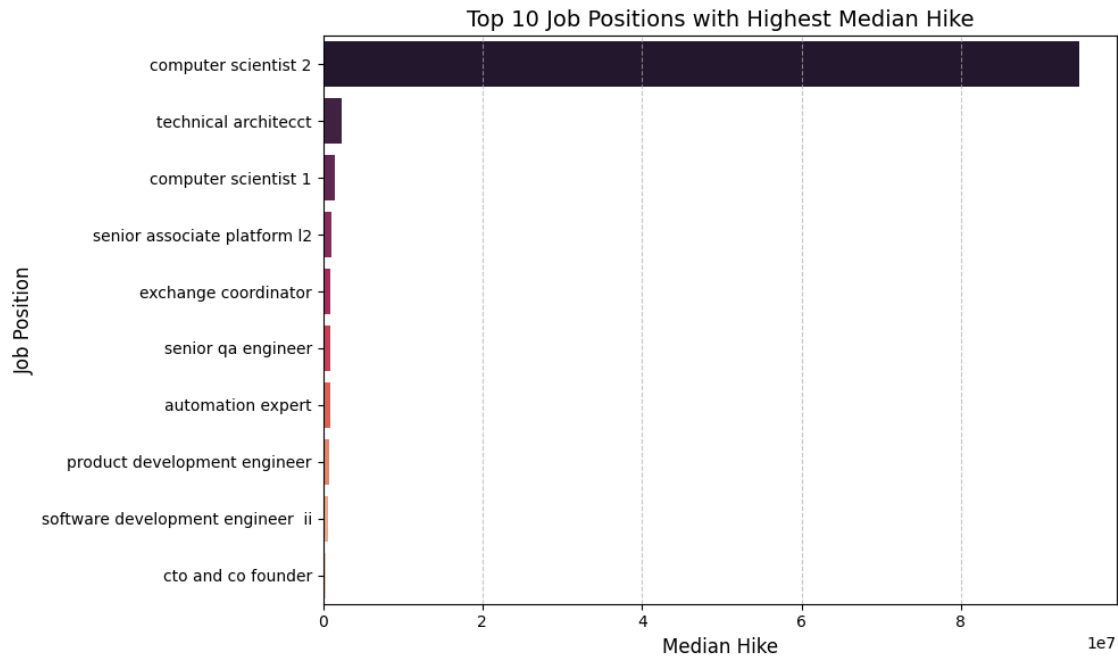
    # Rename columns for clarity
    top_positions_Hike.columns = ["job_position", "median_Hike"]

    # Step 2: Create a bar plot
    plt.figure(figsize=(10, 6))
    sns.barplot(
        data=top_positions_Hike,
        x="median_Hike",
        y="job_position",
        palette="rocket"
    )

    # Step 3: Customize the plot
    plt.title("Top 10 Job Positions with Highest Median Hike", fontsize=14)
    plt.xlabel("Median Hike", fontsize=12)
```

```
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()
```

```
# Step 4: Display the plot
plt.show()
```



```
[472]: data[data["job_position"] == "computer scientist 2"]
```

```
[472]:
          email_hash \
96559  950c1dfb4d651ec9592c640ba2d835e13454f25c0dac2a3780011c96994e7ef8

      company_hash      ctc      job_position  orgyear \
96559      vagmt  100000000  computer scientist 2  2012.0

      ctc_updated_year  total_experience      Hike  Hike_From_Same_Company \
96559      2020.0      12.0  94800000.0      0.0

      Is_Job_Position_Changed  Is_CTC_Changed  Working_in_multiple_companies \
96559      1      1      2

      tenure
96559      12.0
```

Observation: > Removing computer scientist 2 job position as it has only one row as outlier.

```
[473]: data = data[~(data["job_position"] == "computer scientist 2")]
```

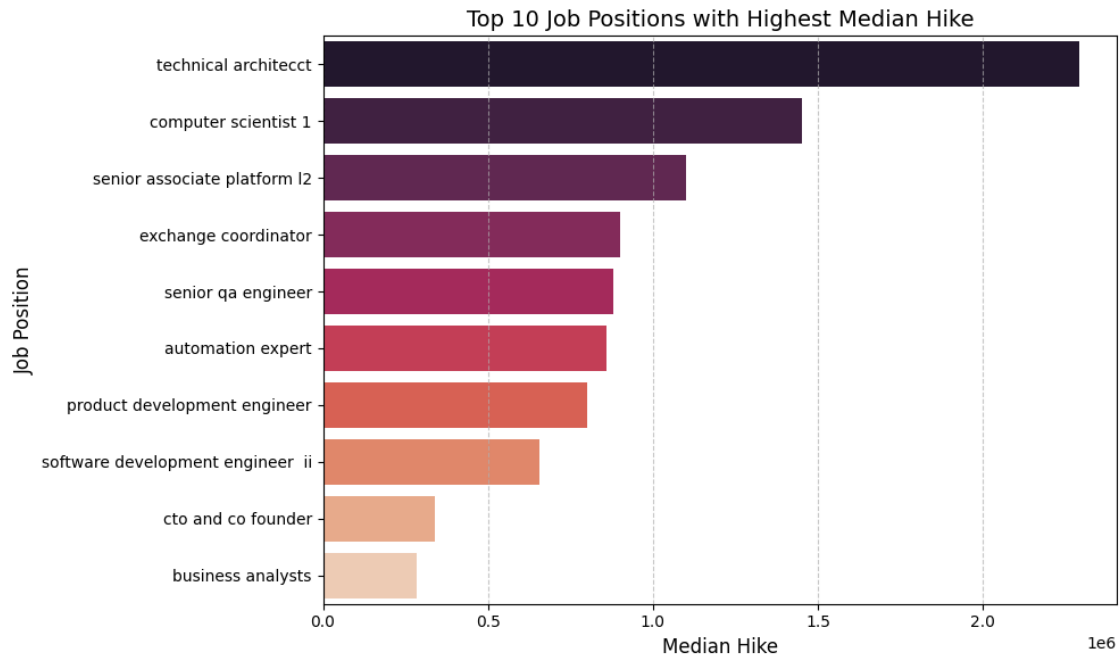
```
[474]: # Step 1: Aggregate the data
top_positions_Hike = (
    data.groupby("job_position")["Hike"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()

# Rename columns for clarity
top_positions_Hike.columns = ["job_position", "median_Hike"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_positions_Hike,
    x="median_Hike",
    y="job_position",
    palette="rocket"
)

# Step 3: Customize the plot
plt.title("Top 10 Job Positions with Highest Median Hike", fontsize=14)
plt.xlabel("Median Hike", fontsize=12)
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()
```



3.2.10 Top 10 Companies with Highest Median Hike

```
[475]: # Step 1: Aggregate the data
top_median_Hike = (
    data.groupby("company_hash")["Hike"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# Rename columns for clarity
top_median_Hike.columns = ["company_hash", "median_Hike"]

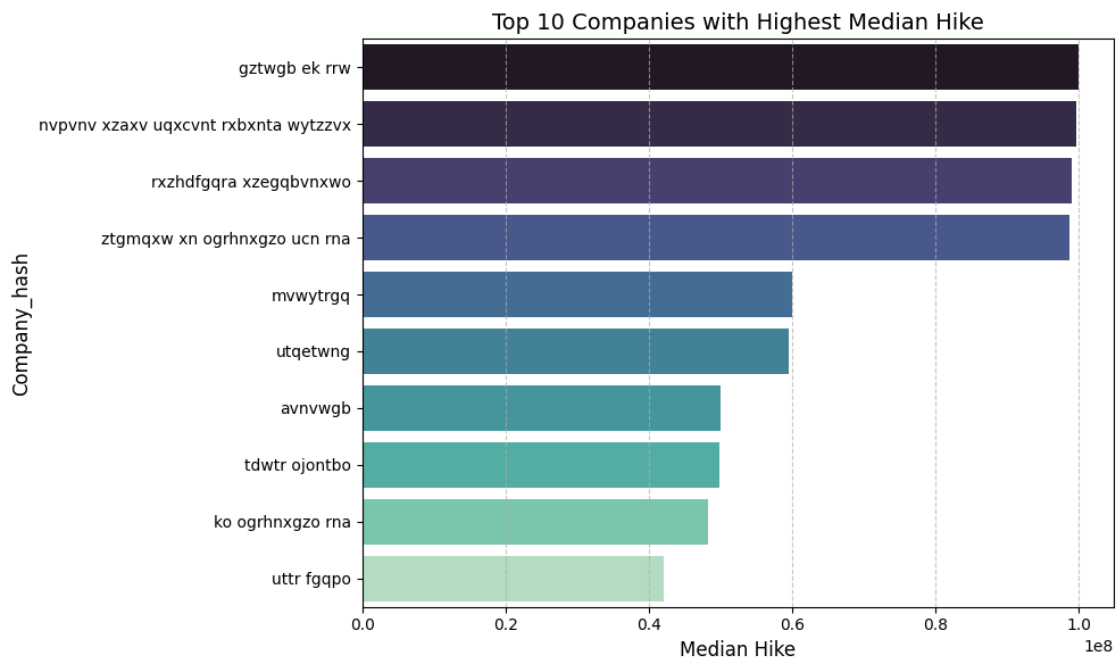
# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_median_Hike,
    x="median_Hike",
    y="company_hash",
    palette="mako"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies with Highest Median Hike", fontsize=14)
```



```
plt.xlabel("Median Hike", fontsize=12)
plt.ylabel("Company_hash", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()
```

```
# Step 4: Display the plot
plt.show()
```



3.2.11 Top 10 Job Positions with Highest Median Hike_From_Same_Company

```
[476]: # Step 1: Aggregate the data
top_positions_Hike_From_Same_Company = (
    data.groupby("job_position")["Hike_From_Same_Company"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()

# Rename columns for clarity
top_positions_Hike_From_Same_Company.columns = ["job_position", "
    ↪"median_Hike_From_Same_Company"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
```

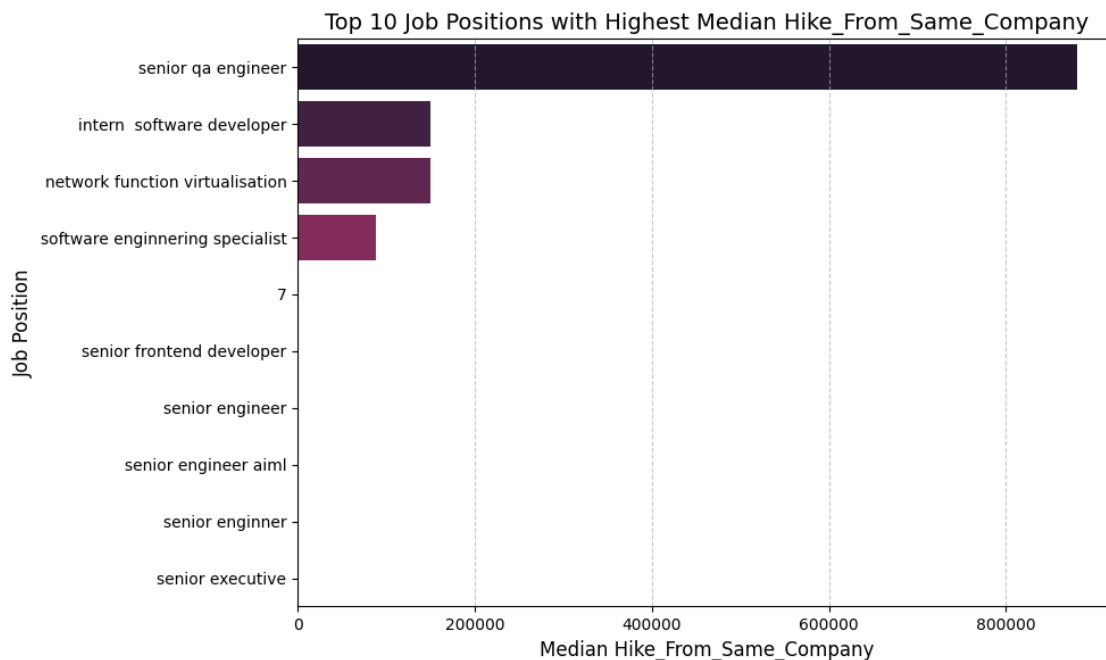
```

data=top_positions_Hike_From_Same_Company,
x="median_Hike_From_Same_Company",
y="job_position",
palette="rocket"
)

# Step 3: Customize the plot
plt.title("Top 10 Job Positions with Highest Median Hike_From_Same_Company",
fontsize=14)
plt.xlabel("Median Hike_From_Same_Company", fontsize=12)
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



```

[477]: pd.set_option('display.max_colwidth',1000)
data[data["job_position"] == "senior qa engineer"]

```

```

[477]:
      company_hash  ctc  job_position  orgyear  ctc_updated_year \
86262  xzegqbvnxwv  1950000  senior qa engineer  2015.0  2021.0

```

	total_experience	Hike	Hike_From_Same_Company	\
86262	9.0	880000.0	880000.0	

	Is_Job_Position_Changed	Is_CTC_Changed	Working_in_multiple_companies	\
86262	1	1	1	

	tenure
86262	9.0

```
[478]: pd.reset_option('all')
```

```
[479]: data[data["email_hash"] == "854a957e70432f9bdc77385f6defc96d759c41cb7d19384e3c7a3415bc7db468"]
```

```
[479]:
```

	email_hash	company_hash	\
86261	854a957e70432f9bdc77385f6defc96d759c41cb7d1938...	xzegqbvnxwv	
86262	854a957e70432f9bdc77385f6defc96d759c41cb7d1938...	xzegqbvnxwv	

	ctc	job_position	orgyear	ctc_updated_year	\
86261	1070000	database administrator	2015.0	2019.0	
86262	1950000	senior qa engineer	2015.0	2021.0	

	total_experience	Hike	Hike_From_Same_Company	\
86261	9.0	0.0	0.0	
86262	9.0	880000.0	880000.0	

	Is_Job_Position_Changed	Is_CTC_Changed	Working_in_multiple_companies	\
86261	0	0	1	
86262	1	1	1	

	tenure
86261	9.0
86262	9.0

Observation: > I don't think above case as an outlier. Hike is around 82%. It is possible in reality.

3.2.12 Top 10 Companies with Highest Median Hike_From_Same_Company

```
[480]: # Step 1: Aggregate the data
top_median_Hike_From_Same_Company = (
    data.groupby("company_hash")["Hike_From_Same_Company"]
    .median()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
```

```

)

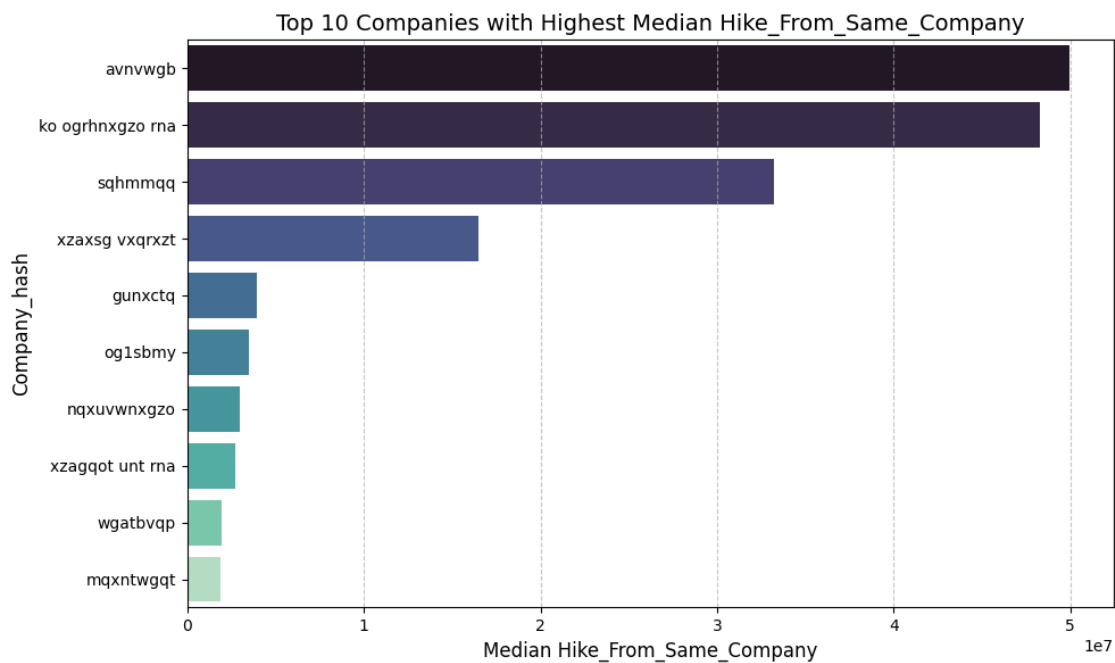
# Rename columns for clarity
top_median_Hike_From_Same_Company.columns = ["company_hash",
↪ "median_Hike_From_Same_Company"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_median_Hike_From_Same_Company,
    x="median_Hike_From_Same_Company",
    y="company_hash",
    palette="mako"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies with Highest Median Hike_From_Same_Company",
↪ fontsize=14)
plt.xlabel("Median Hike_From_Same_Company", fontsize=12)
plt.ylabel("Company_hash", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



```
[481]: data[data["company_hash"].isin( ["avnvwgb",'ko ogrhnxgzo rna','sqhmmqq','xzaxsg',
↪vxqrxzt'])]
```

```
[481]:
```

	email_hash	company_hash	\
5771	089953112b145b21c71e1a36d2e36024d261c0baa67927...	avnvwgb	
5772	089953112b145b21c71e1a36d2e36024d261c0baa67927...	avnvwgb	
13863	14df647edb4209bf283db88e2fea7d52f2e321481dcc4e...	xzaxsg vxqrxzt	
13864	14df647edb4209bf283db88e2fea7d52f2e321481dcc4e...	xzaxsg vxqrxzt	
127796	c520f6cd75fbb93c81fe0aa48917118d77ea211f297155...	sqhmmqq	
127797	c520f6cd75fbb93c81fe0aa48917118d77ea211f297155...	sqhmmqq	
156021	f089ccc6256b6700aa41db90281903f4cee94be0c89d0e...	ko ogrhnxgzo rna	
156022	f089ccc6256b6700aa41db90281903f4cee94be0c89d0e...	ko ogrhnxgzo rna	

	ctc	job_position	orgyear	ctc_updated_year	\
5771	80000	support engineer	2007.0	2020.0	
5772	100000000	support engineer	2007.0	2020.0	
13863	400000	support engineer	2017.0	2020.0	
13864	33400000	engineering leadership	2017.0	2020.0	
127796	600000	android engineer	2015.0	2020.0	
127797	67000000	android engineer	2015.0	2020.0	
156021	3500000	engineering leadership	2014.0	2019.0	
156022	100000000	backend engineer	2014.0	2020.0	

	total_experience	Hike	Hike_From_Same_Company	\
5771	17.0	0.0	0.0	
5772	17.0	99920000.0	99920000.0	
13863	7.0	0.0	0.0	
13864	7.0	33000000.0	33000000.0	
127796	9.0	0.0	0.0	
127797	9.0	66400000.0	66400000.0	
156021	10.0	0.0	0.0	
156022	10.0	96500000.0	96500000.0	

	Is_Job_Position_Changed	Is_CTC_Changed	\
5771	0	0	
5772	0	1	
13863	0	0	
13864	1	1	
127796	0	0	
127797	0	1	
156021	0	0	
156022	1	1	

	Working_in_multiple_companies	tenure
5771	1	17.0
5772	1	17.0
13863	1	7.0

13864	1	7.0
127796	1	9.0
127797	1	9.0
156021	1	10.0
156022	1	10.0

```
[482]: data = data[~((data["company_hash"].isin( ["avnvwgb", 'ko ogrhnxgzo', 'sqhmmqq', 'xzaxsg vxqrxzt']) & (data["Hike_From_Same_Company"] > 0)))]
```

```
[483]: data[data["company_hash"].isin( ["avnvwgb", 'ko ogrhnxgzo rna', 'sqhmmqq', 'xzaxsg vxqrxzt'])]
```

```
[483]:
```

	email_hash	company_hash \
5771	089953112b145b21c71e1a36d2e36024d261c0baa67927...	avnvwgb
13863	14df647edb4209bf283db88e2fea7d52f2e321481dcc4e...	xzaxsg vxqrxzt
127796	c520f6cd75fbb93c81fe0aa48917118d77ea211f297155...	sqhmmqq
156021	f089ccc6256b6700aa41db90281903f4cee94be0c89d0e...	ko ogrhnxgzo rna

	ctc	job_position	orgyear	ctc_updated_year \
5771	80000	support engineer	2007.0	2020.0
13863	400000	support engineer	2017.0	2020.0
127796	600000	android engineer	2015.0	2020.0
156021	3500000	engineering leadership	2014.0	2019.0

	total_experience	Hike	Hike_From_Same_Company \
5771	17.0	0.0	0.0
13863	7.0	0.0	0.0
127796	9.0	0.0	0.0
156021	10.0	0.0	0.0

	Is_Job_Position_Changed	Is_CTC_Changed \
5771	0	0
13863	0	0
127796	0	0
156021	0	0

	Working_in_multiple_companies	tenure
5771	1	17.0
13863	1	7.0
127796	1	9.0
156021	1	10.0

```
[484]: # Step 1: Aggregate the data
top_median_Hike_From_Same_Company = (
    data.groupby("company_hash")["Hike_From_Same_Company"]
    .median()
    .sort_values(ascending=False)
```

```

.head(10)
.reset_index()
)

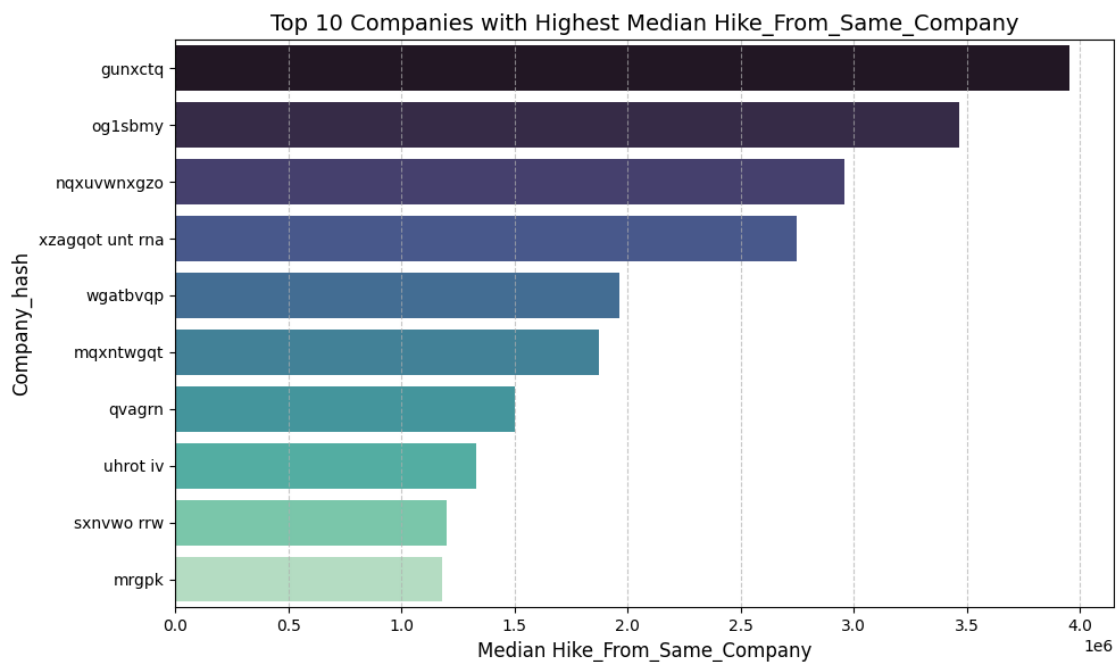
# Rename columns for clarity
top_median_Hike_From_Same_Company.columns = ["company_hash",
↪ "median_Hike_From_Same_Company"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_median_Hike_From_Same_Company,
    x="median_Hike_From_Same_Company",
    y="company_hash",
    palette="mako"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies with Highest Median Hike_From_Same_Company",
↪ fontsize=14)
plt.xlabel("Median Hike_From_Same_Company", fontsize=12)
plt.ylabel("Company_hash", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



Observations: > Hikes observed in above cases are unrealistic (more than 1000% hike)

Treating them as outlier and removing them

3.2.13 Top 10 Companies with Highest Number of Jobs Simultaneously

```
[485]: # Step 1: Filter the data for values 2 and 3 in 'Working_in_multiple_companies'
filtered_data = data[data['Working_in_multiple_companies'].isin([2, 3])]

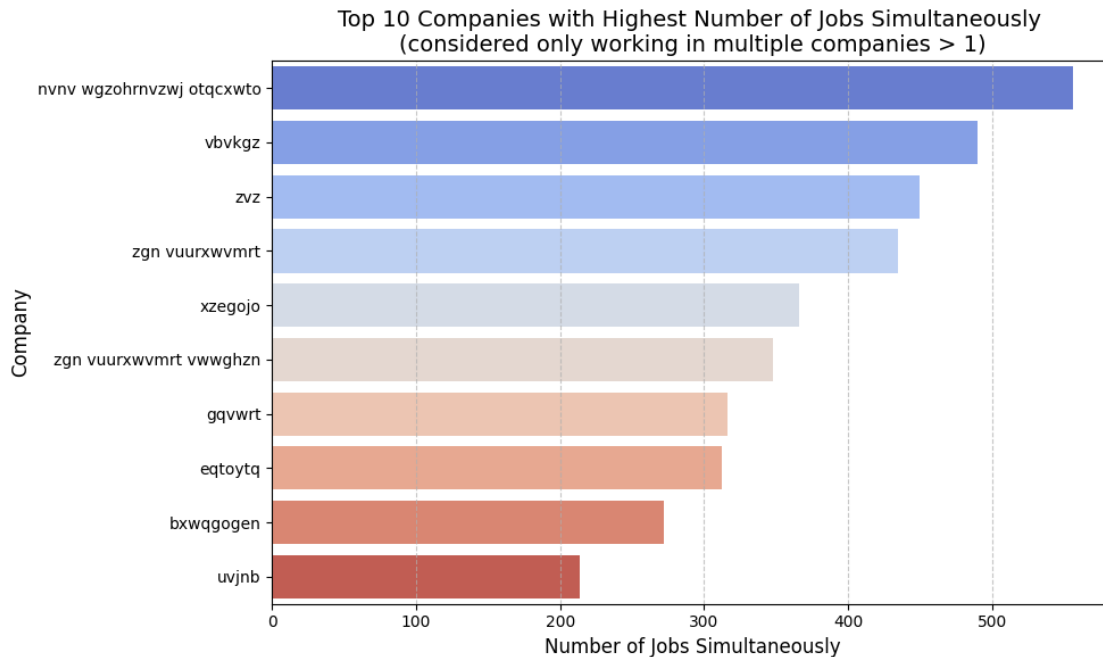
# Step 2: Aggregate the data
net_job_simultaneously = (
    filtered_data.groupby("company_hash")["Working_in_multiple_companies"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

# Rename columns for clarity
net_job_simultaneously.columns = ["company_hash", "Job_change_score"]

# Step 3: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=net_job_simultaneously,
    x="Job_change_score",
    y="company_hash",
    palette="coolwarm"
)

# Step 4: Customize the plot
plt.title("Top 10 Companies with Highest Number of Jobs Simultaneously_↵
↵(considered only working in multiple companies > 1)", fontsize=14)
plt.xlabel("Number of Jobs Simultaneously", fontsize=12)
plt.ylabel("Company", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 5: Display the plot
plt.show()
```

3.2.14 Top 10 Job Postions with Highest Number of Jobs Simultaneously

```
[486]: # Step 1: Filter the data for values 2 and 3 in 'Working_in_multiple_companies'
filtered_data = data[data['Working_in_multiple_companies'].isin([2, 3])]

# Step 2: Aggregate the data
net_job_simultaneously = (
    filtered_data.groupby("job_position")["Working_in_multiple_companies"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

# Rename columns for clarity
net_job_simultaneously.columns = ["job_position", "Job_change_score"]

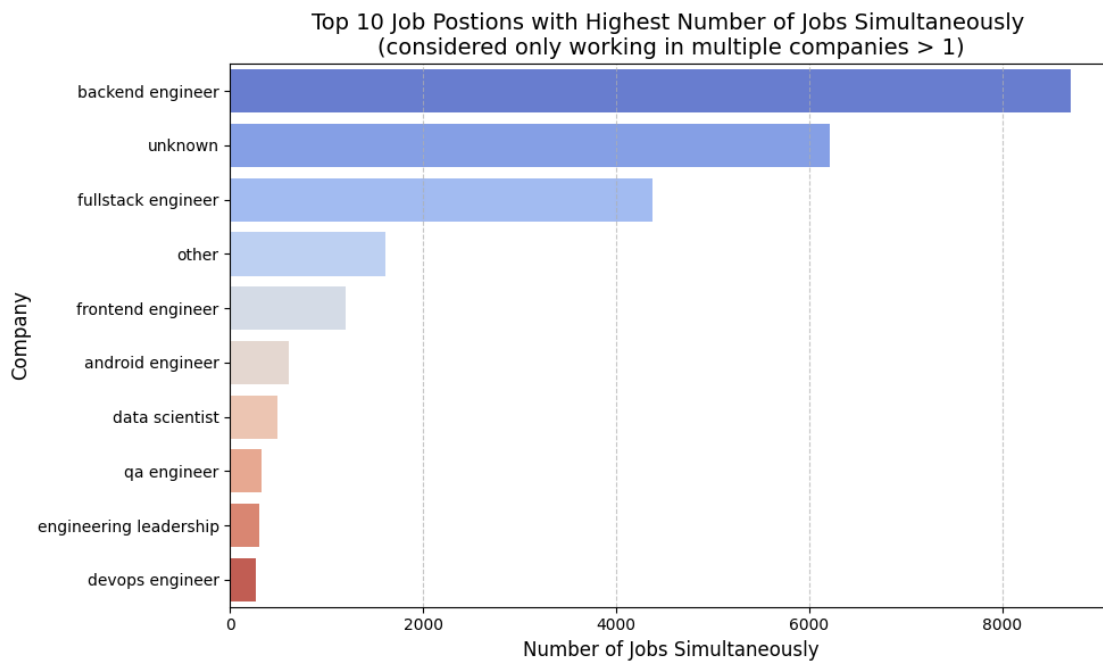
# Step 3: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=net_job_simultaneously,
    x="Job_change_score",
    y="job_position",
    palette="coolwarm"
)
```

```

# Step 4: Customize the plot
plt.title("Top 10 Job Postions with Highest Number of Jobs Simultaneously_↵
↵\n(considered only working in multiple companies > 1)", fontsize=14)
plt.xlabel("Number of Jobs Simultaneously", fontsize=12)
plt.ylabel("Company", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 5: Display the plot
plt.show()

```



3.2.15 Top 10 Companies by Median total_experience

```

[487]: # Step 1: Aggregate the data
median_total_experience = (
    data.groupby("company_hash")["total_experience"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# Rename columns for clarity
median_total_experience.columns = ["company_hash", "Median_total_experience"]

```

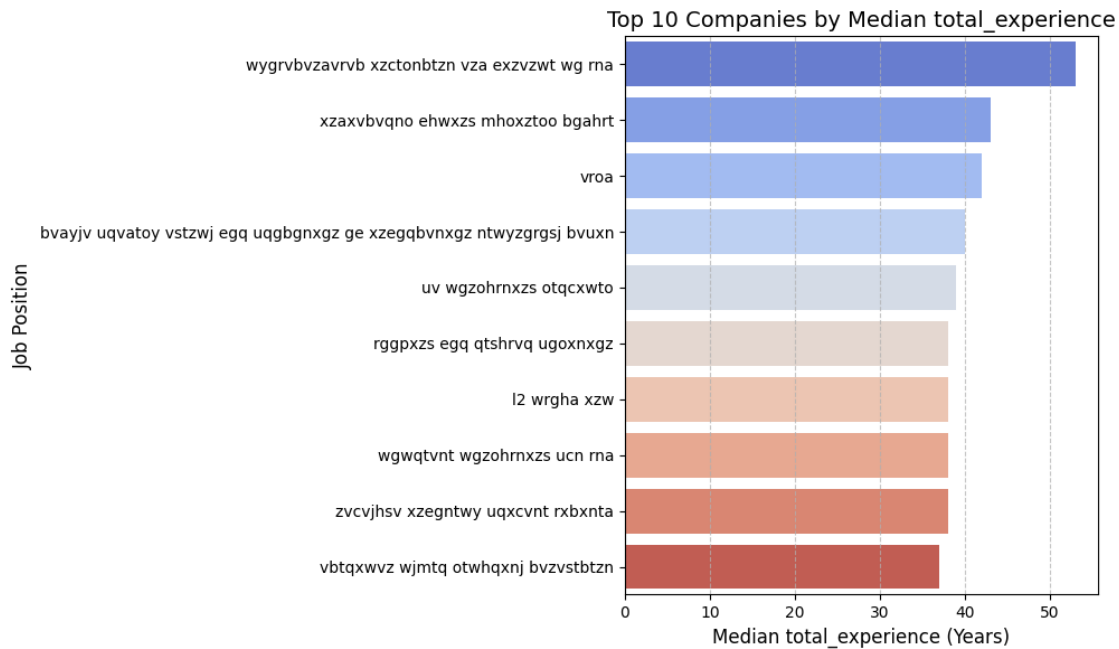
```

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=median_total_experience,
    x="Median_total_experience",
    y="company_hash",
    palette="coolwarm"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies by Median total_experience", fontsize=14)
plt.xlabel("Median total_experience (Years)", fontsize=12)
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()

```



3.2.16 Top 10 Job Positions by Median total_experience

```

[488]: # Step 1: Aggregate the data
median_total_experience = (
    data.groupby("job_position")["total_experience"]

```

```

        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
    )

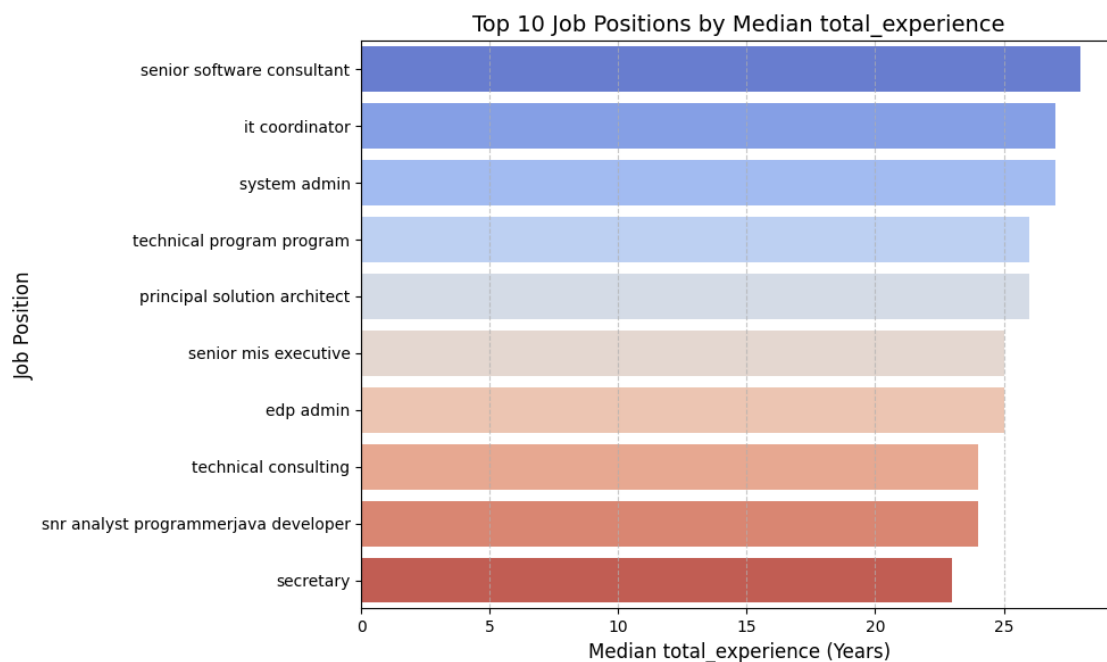
    # Rename columns for clarity
    median_total_experience.columns = ["job_position", "Median_total_experience"]

    # Step 2: Create a bar plot
    plt.figure(figsize=(10, 6))
    sns.barplot(
        data=median_total_experience,
        x="Median_total_experience",
        y="job_position",
        palette="coolwarm"
    )

    # Step 3: Customize the plot
    plt.title("Top 10 Job Positions by Median total_experience", fontsize=14)
    plt.xlabel("Median total_experience (Years)", fontsize=12)
    plt.ylabel("Job Position", fontsize=12)
    plt.grid(axis="x", linestyle="--", alpha=0.7)
    plt.tight_layout()

    # Step 4: Display the plot
    plt.show()

```



3.2.17 Top 10 Companies by Median Tenure

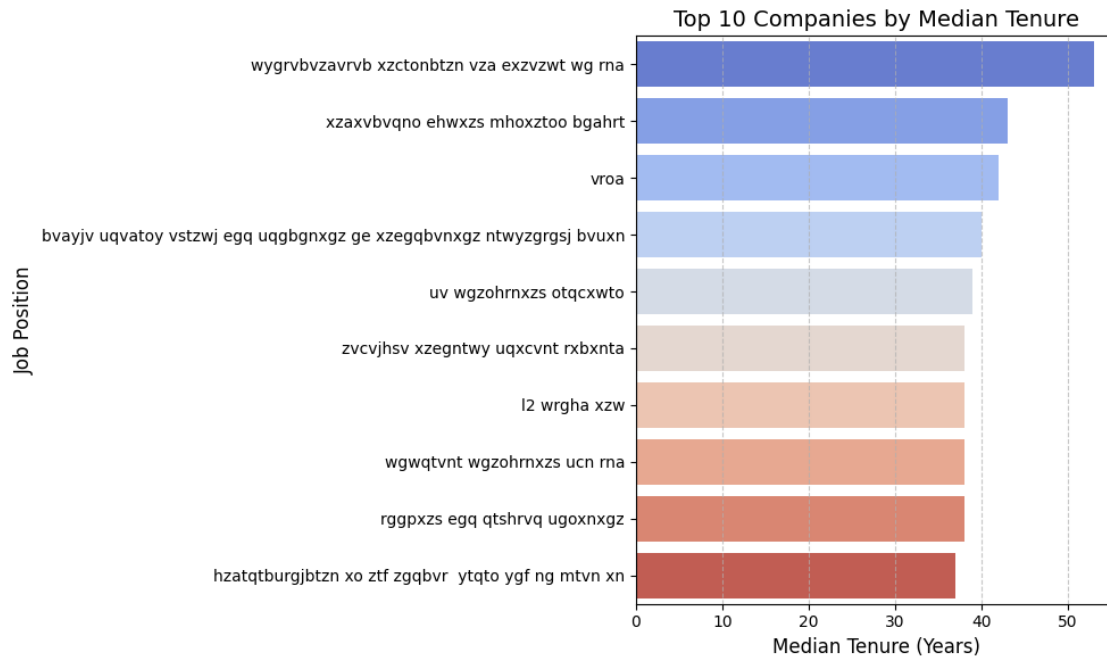
```
[489]: # Step 1: Aggregate the data
median_tenure = (
    data.groupby("company_hash")["tenure"]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
)

# Rename columns for clarity
median_tenure.columns = ["company_hash", "Median_Tenure"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=median_tenure,
    x="Median_Tenure",
    y="company_hash",
    palette="coolwarm"
)

# Step 3: Customize the plot
plt.title("Top 10 Companies by Median Tenure", fontsize=14)
plt.xlabel("Median Tenure (Years)", fontsize=12)
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()

# Step 4: Display the plot
plt.show()
```



3.2.18 Top 10 Job Positions by Median Tenure

```
[490]: # Step 1: Aggregate the data
median_tenure = (
    data.groupby("job_position")["tenure"]
    .median()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

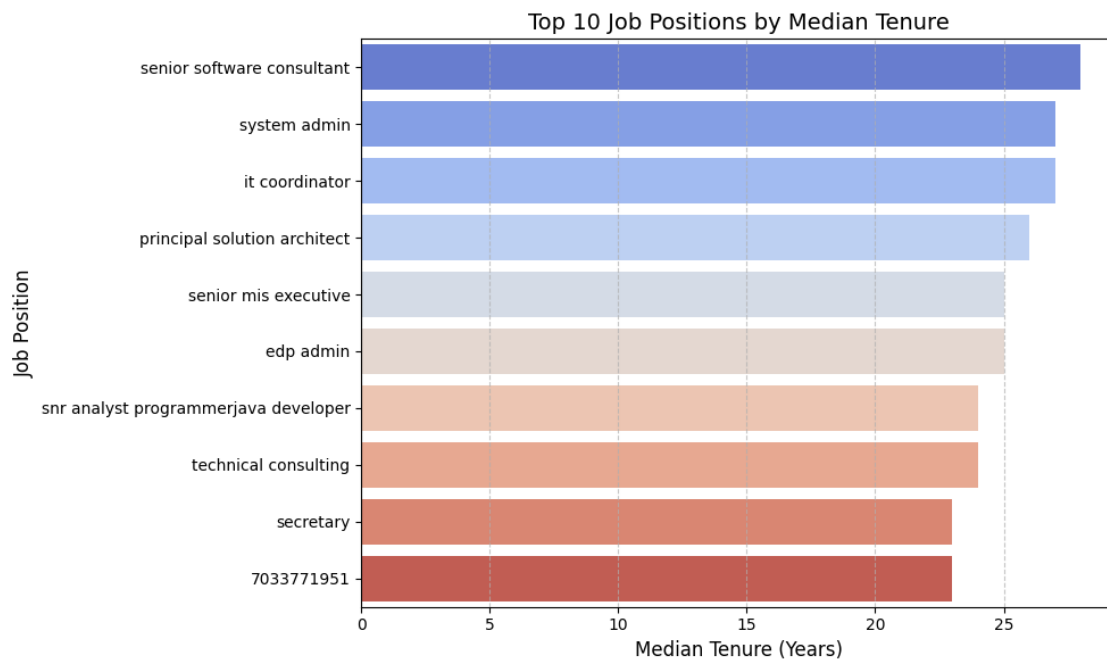
# Rename columns for clarity
median_tenure.columns = ["job_position", "Median_Tenure"]

# Step 2: Create a bar plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=median_tenure,
    x="Median_Tenure",
    y="job_position",
    palette="coolwarm"
)

# Step 3: Customize the plot
plt.title("Top 10 Job Positions by Median Tenure", fontsize=14)
```

```
plt.xlabel("Median Tenure (Years)", fontsize=12)
plt.ylabel("Job Position", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.7)
plt.tight_layout()
```

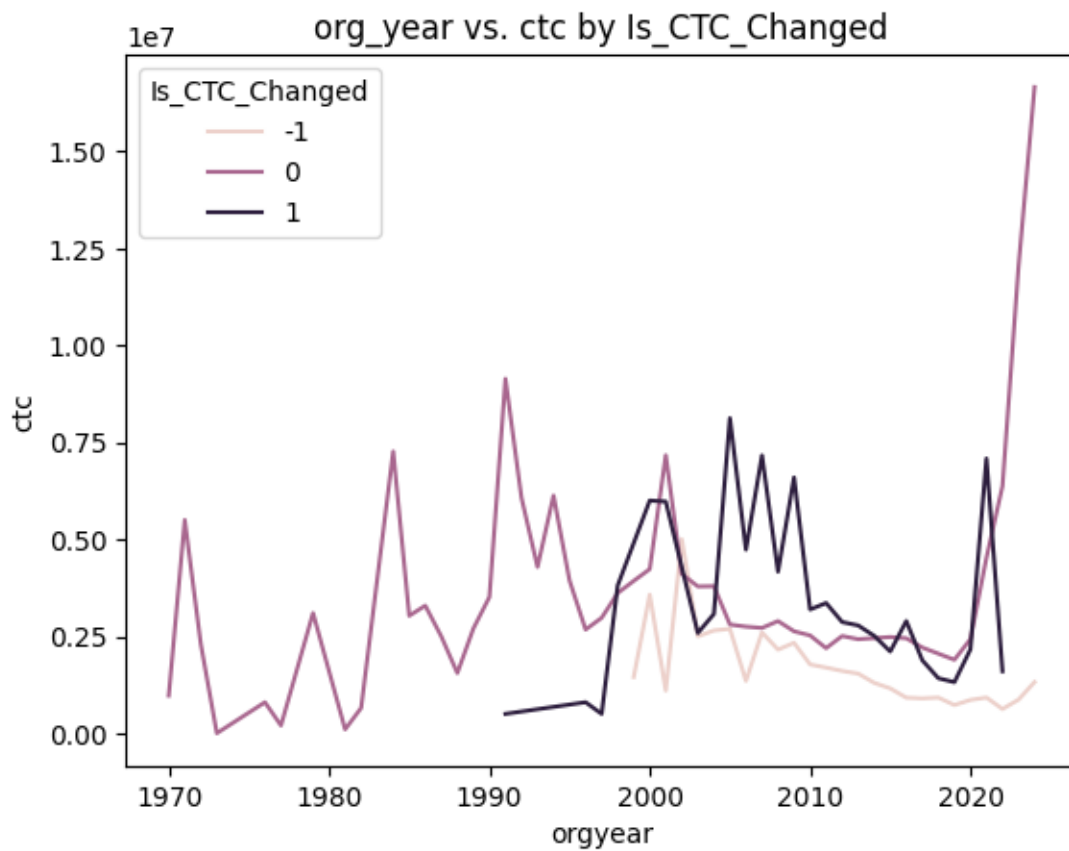
```
# Step 4: Display the plot
plt.show()
```



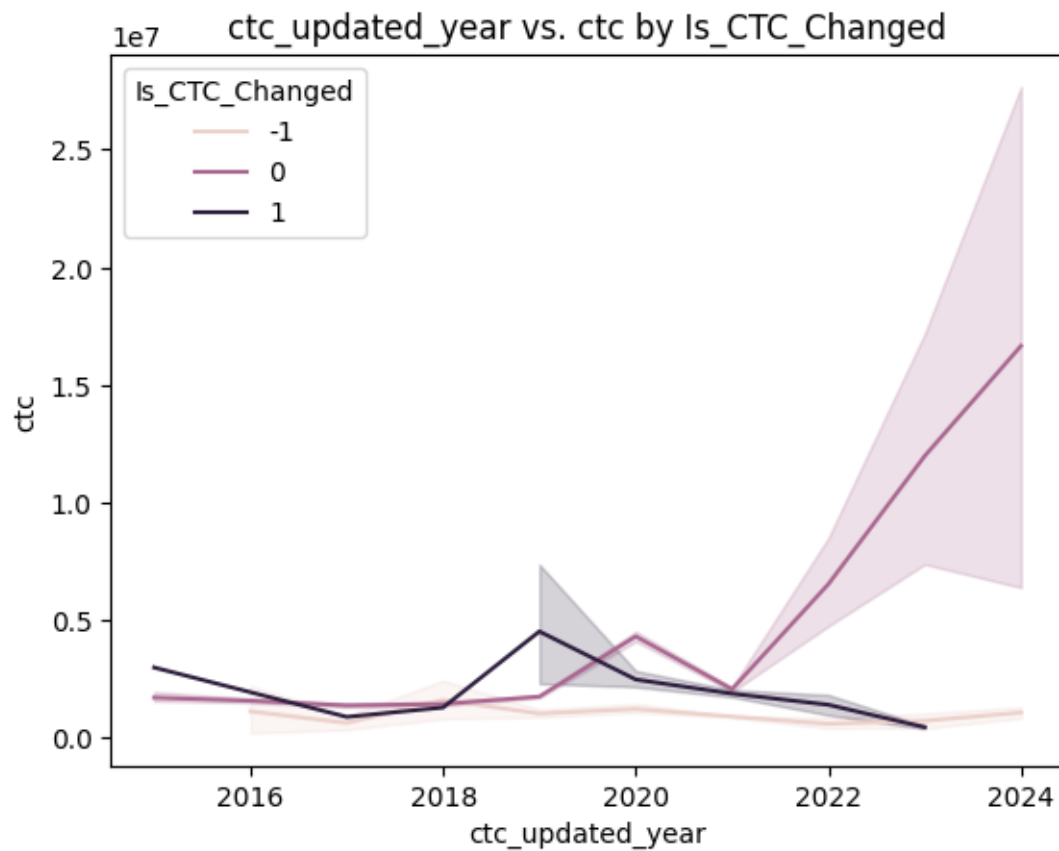
3.3 MULTI-VARIATE PLOTS

3.3.1 Line Plots orgyear/ctc_updated_year vs ctc wrt Is_CTC_Changed

```
[491]: sns.lineplot(data=data, x='orgyear', y='ctc', hue='Is_CTC_Changed', ci=None)
plt.title('org_year vs. ctc by Is_CTC_Changed')
plt.show()
```

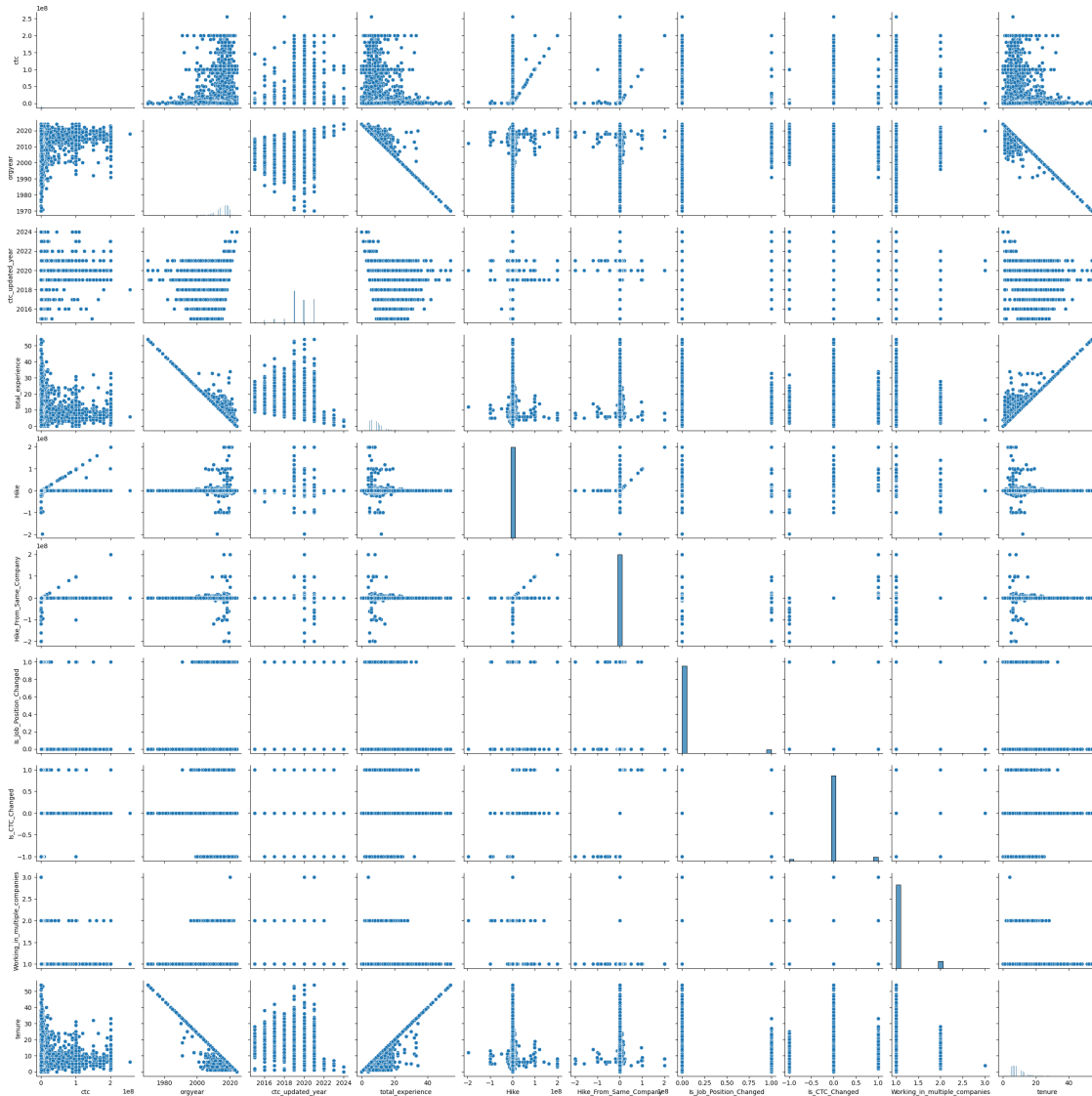


```
[492]: sns.lineplot(data=data, x='ctc_updated_year', y='ctc', hue='Is_CTC_Changed')
plt.title('ctc_updated_year vs. ctc by Is_CTC_Changed')
plt.show()
```

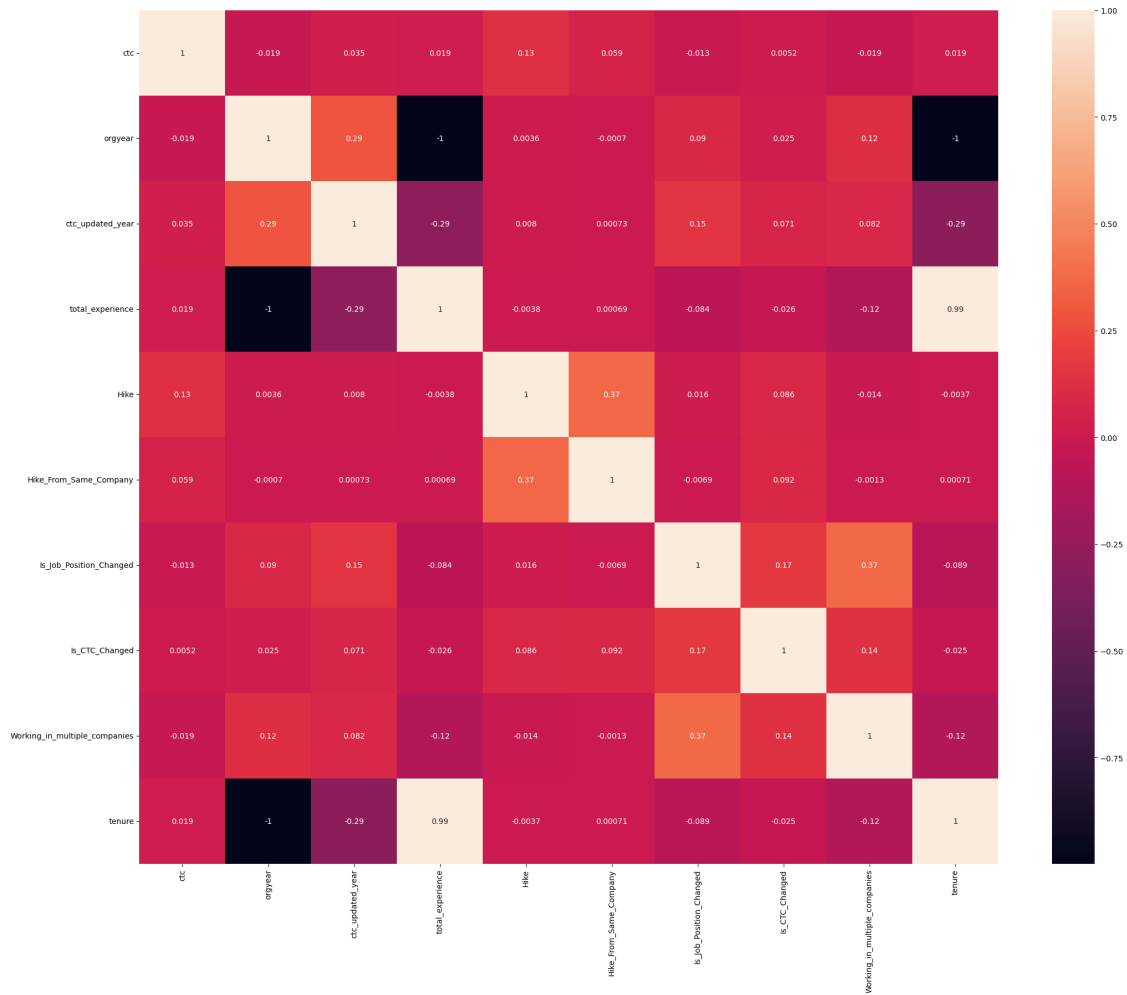



[493]: `sns.pairplot(data)`

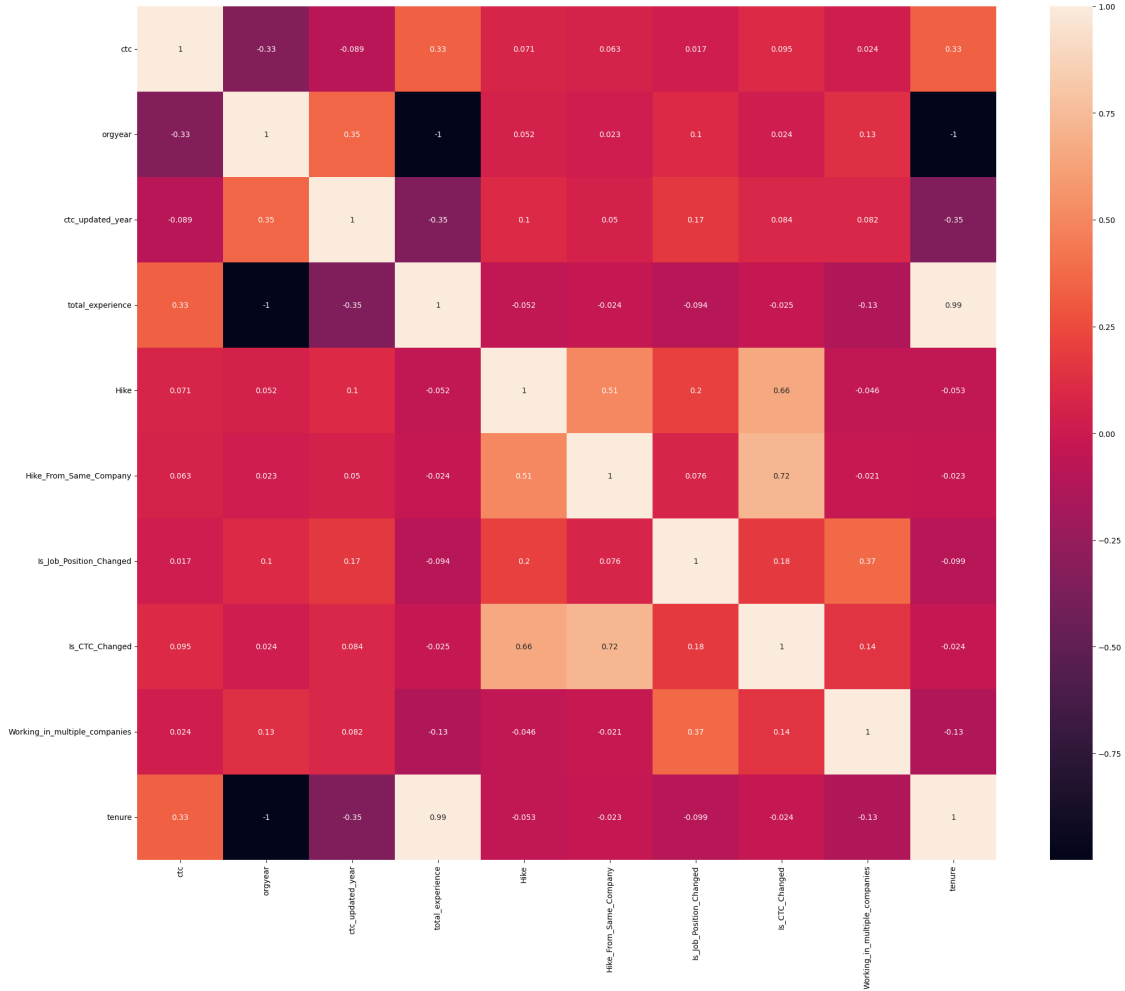
[493]: `<seaborn.axisgrid.PairGrid at 0x1f77f6d1a00>`



```
[494]: corr = data.corr(numeric_only = True)
plt.figure(figsize=(25,20))
sns.heatmap(corr,annot=True)
plt.show()
```



```
[495]: corr = data.corr(method = "spearman", numeric_only = True)
plt.figure(figsize=(25,20))
sns.heatmap(corr,annot=True)
plt.show()
```



3.4 INSIGHTS

There is some clustering of data points in terms of CTC and total_experience. This suggests that there are groups of employees with similar salaries and experience levels.

There appears to be linear relationships in some pairs of variables, such as ctc and Hike.

The distribution of orgyear is skewed towards recent years, indicating that most organizations in the dataset were established after 2000.

total_experience and tenure are strongly positively correlated (~ 0.99), as expected (more experience tends to correspond to longer tenure).

Hike and Hike_From_Same_Company have a positive correlation (~ 0.37), indicating that staying in the same company might contribute to salary hikes.

Is_Job_Position_Changed and Is_CTC_Changed show some positive correlation (~ 0.17), implying that role changes might be linked to salary changes.

Working_in_Multiple_Companies and Is_Job_Changed has some positive correlation(~ 0.37), indicating that people changes the job for flexibility of working in multiple companies.

```
[496]: data.to_csv("data_for_manual_clustering.csv", index=False)
```

```
[650]: data_mc = pd.read_csv("data_for_manual_clustering.csv")
```

```
[651]: data_mc.head()
```

```
[651]:
```

	email_hash	company_hash \
0	00003288036a44374976948c327f246fdbdf0778546904...	bxwqgogen
1	0000aaa0e6b61f7636af1954b43d294484cd151c9b3cf6...	nqsn axsxivr
2	0000d58fbc18012bf6fa2605a7b0357d126ee69bc41032...	gunhb
3	000120d0c8aa304fcf12ab4b85e21feb80a342cfea03d4...	bxwqgotbx wgqugqvnxyz
4	00014d71a389170e668ba96ae8e1f9d991591acc899025...	fvrbvqn rvmo

	ctc	job_position	orgyear	ctc_updated_year	total_experience \
0	3500000	backend engineer	2012.0	2019.0	12.0
1	250000	backend engineer	2013.0	2020.0	11.0
2	1300000	fullstack engineer	2021.0	2021.0	3.0
3	2000000	fullstack engineer	2004.0	2021.0	20.0
4	3400000	unknown	2009.0	2018.0	15.0

	Hike	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed \
0	0.0	0.0	0	0
1	0.0	0.0	0	0
2	0.0	0.0	0	0
3	0.0	0.0	0	0
4	0.0	0.0	0	0

	Working_in_multiple_companies	tenure
0	1	12.0
1	1	11.0
2	1	3.0
3	1	20.0
4	1	15.0

```
[652]: data_mc.isna().sum()
```

```
[652]: email_hash      0
company_hash      0
ctc               0
job_position      0
orgyear           0
ctc_updated_year  0
total_experience   0
```

```
Hike 0
Hike_From_Same_Company 0
Is_Job_Position_Changed 0
Is_CTC_Changed 0
Working_in_multiple_companies 0
tenure 0
dtype: int64
```

```
[653]: data_mc.duplicated().sum()
```

```
[653]: 0
```

```
[654]: data_mc.columns
```

```
[654]: Index(['email_hash', 'company_hash', 'ctc', 'job_position', 'orgyear',
          'ctc_updated_year', 'total_experience', 'Hike',
          'Hike_From_Same_Company', 'Is_Job_Position_Changed', 'Is_CTC_Changed',
          'Working_in_multiple_companies', 'tenure'],
          dtype='object')
```

4 CHAPTER 4: MANUAL CLUSTERING

4.1 GETTING 5 POINT SUMMARY OF CTC (MEAN, MEDIAN, MAX, MIN, COUNT) ON THE BASIS OF COMPANY, JOB_POSITION, TOTAL_EXPERIENCE

4.1.1 Group by company_hash

```
[655]: ctc_company = data_mc.groupby('company_hash').agg({'ctc': ['mean', 'median', 'max', 'min', 'count']})
ctc_company.head()
```

```
[655]:
```

	ctc				
	mean	median	max	min	count
company_hash					
0	100000.0	100000.0	100000	100000	1
0000	300000.0	300000.0	300000	300000	1
01 ojztqsj	550000.0	550000.0	830000	270000	2
05mz exzytvrny uqxcvnt rxbxnta	1100000.0	1100000.0	1100000	1100000	1
1	175000.0	175000.0	250000	100000	2

```
[656]: ctc_company.shape
```

```
[656]: (37271, 5)
```

4.1.2 Group by job_position

```
[657]: ctc_job_position=data_mc.groupby('job_position').agg({'ctc':['mean', 'median', 'max', 'min', 'count' ]})
ctc_job_position.tail()
```

```
[657]:
```

	ctc				
	mean	median	max	min	count
job_position					
web developer full stack	300000.0	300000.0	300000	300000	1
web engineer	1150000.0	1150000.0	1150000	1150000	1
webcenter consultant	300000.0	300000.0	300000	300000	1
worker	200000.0	200000.0	200000	200000	1
zomato	300000.0	300000.0	500000	100000	2

```
[658]: ctc_job_position.shape
```

```
[658]: (619, 5)
```

4.1.3 Group by total_experience

```
[659]: ctc_job_exp = data_mc.groupby('total_experience').agg({'ctc':['mean', 'median', 'max', 'min', 'count' ]})
ctc_job_exp.head()
```

```
[659]:
```

	ctc				
	mean	median	max	min	count
total_experience					
0.0	1.666167e+07	800000.0	110000000	60000	36
1.0	1.208540e+07	880000.0	200000000	600	216
2.0	6.357998e+06	900000.0	200000000	200	756
3.0	4.477825e+06	700000.0	200000000	1000	2880
4.0	2.388825e+06	700000.0	200000000	24	10257

```
[660]: ctc_job_exp.shape
```

```
[660]: (50, 5)
```

4.2 CREATING DESIGNATION, CLASS & TIER FEATURES BY OBTAINING GROUPED STATISTICS

4.2.1 Designation Feature Engineering from grouped Statistics by Experience, Job Position, and Company

```
[661]: # Group by the required columns for 'Designation'
data_mc3 = data_mc.groupby(['company_hash', 'job_position', 'total_experience'])
```

```
[662]: # Calculate the 50th percentile (median) and 75th percentile for 'Designation'
data_mc['ctc_median of 3'] = data_mc3['ctc'].transform(np.median) # 50th
↳percentile
data_mc['ctc_75th of 3'] = data_mc3['ctc'].transform(lambda x: np.percentile(x,
↳75)) # 75th percentile
```

```
[663]: # Apply the conditions to classify into 'Designation'
designation_conditions = [
    data_mc['ctc'] < data_mc['ctc_median of 3'], # Below the 50th percentile
↳indicated with 3
    (data_mc['ctc'] >= data_mc['ctc_median of 3']) & (data_mc['ctc'] <=
↳data_mc['ctc_75th of 3']), # Between 50th and 75th percentiles indicated
↳with 2
    data_mc['ctc'] > data_mc['ctc_75th of 3'] # Above the 75th percentile
↳Indicated with 1
]
designation_choices = [3, 2, 1]
data_mc["Designation"] = np.select(designation_conditions, designation_choices,
↳default=np.nan)
```

```
[664]: # # Drop unnecessary columns for 'Designation'
# data_mc.drop(columns=['ctc_median of 3', 'ctc_75th of 3'], inplace=True)
```

4.2.2 Class Feature Engineering from grouped Statistics by Job Position and Company

```
[665]: # Group by the required columns for 'Class'
data_mc2 = data_mc.groupby(['company_hash', 'job_position'])
```

```
[666]: # Calculate the 50th percentile (median) and 75th percentile for 'Class'
data_mc['ctc_median of 2'] = data_mc2['ctc'].transform(np.median) # 50th
↳percentile
data_mc['ctc_75th of 2'] = data_mc2['ctc'].transform(lambda x: np.percentile(x,
↳75)) # 75th percentile
```

```
[667]: # Apply the conditions to classify into 'Class'
class_conditions = [
    data_mc['ctc'] < data_mc['ctc_median of 2'], # Below the 50th percentile
↳indicated with 3
    (data_mc['ctc'] >= data_mc['ctc_median of 2']) & (data_mc['ctc'] <=
↳data_mc['ctc_75th of 2']), # Between 50th and 75th percentiles indicated
↳with 2
    data_mc['ctc'] > data_mc['ctc_75th of 2'] # Above the 75th percentile
↳indicated with 1
]
class_choices = [3, 2, 1]
```



```
data_mc["Class"] = np.select(class_conditions, class_choices, default=np.nan)
```

```
[668]: # # Drop unnecessary columns for 'Class'
# data_mc.drop(columns=['ctc_median of 2', 'ctc_75th of 2'], inplace=True)
```

4.2.3 Tier Feature Engineering from grouped Statistics by Company

```
[669]: # Group by the required columns for 'Tier'
data_mc1 = data_mc.groupby(['company_hash'])
```

```
[670]: # Calculate the 50th percentile (median) and 75th percentile for 'Tier'
data_mc['ctc_median of 1'] = data_mc1['ctc'].transform(np.median) # 50th
    ↪ percentile
data_mc['ctc_75th of 1'] = data_mc1['ctc'].transform(lambda x: np.percentile(x,
    ↪ 75)) # 75th percentile
```

```
[671]: # Apply the conditions to classify into 'Tier'
tier_conditions = [
    data_mc['ctc'] < data_mc['ctc_median of 1'], # Below the 50th percentile
    ↪ indicated with 3
    (data_mc['ctc'] >= data_mc['ctc_median of 1']) & (data_mc['ctc'] <=
    ↪ data_mc['ctc_75th of 1']), # Between 50th and 75th percentiles indicaed
    ↪ with 2
    data_mc['ctc'] > data_mc['ctc_75th of 1'] # Above the 75th percentile
    ↪ indicated with 1
]
tier_choices = [3, 2, 1]
data_mc["Tier"] = np.select(tier_conditions, tier_choices, default=np.nan)
```

```
[672]: # # Drop unnecessary columns for 'Tier'
# data_mc.drop(columns=['ctc_median of 1', 'ctc_75th of 1'], inplace=True)
```

```
[673]: # Final dataset preview
data_mc.head()
```

```
[673]:
```

	email_hash	company_hash	\
0	00003288036a44374976948c327f246fdbdf0778546904...	bxwqgogen	
1	0000aaa0e6b61f7636af1954b43d294484cd151c9b3cf6...	nqsn axsnvr	
2	0000d58fbc18012bf6fa2605a7b0357d126ee69bc41032...	gunhb	
3	000120d0c8aa304fcf12ab4b85e21feb80a342cfea03d4...	bxwqgotbx wgugqvnxyz	
4	00014d71a389170e668ba96ae8e1f9d991591acc899025...	fvrbvqn rvmo	

	ctc	job_position	orgyear	ctc_updated_year	total_experience	\
0	3500000	backend engineer	2012.0	2019.0	12.0	
1	250000	backend engineer	2013.0	2020.0	11.0	
2	1300000	fullstack engineer	2021.0	2021.0	3.0	

3	2000000	fullstack engineer	2004.0	2021.0	20.0
4	3400000	unknown	2009.0	2018.0	15.0

	Hike	Hike_From_Same_Company	Is_Job_Position_Changed	...	tenure	\
0	0.0	0.0	0	...	12.0	
1	0.0	0.0	0	...	11.0	
2	0.0	0.0	0	...	3.0	
3	0.0	0.0	0	...	20.0	
4	0.0	0.0	0	...	15.0	

	ctc_median of 3	ctc_75th of 3	Designation	ctc_median of 2	\
0	2800000.0	3425000.0	1.0	2600000.0	
1	250000.0	250000.0	2.0	250000.0	
2	1300000.0	1431000.0	2.0	1200000.0	
3	2000000.0	2000000.0	2.0	1067500.0	
4	3900000.0	3950000.0	3.0	2127500.0	

	ctc_75th of 2	Class	ctc_median of 1	ctc_75th of 1	Tier
0	3600000.0	2.0	2500000.0	3500000.0	2.0
1	250000.0	2.0	250000.0	250000.0	2.0
2	1423250.0	2.0	1200000.0	1625000.0	2.0
3	1533750.0	1.0	1625000.0	2170000.0	2.0
4	3262500.0	1.0	2500000.0	3375000.0	1.0

[5 rows x 22 columns]

```
[674]: data_mc.shape
```

```
[674]: (166052, 22)
```

4.3 INSIGHTS FROM MANUAL CLUSTERING

4.3.1 Top 10 employees (earning more than most of the employees in the company) - Tier 1

```
[675]: data_mc[(data_mc['Tier'] == 1) ].nlargest(n=10, columns=['ctc'])
```

```
[675]:
```

	email_hash	\
59098	5b4bed51797140db4ed52018a979db1e34cee49e27b488...	
600	00eebd69d81c0bb5d4ad91392d7b994395333d98cae7f0...	
2295	0391b9c03e6601ce26e7257e6b9eb80a49304123a05217...	
3187	04e01c24282a7262949b2efa7f1b3663aad9bfe2649232...	
3675	05a0aec8db0d251460a2e200d835b7818e050308ef1e67...	
4502	06c7cd6c1a50803bac81950ad450a0e78165cb7c09ddfe...	
6131	0914aa3a4f2394bb68aabe99f5ca93c769fc3e05917752...	
8434	0c9c37269bd373ef507df0bc1bb318787fd895c858b74e...	
8777	0d235f7e73cd9484909b32a35c69df12296a051f68ef83...	

9311 0df461881c039e7d8fa8db9b24f28ff190c74fed483594...

	company_hash	ctc	job_position	orgyear	\
59098	obvqnuqxdwgb	255555555	unknown	2018.0	
600	wgszxkvzn	200000000	backend engineer	2013.0	
2295	mvzp ge vbtqxwv	200000000	devops engineer	2014.0	
3187	lubgqsvz wyvot wg	200000000	fullstack engineer	2021.0	
3675	egdwgzz	200000000	engineering leadership	2018.0	
4502	gnytqo	200000000	product manager	2018.0	
6131	nvnv wgzohrnvwj otqcxwto	200000000	unknown	2016.0	
8434	xzegojo	200000000	unknown	2020.0	
8777	nvnv wgzohrnvwj otqcxwto	200000000	other	2017.0	
9311	ywr ntwyzgrgsxto	200000000	database administrator	2012.0	

	ctc_updated_year	total_experience	Hike	Hike_From_Same_Company	\
59098	2018.0	6.0	0.0	0.0	
600	2020.0	11.0	0.0	0.0	
2295	2020.0	10.0	0.0	0.0	
3187	2021.0	3.0	0.0	0.0	
3675	2020.0	6.0	0.0	0.0	
4502	2020.0	6.0	0.0	0.0	
6131	2020.0	8.0	0.0	0.0	
8434	2020.0	4.0	0.0	0.0	
8777	2020.0	7.0	0.0	0.0	
9311	2020.0	12.0	0.0	0.0	

	Is_Job_Position_Changed	...	tenure	ctc_median of 3	ctc_75th of 3	\
59098	0	...	6.0	128977777.5	1.922667e+08	
600	0	...	11.0	850000.0	1.000000e+06	
2295	0	...	10.0	150000000.0	1.750000e+08	
3187	0	...	3.0	1600000.0	2.050000e+06	
3675	0	...	6.0	200000000.0	2.000000e+08	
4502	0	...	6.0	200000000.0	2.000000e+08	
6131	0	...	8.0	500000.0	7.225000e+05	
8434	0	...	4.0	500000.0	7.600000e+05	
8777	0	...	7.0	420000.0	6.300000e+05	
9311	0	...	12.0	200000000.0	2.000000e+08	

	Designation	ctc_median of 2	ctc_75th of 2	Class	ctc_median of 1	\
59098	1.0	128977777.5	1.922667e+08	1.0	1955000.0	
600	1.0	700000.0	1.000000e+06	1.0	571500.0	
2295	1.0	1500000.0	5.400000e+07	1.0	1100000.0	
3187	1.0	1675000.0	2.050000e+06	1.0	1750000.0	
3675	2.0	200000000.0	2.000000e+08	2.0	620000.0	
4502	2.0	200000000.0	2.000000e+08	2.0	500000.0	
6131	1.0	450000.0	7.000000e+05	1.0	455000.0	
8434	1.0	500000.0	7.200000e+05	1.0	500000.0	

8777	1.0	400000.0	6.225000e+05	1.0	455000.0
9311	2.0	360000.0	1.012500e+06	1.0	700000.0

	ctc_75th of 1	Tier
59098	2350000.0	1.0
600	819999.0	1.0
2295	1900000.0	1.0
3187	2500000.0	1.0
3675	3700000.0	1.0
4502	840000.0	1.0
6131	700000.0	1.0
8434	720000.0	1.0
8777	700000.0	1.0
9311	1150000.0	1.0

[10 rows x 22 columns]

4.3.2 Top 10 employees of data science in each company earning more than their peers - Class 1

```
[676]: data_mc[(data_mc['job_position'].str.contains('(?!i)data scientist')) &
        ↪(data_mc['Class']==1) ].nlargest(n=10, columns=['ctc'])
```

```
[676]:                                     email_hash \
122634  bd222ea783ee372da4e0ad60fdccec0b8f37999a032025...
133281  cda8d723438e81185d2ee8c348870a4612eea974cdb2db...
69176   6ad86d120e39db485331f9a0b2b1f15ce2a7bdaee778ab...
69596   6b6dd66bae787dd4dd417e1777f8ea5a057257e9019995...
76239   75f5b46d47310c3923e93329a62a1aa78d478803f0a685...
54445   544e75b477f8644eb71281133c62c19732547837e80e51...
14372   15adaeb2eef9c0ee8a0f18e189bf426be390f5d1e911fd...
28680   2bede29959707d8c6f283d98319361c386baa6fa5c8028...
160747  f7b7c771ccdbbca7248002ba83f7a176baa974c2c7bb8f...
40344   3e290b892b73283b96293c53e4ce4dce2cc6a22399b95c...

                                     company_hash      ctc  job_position \
122634      ihvaqvnwx xzoxsyno ucn rna  200000000  data scientist
133281  mqxonrtwgzt v bvyxzaqv sqghu wgbuvzj  200000000  data scientist
69176      ntwy bvyxzaqv  100000000  data scientist
69596      xzzgcv ogrhnxgzo  100000000  data scientist
76239      wgqt wgbutnt  99990000  data scientist
54445      zgn vuurxwvmrt vwwghzn  99900000  data scientist
14372      zvsvqqg  60000000  data scientist
28680      bgqsvz onvzrtj  50000000  data scientist
160747      wxnx  24200000  data scientist
40344      sggsrt  22000000  data scientist
```

	orgyear	ctc_updated_year	total_experience	Hike	\
122634	2015.0	2019.0	9.0	0.0	
133281	2017.0	2020.0	7.0	0.0	
69176	2021.0	2021.0	3.0	0.0	
69596	2016.0	2020.0	8.0	0.0	
76239	2016.0	2021.0	8.0	98290000.0	
54445	2021.0	2021.0	3.0	0.0	
14372	2021.0	2021.0	3.0	0.0	
28680	2021.0	2021.0	3.0	0.0	
160747	2011.0	2020.0	13.0	0.0	
40344	2023.0	2023.0	1.0	0.0	

	Hike_From_Same_Company	Is_Job_Position_Changed	...	tenure	\
122634	0.0	0	...	9.0	
133281	0.0	0	...	7.0	
69176	0.0	0	...	3.0	
69596	0.0	0	...	8.0	
76239	98290000.0	0	...	8.0	
54445	0.0	0	...	3.0	
14372	0.0	0	...	3.0	
28680	0.0	0	...	3.0	
160747	0.0	0	...	13.0	
40344	0.0	0	...	1.0	

	ctc_median of 3	ctc_75th of 3	Designation	ctc_median of 2	\
122634	200000000.0	200000000.0	2.0	1560000.0	
133281	200000000.0	200000000.0	2.0	1400000.0	
69176	100000000.0	100000000.0	2.0	890000.0	
69596	50400000.0	75200000.0	1.0	50400000.0	
76239	50845000.0	75417500.0	1.0	50845000.0	
54445	1260000.0	3850000.0	1.0	950000.0	
14372	60000000.0	60000000.0	2.0	900000.0	
28680	50000000.0	50000000.0	2.0	3240000.0	
160747	24200000.0	24200000.0	2.0	2200000.0	
40344	22000000.0	22000000.0	2.0	1700000.0	

	ctc_75th of 2	Class	ctc_median of 1	ctc_75th of 1	Tier
122634	100780000.0	1.0	884999.5	1424999.75	1.0
133281	2500000.0	1.0	890000.0	1200000.00	1.0
69176	1422500.0	1.0	600000.0	994000.00	1.0
69596	75200000.0	1.0	920000.0	1312500.00	1.0
76239	75417500.0	1.0	2250000.0	2610000.00	1.0
54445	1755000.0	1.0	600000.0	1100000.00	1.0
14372	1800000.0	1.0	1000000.0	1500000.00	1.0
28680	26620000.0	1.0	1900000.0	2600000.00	1.0
160747	2500000.0	1.0	1590000.0	2100000.00	1.0
40344	3800000.0	1.0	2500000.0	3900000.00	1.0

[10 rows x 22 columns]

4.3.3 Bottom 10 employees of data science in each company earning less than their peers - Class 3

```
[677]: data_mc[(data_mc['job_position'].str.contains('(?!i)data scientist')) &
        (data_mc['Class']==3) ].nsmallest(n=10, columns=['ctc'])
```

```
[677]:
```

	email_hash \
110985	ab2dc9db23c3104f0b6b3dbd4cdd5bfb9e5829b8b7943d...
32125	3175d03fd4618eb293d6f5a1d13d42a0c79f68e9acaaa3...
35412	3675f79c7e05de96ccf189c818b84b487cb1aa3f6b80e8...
39430	3cc0c85d198d0e56a4cdefb6496333f59b97f87c293262...
84383	8274b3188470cd1c4914e7face490111e27f239457e62d...
163112	fb64af615420e06d46a1965f59068b34460fb3cbe70541...
156034	f091e63c9cc72c1159ad686e32a0a813a617976e44843e...
44849	455bb6d1cbeae00c7844261cbe61ead8fa2c4dd73a8021...
126003	c26de71bef1887f9d30e01e499422111cb6ffba2162838...
40638	3e97cebf7372ac75eadf150adcb41d623e68d3b91caf86...

	company_hash	ctc	job_position	orgyear \
110985	exznqhon ogrhnxgzo ucn rna	7200	data scientist	2017.0
32125	nvnv wgzohrnvwzj otqcxwto	7500	data scientist	2020.0
35412	vqxosrgmvr	8800	data scientist	2015.0
39430	uvjovet sqghu	10000	data scientist	2018.0
84383	ytfrtnn uvwpvqa tzntquqxot	10000	data scientist	2018.0
163112	sggsrt	10000	data scientist	2018.0
156034	hmtq	10800	data scientist	2017.0
44849	hztburgjta	18000	data scientist	2014.0
126003	grv vzx ntwyzgrgsxto ucn rna	18000	data scientist	2010.0
40638	nqtatzwt xzw	20000	data scientist	2018.0

	ctc_updated_year	total_experience	Hike	Hike_From_Same_Company \
110985	2019.0	7.0	0.0	0.0
32125	2020.0	4.0	0.0	0.0
35412	2019.0	9.0	0.0	0.0
39430	2019.0	6.0	0.0	0.0
84383	2019.0	6.0	0.0	0.0
163112	2021.0	6.0	0.0	0.0
156034	2019.0	7.0	0.0	0.0
44849	2019.0	10.0	0.0	0.0
126003	2019.0	14.0	0.0	0.0
40638	2019.0	6.0	0.0	0.0

	Is_Job_Position_Changed	...	tenure	ctc_median of 3	ctc_75th of 3 \
110985	0	...	7.0	8600.0	9300.0

32125	0	...	4.0	345000.0	370000.0
35412	0	...	9.0	552900.0	824950.0
39430	0	...	6.0	705000.0	1052500.0
84383	0	...	6.0	2255000.0	3377500.0
163112	0	...	6.0	255000.0	377500.0
156034	0	...	7.0	10800.0	10800.0
44849	1	...	10.0	18000.0	18000.0
126003	0	...	14.0	18000.0	18000.0
40638	0	...	6.0	20000.0	20000.0

	Designation	ctc_median of 2	ctc_75th of 2	Class	ctc_median of 1 \
110985	3.0	8600.0	9300.00	3.0	8600.0
32125	3.0	500000.0	800000.00	3.0	455000.0
35412	3.0	552900.0	824950.00	3.0	850000.0
39430	3.0	705000.0	1052500.00	3.0	2000000.0
84383	3.0	1650000.0	2365000.00	3.0	1150000.0
163112	3.0	1700000.0	3800000.00	3.0	2500000.0
156034	2.0	755400.0	1127700.00	3.0	3500000.0
44849	2.0	103000.0	541500.00	3.0	300000.0
126003	2.0	1850000.0	2300000.00	3.0	1700000.0
40638	2.0	965000.0	1234999.75	3.0	865000.0

	ctc_75th of 1	Tier
110985	9300.0	3.0
32125	700000.0	3.0
35412	1048500.0	3.0
39430	2977500.0	3.0
84383	1700000.0	3.0
163112	3900000.0	3.0
156034	5200000.0	3.0
44849	700000.0	3.0
126003	2500000.0	3.0
40638	1275000.0	3.0

[10 rows x 22 columns]

4.3.4 Bottom 10 employees (earning less than most of the employees in the company)- Tier 3

```
[678]: data_mc[(data_mc['Tier'] == 3) ].nsmallest(n=10, columns=['ctc'])
```

```
[678]: email_hash \
34504 3505b02549ebe2c95840ac6f0a35561a3b4cbe4b79cdb1...
157442 f2b58aeed3c074652de2cfd3c0717a5d21d6fbcf342a78...
23306 23ad96d6b6f1ecf554a52f6e9b61677c7d73d8a409a143...
119769 b8a0bb340583936b5a7923947e9aec21add5ebc50cd60b...
160852 f7e5e788676100d7c4146740ada9e2f8974defc01f571d...
```

83267	80ba0259f9f59034c4927cf3bd38dc9ce2eb60ff18135b...
100383	9af3dca6c9d705d8d42585ccfce2627f00e1629130d14e...
120401	b995d7a2ae5c6f8497762ce04dc5c04ad6ec734d70802a...
8535	0cc27665f8a9700eeeeec545a88a3067b31a8d6ecb031a3...
14925	1694233be08738b7b50bdb7649b792f0ab8a514c01bec9...

	company_hash	ctc	job_position	orgyear	\
34504	xzntqcxtfmxn	2	backend engineer	2014.0	
157442	xzntqcxtfmxn	6	unknown	2013.0	
23306	xzntqcxtfmxn	14	unknown	2013.0	
119769	xm	15	unknown	2016.0	
160852	hzxctqoxnj ge fvoyxzsnzg	200	unknown	2022.0	
83267	nvvnv wgzohrnvwj otqcxwto	600	backend engineer	2012.0	
100383	zvz	600	unknown	2023.0	
120401	gjjg	600	fullstack engineer	2018.0	
8535	xehnhqt	1000	fullstack engineer	2015.0	
14925	sttpoegqsttpo	1000	unknown	2016.0	

	ctc_updated_year	total_experience	Hike	Hike_From_Same_Company	\
34504	2019.0	10.0	0.0	0.0	
157442	2018.0	11.0	0.0	0.0	
23306	2018.0	11.0	0.0	0.0	
119769	2018.0	8.0	0.0	0.0	
160852	2022.0	2.0	0.0	0.0	
83267	2017.0	12.0	0.0	0.0	
100383	2023.0	1.0	0.0	0.0	
120401	2021.0	6.0	0.0	-1649400.0	
8535	2018.0	9.0	0.0	0.0	
14925	2019.0	8.0	0.0	0.0	

	Is_Job_Position_Changed	...	tenure	ctc_median of 3	ctc_75th of 3	\
34504	0	...	10.0	1000001.0	1500000.5	
157442	0	...	11.0	10.0	12.0	
23306	0	...	11.0	10.0	12.0	
119769	0	...	8.0	15.0	15.0	
160852	0	...	2.0	200.0	200.0	
83267	0	...	12.0	720000.0	990000.0	
100383	0	...	1.0	2600000.0	3200000.0	
120401	0	...	6.0	1325000.0	1580000.0	
8535	0	...	9.0	1000.0	1000.0	
14925	0	...	8.0	1000.0	1000.0	

	Designation	ctc_median of 2	ctc_75th of 2	Class	ctc_median of 1	\
34504	3.0	735000.0	1225000.00	3.0	930000.0	
157442	3.0	1000000.0	1200000.00	3.0	930000.0	
23306	1.0	1000000.0	1200000.00	3.0	930000.0	
119769	2.0	15.5	15.75	3.0	15.5	

160852	2.0	200.0	200.00	2.0	4000.0
83267	3.0	460000.0	700000.00	3.0	455000.0
100383	3.0	1000000.0	1760000.00	3.0	900000.0
120401	3.0	1250000.0	1515000.00	3.0	1470000.0
8535	2.0	1000.0	1000.00	2.0	100500.0
14925	2.0	500000.0	1325000.00	3.0	705000.0

	ctc_75th of 1	Tier
34504	2000000.00	3.0
157442	2000000.00	3.0
23306	2000000.00	3.0
119769	15.75	3.0
160852	47000.00	3.0
83267	700000.00	3.0
100383	1600000.00	3.0
120401	2100000.00	3.0
8535	150250.00	3.0
14925	1143750.00	3.0

[10 rows x 22 columns]

4.3.5 Top 10 employees in each company - X department - having 5/6/7 years of experience earning more than their peers - Tier X

Tier = Class = Designation = 1

```
[679]: data_mc[(data_mc['Tier']==1) & (data_mc['Class']==1) &
        ↪(data_mc['Designation']==1) & (data_mc['total_experience'].isin([5,6,7]))].
        ↪nlargest(n=10, columns=['ctc'])
```

```
[679]:                                     email_hash \
59098  5b4bed51797140db4ed52018a979db1e34cee49e27b488...
8777   0d235f7e73cd9484909b32a35c69df12296a051f68ef83...
11154  10c67fa43291396c3f72c9ac34b99a6d9fb2c2007e6964...
16798  199146d59076620fa919a1b4e510303885a9b47d079243...
17604  1ad3d8c2b855e9cbe6bb187e8140a07ed8a8b29d275ae0...
18112  1b95e7ba0ee82100ca5a034239fa0203a1bec14280b82a...
18130  1b9cb9fd05dd794efc20da0f5e873f1afc92767306166b...
19392  1d8bbc3a2b8fb477f78ab3b1ca3f5a7ae0f256555e44ed...
22910  2311bf023218afe93d650cac03abb7a40f7fa55c08d260...
25660  2744c7f42fd4d492fa66cb2ba5168921c444dc8611ffa2...
```

	company_hash	ctc	job_position	orgyear	\
59098	obvqnuqxwgb	255555555	unknown	2018.0	
8777	nvnv wgzohrnvwj	otqcxwto	200000000	other	2017.0
11154	nvnv wgzohrnvwj	otqcxwto	200000000	unknown	2018.0
16798	nvnv wgzohrnvwj	otqcxwto	200000000	support engineer	2017.0
17604	yaew mvzp	200000000	other	2017.0	

18112	nvnv	wgzohrnvwj	otqcxwto	200000000	engineering leadership	2018.0
18130			psxor	200000000	other	2017.0
19392			vwwtznht	200000000	backend engineer	2019.0
22910			xwxwx mvzp	200000000	other	2018.0
25660			vwwtznht	200000000	backend engineer	2018.0

	ctc_updated_year	total_experience	Hike	Hike_From_Same_Company	\
59098	2018.0	6.0	0.0	0.0	
8777	2020.0	7.0	0.0	0.0	
11154	2020.0	6.0	0.0	0.0	
16798	2020.0	7.0	0.0	0.0	
17604	2020.0	7.0	0.0	0.0	
18112	2021.0	6.0	0.0	0.0	
18130	2020.0	7.0	0.0	0.0	
19392	2020.0	5.0	0.0	0.0	
22910	2020.0	6.0	0.0	0.0	
25660	2019.0	6.0	0.0	0.0	

	Is_Job_Position_Changed	...	tenure	ctc_median of 3	ctc_75th of 3	\
59098	0	...	6.0	128977777.5	1.922667e+08	
8777	0	...	7.0	420000.0	6.300000e+05	
11154	0	...	6.0	400000.0	7.000000e+05	
16798	0	...	7.0	480000.0	7.300000e+05	
17604	0	...	7.0	750000.0	7.600000e+05	
18112	0	...	6.0	145000000.0	1.725000e+08	
18130	0	...	7.0	100337500.0	1.501688e+08	
19392	0	...	5.0	499999.0	7.100000e+05	
22910	0	...	6.0	270000.0	5.022500e+07	
25660	0	...	6.0	540000.0	7.415000e+05	

	Designation	ctc_median of 2	ctc_75th of 2	Class	ctc_median of 1	\
59098	1.0	128977777.5	1.922667e+08	1.0	1955000.0	
8777	1.0	400000.0	6.225000e+05	1.0	455000.0	
11154	1.0	450000.0	7.000000e+05	1.0	455000.0	
16798	1.0	440000.0	6.175000e+05	1.0	455000.0	
17604	1.0	750000.0	1.080000e+06	1.0	750000.0	
18112	1.0	1000000.0	1.650000e+06	1.0	455000.0	
18130	1.0	57837500.0	1.362500e+08	1.0	636500.0	
19392	1.0	670000.0	1.002250e+06	1.0	620000.0	
22910	1.0	490000.0	8.725000e+05	1.0	600000.0	
25660	1.0	670000.0	1.002250e+06	1.0	620000.0	

	ctc_75th of 1	Tier
59098	2350000.00	1.0
8777	700000.00	1.0
11154	700000.00	1.0
16798	700000.00	1.0

17604	1200000.00	1.0
18112	700000.00	1.0
18130	29537500.00	1.0
19392	967499.25	1.0
22910	1350000.00	1.0
25660	967499.25	1.0

[10 rows x 22 columns]

Tier = Class = Designation = 2

```
[680]: data_mc[(data_mc['Tier']==2) & (data_mc['Class']==2) &
↳ (data_mc['Designation']==2) & (data_mc['total_experience'].isin([5,6,7]))].
↳ nlargest(n=10, columns=['ctc'])
```

[680]:

	email_hash \
--	--------------

6975	0a5eaf16728b44b9b5c8ac562df307860433f2fc7ab003...
9855	0ecafc13a8da3fd0bfee332e6d9a8dc0b048dfa35fee1c...
13389	1425506ecb5764fdc996a0f0da6a2fdde6acbb59098dc3...
16714	19719e6ee77db17b6a3a9e9a2b8a786aee4b865e656ff7...
21714	214035fc90945d84b9772dab3fce7fe328b96677d84bf0...
22956	2323f80afa6f3c809ac468997c1cf1ea8572d06bd8904e...
23503	23f778fcb9c8c1cfc177fa5a1c892feca9e24e069e57f5...
28278	2b649949f0a00c444db6ae38d66a972e37cc3a90ba65a0...
32187	318b9bd7c00bf13cd65c930abf7cdde60fd3173e01d242...
36953	38cb5dc5ef81bc9589d50cdde0b93d1d13e91f4d874e97...

	company_hash	ctc	job_position \
--	--------------	-----	----------------

6975	i wgzztin mhoxztoo ogrhnxgzo ucn rna	200000000	other
9855	ctqexohayv ntwyzgrgsxto	200000000	unknown
13389	xxtb	200000000	unknown
16714	touxqxn ntwyzgrgsxto ucnrna	200000000	data analyst
21714	onttr onqxuo fyttro rxbxnta	200000000	unknown
22956	wgbutntzn ojointbo xzw	200000000	other
23503	ztnowqxmto	200000000	other
28278	ihvrxnvo srgmvr rru	200000000	qa engineer
32187	nvrtnz ouvwt xzw	200000000	unknown
36953	wjzzgd	200000000	fullstack engineer

	orgyear	ctc_updated_year	total_experience	Hike \
--	---------	------------------	------------------	--------

6975	2017.0	2020.0	7.0	0.0
9855	2018.0	2020.0	6.0	0.0
13389	2018.0	2020.0	6.0	0.0
16714	2017.0	2020.0	7.0	0.0
21714	2019.0	2019.0	5.0	0.0
22956	2017.0	2020.0	7.0	0.0
23503	2018.0	2020.0	6.0	0.0

28278	2017.0	2020.0	7.0	0.0
32187	2019.0	2020.0	5.0	0.0
36953	2018.0	2020.0	6.0	0.0

	Hike_From_Same_Company	Is_Job_Position_Changed	...	tenure	\
6975	0.0	0	...	7.0	
9855	0.0	0	...	6.0	
13389	0.0	0	...	6.0	
16714	0.0	0	...	7.0	
21714	0.0	0	...	5.0	
22956	0.0	0	...	7.0	
23503	0.0	0	...	6.0	
28278	0.0	0	...	7.0	
32187	0.0	0	...	5.0	
36953	0.0	0	...	6.0	

	ctc_median of 3	ctc_75th of 3	Designation	ctc_median of 2	\
6975	200000000.0	200000000.0	2.0	200000000.0	
9855	200000000.0	200000000.0	2.0	200000000.0	
13389	200000000.0	200000000.0	2.0	200000000.0	
16714	200000000.0	200000000.0	2.0	200000000.0	
21714	200000000.0	200000000.0	2.0	200000000.0	
22956	200000000.0	200000000.0	2.0	200000000.0	
23503	200000000.0	200000000.0	2.0	200000000.0	
28278	200000000.0	200000000.0	2.0	200000000.0	
32187	200000000.0	200000000.0	2.0	200000000.0	
36953	200000000.0	200000000.0	2.0	200000000.0	

	ctc_75th of 2	Class	ctc_median of 1	ctc_75th of 1	Tier
6975	200000000.0	2.0	200000000.0	200000000.0	2.0
9855	200000000.0	2.0	200000000.0	200000000.0	2.0
13389	200000000.0	2.0	200000000.0	200000000.0	2.0
16714	200000000.0	2.0	200000000.0	200000000.0	2.0
21714	200000000.0	2.0	200000000.0	200000000.0	2.0
22956	200000000.0	2.0	200000000.0	200000000.0	2.0
23503	200000000.0	2.0	200000000.0	200000000.0	2.0
28278	200000000.0	2.0	200000000.0	200000000.0	2.0
32187	200000000.0	2.0	200000000.0	200000000.0	2.0
36953	200000000.0	2.0	200000000.0	200000000.0	2.0

[10 rows x 22 columns]

Tier = Class = Designation = 3

```
[681]: data_mc[(data_mc['Tier']==3) & (data_mc['Class']==3) &
↪(data_mc['Designation']==3) & (data_mc['total_experience'].isin([5,6,7]))].
↪nlargest(n=10, columns=['ctc'])
```

[681]:

	email_hash \
16763	19839c1100097c37582a06a4dbb8ec28194883f0c6b70f...
21157	205fd39eb7318279f75b0b292f6b05a16f8666f213193b...
131667	cb28bd7e524939d0ec9e035780933234a87ec78ff5eeca...
50978	4ef563eb48a30bdbcc35c2bed8c5e8d179ee2d2b12c3a4...
37730	3a0943675fbbab95a8ccd256def6d057e53adbe2441887...
89569	8a6c52a8e7f59829e0ac0af8b9e1c06915b838003665e6...
105520	a2cb1183178925f56edb878ada880de338c93a3d0ae163...
89568	8a6c52a8e7f59829e0ac0af8b9e1c06915b838003665e6...
82228	7f1e3b950800344cb0a887fc8a3df004bd1cd9b58618d6...
99605	99b87cae8504feb6130c30f4f48e2633b20b20ce5cad6b...

	company_hash	ctc	job_position	orgyear \
16763	wvqttb	5600000	backend engineer	2017.0
21157	oeq mhoxztoo	3500000	other	2017.0
131667	qhmqxp xzw	3300000	backend engineer	2017.0
50978	vruyvsqtu otwhqxnxt	3250000	backend engineer	2017.0
37730	hmtq	3200000	backend engineer	2018.0
89569	hmtq	3200000	unknown	2019.0
105520	qhmqxp xzw	3200000	backend engineer	2017.0
89568	hmtq	3100000	unknown	2019.0
82228	hmtq	3000000	backend engineer	2017.0
99605	vsgav	3000000	backend engineer	2017.0

	ctc_updated_year	total_experience	Hike	Hike_From_Same_Company \
16763	2020.0	7.0	0.0	0.0
21157	2019.0	7.0	0.0	0.0
131667	2019.0	7.0	0.0	0.0
50978	2020.0	7.0	0.0	0.0
37730	2020.0	6.0	0.0	0.0
89569	2020.0	5.0	100000.0	100000.0
105520	2019.0	7.0	0.0	0.0
89568	2020.0	5.0	0.0	0.0
82228	2021.0	7.0	0.0	0.0
99605	2020.0	7.0	0.0	0.0

	Is_Job_Position_Changed	...	tenure	ctc_median of 3	ctc_75th of 3 \
16763	0	...	7.0	8600000.0	10100000.0
21157	0	...	7.0	3550000.0	3575000.0
131667	0	...	7.0	5250000.0	5700000.0
50978	0	...	7.0	4250000.0	4475000.0
37730	0	...	6.0	3300000.0	4392500.0
89569	0	...	5.0	3300000.0	3700000.0
105520	0	...	7.0	5250000.0	5700000.0
89568	0	...	5.0	3300000.0	3700000.0
82228	0	...	7.0	3400000.0	4025000.0
99605	0	...	7.0	4000000.0	4500000.0

	Designation	ctc_median of 2	ctc_75th of 2	Class	ctc_median of 1 \
16763	3.0	7100000.0	8225000.0	3.0	7100000.0
21157	3.0	3550000.0	3575000.0	3.0	3550000.0
131667	3.0	4750000.0	5212500.0	3.0	3700000.0
50978	3.0	4525000.0	5025000.0	3.0	4450000.0
37730	3.0	3925000.0	5300000.0	3.0	3500000.0
89569	3.0	3585000.0	5075000.0	3.0	3500000.0
105520	3.0	4750000.0	5212500.0	3.0	3700000.0
89568	3.0	3585000.0	5075000.0	3.0	3500000.0
82228	3.0	3925000.0	5300000.0	3.0	3500000.0
99605	3.0	3200000.0	4705000.0	3.0	3200000.0

	ctc_75th of 1	Tier
16763	8625000.0	3.0
21157	3575000.0	3.0
131667	4950000.0	3.0
50978	4662500.0	3.0
37730	5200000.0	3.0
89569	5200000.0	3.0
105520	4950000.0	3.0
89568	5200000.0	3.0
82228	5200000.0	3.0
99605	4477500.0	3.0

[10 rows x 22 columns]

4.3.6 Top 10 companies (based on their CTC)

```
[682]: data_mc.sort_values(['ctc_median of 1','ctc_75th of 1'], ascending=False).
        head(10)
```

```
[682]:                                     email_hash \
234      005f352591238b78024ed320789c30e7815f0eea52ed98...
6975      0a5eaf16728b44b9b5c8ac562df307860433f2fc7ab003...
9388      0e1a0935c6c942c79ef6a390e06645f2a51f6285a60ef3...
9855      0ecafc13a8da3fd0bfee332e6d9a8dc0b048dfa35fee1c...
13389     1425506ecb5764fdc996a0f0da6a2fdde6acbb59098dc3...
15686     17cf7525d32a949efa43290fa4a95ab31d285225e61d3c...
16714     19719e6ee77db17b6a3a9e9a2b8a786aee4b865e656ff7...
20591     1f6eb9a9689f6d1c45352d5f290cb0fb799671c34b10f2...
20995     201f50d76d5a52d91e78d7dc126fc2aeeb9868b19dd922...
21714     214035fc90945d84b9772dab3fce7fe328b96677d84bf0...
```

	company_hash	ctc \
234	vbvkgz wvzvav	200000000
6975	i wgzztin mhoxztoo ogrhnxgzo ucn rna	200000000

9388	ntwyzguvqp xnqvxnk xn ogrhnxgz	200000000
9855	ctqexohayv ntwyzgrgsxto	200000000
13389	xxtb	200000000
15686	ehlxonh	200000000
16714	touxqxn ntwyzgrgsxto ucnrna	200000000
20591	qn	200000000
20995	egd z wrgha ntwyzgrgsxto	200000000
21714	onttr onqxuo fyttro rxbxnta	200000000

	job_position	orgyear	ctc_updated_year	total_experience	\
234	unknown	2020.0	2020.0	4.0	
6975	other	2017.0	2020.0	7.0	
9388	database administrator	2015.0	2020.0	9.0	
9855	unknown	2018.0	2020.0	6.0	
13389	unknown	2018.0	2020.0	6.0	
15686	frontend engineer	2013.0	2020.0	11.0	
16714	data analyst	2017.0	2020.0	7.0	
20591	engineering intern	2012.0	2020.0	12.0	
20995	qa engineer	2015.0	2020.0	9.0	
21714	unknown	2019.0	2019.0	5.0	

	Hike	Hike_From_Same_Company	Is_Job_Position_Changed	...	tenure	\
234	0.0	0.0	0	...	4.0	
6975	0.0	0.0	0	...	7.0	
9388	0.0	0.0	0	...	9.0	
9855	0.0	0.0	0	...	6.0	
13389	0.0	0.0	0	...	6.0	
15686	0.0	0.0	0	...	11.0	
16714	0.0	0.0	0	...	7.0	
20591	0.0	0.0	0	...	12.0	
20995	0.0	0.0	0	...	9.0	
21714	0.0	0.0	0	...	5.0	

	ctc_median of 3	ctc_75th of 3	Designation	ctc_median of 2	\
234	200000000.0	200000000.0	2.0	200000000.0	
6975	200000000.0	200000000.0	2.0	200000000.0	
9388	200000000.0	200000000.0	2.0	200000000.0	
9855	200000000.0	200000000.0	2.0	200000000.0	
13389	200000000.0	200000000.0	2.0	200000000.0	
15686	200000000.0	200000000.0	2.0	200000000.0	
16714	200000000.0	200000000.0	2.0	200000000.0	
20591	200000000.0	200000000.0	2.0	200000000.0	
20995	200000000.0	200000000.0	2.0	200000000.0	
21714	200000000.0	200000000.0	2.0	200000000.0	

	ctc_75th of 2	Class	ctc_median of 1	ctc_75th of 1	Tier
234	200000000.0	2.0	200000000.0	200000000.0	2.0

6975	200000000.0	2.0	200000000.0	200000000.0	2.0
9388	200000000.0	2.0	200000000.0	200000000.0	2.0
9855	200000000.0	2.0	200000000.0	200000000.0	2.0
13389	200000000.0	2.0	200000000.0	200000000.0	2.0
15686	200000000.0	2.0	200000000.0	200000000.0	2.0
16714	200000000.0	2.0	200000000.0	200000000.0	2.0
20591	200000000.0	2.0	200000000.0	200000000.0	2.0
20995	200000000.0	2.0	200000000.0	200000000.0	2.0
21714	200000000.0	2.0	200000000.0	200000000.0	2.0

[10 rows x 22 columns]

4.3.7 Top 10 positions in every company (based on their CTC)

```
[683]: data_mc.sort_values(['ctc_median of 2','ctc_75th of 2'], ascending=False).
        head(10)
```

```
[683]:                                     email_hash \
234      005f352591238b78024ed320789c30e7815f0eea52ed98...
3675      05a0aec8db0d251460a2e200d835b7818e050308ef1e67...
4502      06c7cd6c1a50803bac81950ad450a0e78165cb7c09ddfe...
6975      0a5eaf16728b44b9b5c8ac562df307860433f2fc7ab003...
9388      0e1a0935c6c942c79ef6a390e06645f2a51f6285a60ef3...
9855      0ecafc13a8da3fd0bfee332e6d9a8dc0b048dfa35fee1c...
10029     0f0e7a9db34d1317498d9378a1bd0150bfa022ac6ba3a0...
13389     1425506ecb5764fdc996a0f0da6a2fdde6acbb59098dc3...
15686     17cf7525d32a949efa43290fa4a95ab31d285225e61d3c...
15701     17d7a7983d8f944459e2d114a99f68634447e552f7aff0...
```

```
                                     company_hash      ctc \
234                                     vbvkgz wvzvav  200000000
3675                                     egdwgzz  200000000
4502                                     gnytqo  200000000
6975      i wgzztin mhoxztoo ogrhnxgzo ucn rna  200000000
9388          ntwyzguvqp xnqvxnk xn ogrhnxgz  200000000
9855          ctqexohayv ntwyzgrgsxto  200000000
10029          qhhttp  200000000
13389          xxtb  200000000
15686          ehlxonh  200000000
15701          vao ogrhnxgz  200000000
```

```
          job_position  orgyear  ctc_updated_year  total_experience \
234          unknown    2020.0          2020.0          4.0
3675  engineering leadership    2018.0          2020.0          6.0
4502      product manager    2018.0          2020.0          6.0
6975          other    2017.0          2020.0          7.0
9388  database administrator    2015.0          2020.0          9.0
```


9855	unknown	2018.0	2020.0	6.0
10029	engineering leadership	2013.0	2020.0	11.0
13389	unknown	2018.0	2020.0	6.0
15686	frontend engineer	2013.0	2020.0	11.0
15701	other	2017.0	2020.0	7.0

	Hike	Hike_From_Same_Company	Is_Job_Position_Changed	...	tenure	\
234	0.0	0.0	0	...	4.0	
3675	0.0	0.0	0	...	6.0	
4502	0.0	0.0	0	...	6.0	
6975	0.0	0.0	0	...	7.0	
9388	0.0	0.0	0	...	9.0	
9855	0.0	0.0	0	...	6.0	
10029	0.0	0.0	0	...	11.0	
13389	0.0	0.0	0	...	6.0	
15686	0.0	0.0	0	...	11.0	
15701	0.0	0.0	0	...	7.0	

	ctc_median of 3	ctc_75th of 3	Designation	ctc_median of 2	\
234	200000000.0	200000000.0	2.0	200000000.0	
3675	200000000.0	200000000.0	2.0	200000000.0	
4502	200000000.0	200000000.0	2.0	200000000.0	
6975	200000000.0	200000000.0	2.0	200000000.0	
9388	200000000.0	200000000.0	2.0	200000000.0	
9855	200000000.0	200000000.0	2.0	200000000.0	
10029	200000000.0	200000000.0	2.0	200000000.0	
13389	200000000.0	200000000.0	2.0	200000000.0	
15686	200000000.0	200000000.0	2.0	200000000.0	
15701	200000000.0	200000000.0	2.0	200000000.0	

	ctc_75th of 2	Class	ctc_median of 1	ctc_75th of 1	Tier
234	200000000.0	2.0	200000000.0	200000000.0	2.0
3675	200000000.0	2.0	620000.0	3700000.0	1.0
4502	200000000.0	2.0	500000.0	840000.0	1.0
6975	200000000.0	2.0	200000000.0	200000000.0	2.0
9388	200000000.0	2.0	200000000.0	200000000.0	2.0
9855	200000000.0	2.0	200000000.0	200000000.0	2.0
10029	200000000.0	2.0	950000.0	1600000.0	1.0
13389	200000000.0	2.0	200000000.0	200000000.0	2.0
15686	200000000.0	2.0	200000000.0	200000000.0	2.0
15701	200000000.0	2.0	500000.0	100250000.0	1.0

[10 rows x 22 columns]

4.3.8 Drop unwanted features for Clustering

```
[684]: data_mc.head()
```

```
[684]:
```

	email_hash	company_hash	\
0	00003288036a44374976948c327f246fdbdf0778546904...	bxwqgogen	
1	0000aaa0e6b61f7636af1954b43d294484cd151c9b3cf6...	nqsn axsxnv	
2	0000d58fbc18012bf6fa2605a7b0357d126ee69bc41032...	gunhb	
3	000120d0c8aa304fcf12ab4b85e21feb80a342cfea03d4...	bxwqgotbx wgqugqvnxyz	
4	00014d71a389170e668ba96ae8e1f9d991591acc899025...	fvrbyqn rvmo	

	ctc	job_position	orgyear	ctc_updated_year	total_experience	\
0	3500000	backend engineer	2012.0	2019.0	12.0	
1	250000	backend engineer	2013.0	2020.0	11.0	
2	1300000	fullstack engineer	2021.0	2021.0	3.0	
3	2000000	fullstack engineer	2004.0	2021.0	20.0	
4	3400000	unknown	2009.0	2018.0	15.0	

	Hike	Hike_From_Same_Company	Is_Job_Position_Changed	...	tenure	\
0	0.0	0.0	0	...	12.0	
1	0.0	0.0	0	...	11.0	
2	0.0	0.0	0	...	3.0	
3	0.0	0.0	0	...	20.0	
4	0.0	0.0	0	...	15.0	

	ctc_median of 3	ctc_75th of 3	Designation	ctc_median of 2	\
0	2800000.0	3425000.0	1.0	2600000.0	
1	250000.0	250000.0	2.0	250000.0	
2	1300000.0	1431000.0	2.0	1200000.0	
3	2000000.0	2000000.0	2.0	1067500.0	
4	3900000.0	3950000.0	3.0	2127500.0	

	ctc_75th of 2	Class	ctc_median of 1	ctc_75th of 1	Tier
0	3600000.0	2.0	2500000.0	3500000.0	2.0
1	250000.0	2.0	250000.0	250000.0	2.0
2	1423250.0	2.0	1200000.0	1625000.0	2.0
3	1533750.0	1.0	1625000.0	2170000.0	2.0
4	3262500.0	1.0	2500000.0	3375000.0	1.0

```
[5 rows x 22 columns]
```

```
[685]: data_mc.drop(columns=['email_hash', 'ctc_median of 1', 'ctc_75th of 1',  
    ↪ 'ctc_median of 2', 'ctc_75th of 2', 'ctc_median of 3', 'ctc_75th of 3'],  
    ↪ inplace=True)
```

```
[686]: data_mc.duplicated().sum()
```

[686]: 8852

```
[687]: data_mc.drop_duplicates(inplace=True)
```

```
[688]: data_mc.duplicated().sum()
```

[688]: 0

```
[689]: data_mc.isna().sum()
```

```
[689]: company_hash      0
      ctc                0
      job_position      0
      orgyear           0
      ctc_updated_year   0
      total_experience    0
      Hike               0
      Hike_From_Same_Company 0
      Is_Job_Position_Changed 0
      Is_CTC_Changed     0
      Working_in_multiple_companies 0
      tenure             0
      Designation        0
      Class              0
      Tier               0
      dtype: int64
```

```
[690]: data_mc.to_csv("data_for_cluster_modeling.csv", index=False)
```

5 CHAPTER 5 : DATA PREPROCESSING FOR UNSUPER- VISED CLUSTERING ALGORITHMS

```
[57]: data_mc = pd.read_csv("data_for_cluster_modeling.csv")
```

```
[58]: data_mc.head()
```

```
[58]:
```

	company_hash	ctc	job_position	orgyear	\
0	bxwqgogen	3500000	backend engineer	2012.0	
1	nqsn axsnvr	250000	backend engineer	2013.0	
2	gunhb	1300000	fullstack engineer	2021.0	
3	bxwqgotbx wgqugqvnxz	2000000	fullstack engineer	2004.0	
4	fvrbvqn rvmo	3400000	unknown	2009.0	

	ctc_updated_year	total_experience	Hike	Hike_From_Same_Company	\
0	2019.0	12.0	0.0	0.0	
1	2020.0	11.0	0.0	0.0	

2	2021.0	3.0	0.0	0.0
3	2021.0	20.0	0.0	0.0
4	2018.0	15.0	0.0	0.0

	Is_Job_Position_Changed	Is_CTC_Changed	Working_in_multiple_companies	\
0	0	0	1	
1	0	0	1	
2	0	0	1	
3	0	0	1	
4	0	0	1	

	tenure	Designation	Class	Tier
0	12.0	1.0	2.0	2.0
1	11.0	2.0	2.0	2.0
2	3.0	2.0	2.0	2.0
3	20.0	2.0	1.0	2.0
4	15.0	3.0	1.0	1.0

```
[59]: data_mc.isna().sum()
```

```
[59]: company_hash      0
      ctc                0
      job_position      0
      orgyear           0
      ctc_updated_year  0
      total_experience   0
      Hike              0
      Hike_From_Same_Company  0
      Is_Job_Position_Changed  0
      Is_CTC_Changed    0
      Working_in_multiple_companies  0
      tenure            0
      Designation       0
      Class             0
      Tier              0
      dtype: int64
```

```
[60]: data_mc.duplicated().sum()
```

```
[60]: 0
```

```
[61]: data_mc.shape
```

```
[61]: (157200, 15)
```

5.1 PREPROCESSING FOR ML MODELING

5.1.1 Dropping the orgyear and ctc_updated_year

```
[62]: data_mc.drop(columns = ["orgyear","ctc_updated_year"], inplace = True)
```

5.1.2 Train Test Split

```
[63]: X_train, X_test = train_test_split(data_mc, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape)
```

```
(125760, 13) (31440, 13)
```

5.1.3 Catboost Encoding

```
[64]: cbe_encoder = CatBoostEncoder(random_state=42,a = 0.9)
cbe_encoder.fit(X_train, X_train['ctc'])
```

```
[64]: CatBoostEncoder(a=0.9, cols=['company_hash', 'job_position'], random_state=42)
```

```
[65]: X_train_encoded = cbe_encoder.transform(X_train)
X_test_encoded = cbe_encoder.transform(X_test)
```

```
[66]: X_train_encoded.head()
```

```
[66]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
93482	2.491432e+06	940000	2.262000e+06	7.0	0.0	
132492	6.291953e+06	740000	4.522371e+06	8.0	0.0	
150480	2.549682e+06	401999	1.988036e+06	6.0	0.0	
7842	1.720656e+06	1500000	1.988036e+06	8.0	0.0	
137751	4.264246e+06	250000	2.726392e+06	4.0	0.0	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
93482	0.0	0	0	
132492	0.0	0	0	
150480	0.0	0	0	
7842	0.0	0	0	
137751	0.0	0	0	

	Working_in_multiple_companies	tenure	Designation	Class	Tier
93482	1	7.0	2.0	2.0	2.0
132492	1	8.0	2.0	3.0	3.0
150480	1	6.0	3.0	3.0	3.0
7842	1	8.0	2.0	2.0	2.0
137751	1	4.0	3.0	3.0	3.0

```
[67]: X_test_encoded.head()
```

```
[67]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
91854	2.491432e+06	50000	1.912263e+06	7.0	0.0	
72420	2.491432e+06	3479999	3.982313e+06	20.0	0.0	
134453	1.825324e+06	2900000	3.982313e+06	18.0	0.0	
10222	2.491432e+06	100000	1.468023e+06	12.0	0.0	
135671	1.994070e+06	1680000	2.726392e+06	9.0	0.0	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
91854	0.0	0	0	
72420	0.0	0	0	
134453	0.0	0	0	
10222	0.0	0	0	
135671	0.0	0	0	

	Working_in_multiple_companies	tenure	Designation	Class	Tier
91854	1	7.0	2.0	2.0	2.0
72420	1	20.0	2.0	2.0	2.0
134453	1	18.0	2.0	2.0	1.0
10222	1	12.0	2.0	2.0	2.0
135671	1	9.0	2.0	2.0	3.0

```
[68]: print("companies in X_train",X_train["company_hash"].nunique())
print("job_positions in X_train",X_train["job_position"].nunique())
print("companies in X_train_encoded",X_test_encoded["company_hash"].nunique())
print("job_positions in X_train_encoded",X_test_encoded["job_position"].
↪nunique())
```

```
companies in X_train 31591
job_positions in X_train 539
companies in X_train_encoded 3216
job_positions in X_train_encoded 70
```

```
[69]: with open('catboost_encoder.pkl', 'wb') as f:
pickle.dump(cbe_encoder, f)
```

```
[70]: # # To load the model later
# with open('catboost_encoder.pkl', 'rb') as file:
#     cbe_encoder = pickle.load(file)
```

```
[71]: feature_names_list = X_train_encoded.columns.tolist()
feature_names_list
```

```
[71]: ['company_hash',
'ctc',
'job_position',
'total_experience',
'Hike',
```

```
'Hike_From_Same_Company',
'Is_Job_Position_Changed',
'Is_CTC_Changed',
'Working_in_multiple_companies',
'tenure',
'Designation',
'Class',
'Tier']
```

```
[72]: with open('feature_names_list.pkl', 'wb') as f:
      pickle.dump(feature_names_list, f)
```

```
[73]: # # To load the model later
      # with open('feature_names_list.pkl', 'rb') as file:
      #     feature_names_list = pickle.load(file)
```

5.1.4 Standard Scaling

```
[74]: std_scaler = StandardScaler()
      std_scaler.fit(X_train_encoded)
      X_train_scaled = std_scaler.transform(X_train_encoded)
      X_test_scaled = std_scaler.transform(X_test_encoded)
```

```
[75]: X_train_scaled = pd.DataFrame(X_train_scaled, columns=feature_names_list)
      X_test_scaled = pd.DataFrame(X_test_scaled, columns=feature_names_list)
```

```
[76]: X_train_scaled.head()
```

```
[76]:  company_hash      ctc  job_position  total_experience      Hike  \
0      0.028608 -0.122791    -0.217902    -0.514316 -0.015194
1      1.611623 -0.138620     1.936612    -0.283234 -0.015194
2      0.052871 -0.165372    -0.479036    -0.745398 -0.015194
3     -0.292440 -0.078469    -0.479036    -0.283234 -0.015194
4      0.767031 -0.177402     0.224742    -1.207562 -0.015194

      Hike_From_Same_Company  Is_Job_Position_Changed  Is_CTC_Changed  \
0                -0.002777                -0.208743        -0.098114
1                -0.002777                -0.208743        -0.098114
2                -0.002777                -0.208743        -0.098114
3                -0.002777                -0.208743        -0.098114
4                -0.002777                -0.208743        -0.098114

      Working_in_multiple_companies  tenure  Designation      Class      Tier
0                -0.298125 -0.507774    -0.098588 -0.163924 -0.211203
1                -0.298125 -0.276840    -0.098588  1.250395  1.103237
2                -0.298125 -0.738708     1.545323  1.250395  1.103237
3                -0.298125 -0.276840    -0.098588 -0.163924 -0.211203
```

4	-0.298125	-1.200576	1.545323	1.250395	1.103237
---	-----------	-----------	----------	----------	----------

```
[77]: with open('std_scaler.pkl', 'wb') as f:
      pickle.dump(std_scaler, f)
```

```
[78]: # # To load the model later
      # with open('std_scaler.pkl', 'rb') as file:
      #     std_scaler = pickle.load(file)
```

5.2 ADDING is_outlier USING ISOLATION FOREST

```
[84]: isolation_forest = IsolationForest(contamination=0.1, random_state=42)
      isolation_forest.fit(X_train_scaled)
```

```
[84]: IsolationForest(contamination=0.1, random_state=42)
```

```
[85]: X_train_outlier = X_train_scaled.copy(deep = True)
      X_test_outlier = X_test_scaled.copy(deep = True)
```

```
[86]: X_train_outlier["is_outlier"] = isolation_forest.predict(X_train_scaled)
      X_test_outlier["is_outlier"] = isolation_forest.predict(X_test_scaled)
```

```
[87]: X_train_outlier.head()
```

```
[87]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
0	0.028608	-0.122791	-0.217902	-0.514316	-0.015194	
1	1.611623	-0.138620	1.936612	-0.283234	-0.015194	
2	0.052871	-0.165372	-0.479036	-0.745398	-0.015194	
3	-0.292440	-0.078469	-0.479036	-0.283234	-0.015194	
4	0.767031	-0.177402	0.224742	-1.207562	-0.015194	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
0	-0.002777	-0.208743	-0.098114	
1	-0.002777	-0.208743	-0.098114	
2	-0.002777	-0.208743	-0.098114	
3	-0.002777	-0.208743	-0.098114	
4	-0.002777	-0.208743	-0.098114	

	Working_in_multiple_companies	tenure	Designation	Class	Tier	\
0	-0.298125	-0.507774	-0.098588	-0.163924	-0.211203	
1	-0.298125	-0.276840	-0.098588	1.250395	1.103237	
2	-0.298125	-0.738708	1.545323	1.250395	1.103237	
3	-0.298125	-0.276840	-0.098588	-0.163924	-0.211203	
4	-0.298125	-1.200576	1.545323	1.250395	1.103237	

	is_outlier
0	1


```

1      1
2      1
3      1
4      1

```

```
[88]: X_test_outlier.head()
```

```

[88]:   company_hash      ctc  job_position  total_experience      Hike  \
0      0.028608 -0.193232    -0.551260      -0.514316 -0.015194
1      0.028608  0.078242     1.421846       2.489751 -0.015194
2     -0.248843  0.032337     1.421846       2.027587 -0.015194
3      0.028608 -0.189274    -0.974696       0.641095 -0.015194
4     -0.178556 -0.064222     0.224742      -0.052151 -0.015194

      Hike_From_Same_Company  Is_Job_Position_Changed  Is_CTC_Changed  \
0                -0.002777                -0.208743        -0.098114
1                -0.002777                -0.208743        -0.098114
2                -0.002777                -0.208743        -0.098114
3                -0.002777                -0.208743        -0.098114
4                -0.002777                -0.208743        -0.098114

      Working_in_multiple_companies      tenure  Designation      Class      Tier  \
0                -0.298125 -0.507774    -0.098588 -0.163924 -0.211203
1                -0.298125  2.494369    -0.098588 -0.163924 -0.211203
2                -0.298125  2.032501    -0.098588 -0.163924 -1.525643
3                -0.298125  0.646896    -0.098588 -0.163924 -0.211203
4                -0.298125 -0.045906    -0.098588 -0.163924  1.103237

      is_outlier
0              1
1              1
2              1
3              1
4              1

```

```
[89]: X_train_outlier["is_outlier"].value_counts()
```

```

[89]: is_outlier
1      113184
-1      12576
Name: count, dtype: int64

```

```
[90]: X_test_outlier["is_outlier"].value_counts()
```

```

[90]: is_outlier
1      28260
-1      3180

```

Name: count, dtype: int64

```
[92]: with open('isolation_forest.pkl', 'wb') as f:
      pickle.dump(isolation_forest, f)
```

```
[93]: # # To load the model later
      # with open('isolation_forest.pkl', 'rb') as file:
      #     isolation_forest = pickle.load(file)
```

```
[94]: X_train_outlier.to_csv("X_train_final.csv", index=False)
      X_test_outlier.to_csv("X_test_final.csv", index=False)
```

5.2.1 Outliers UMAP Visualization

```
[159]: outlier_umap = UMAP(init = "random", n_neighbors = 200, min_dist = 0.1,
      ↪ random_state = 42, n_jobs = -1)
      outlier_train_umap = outlier_umap.
      ↪ fit_transform(X_train_outlier[['company_hash', 'ctc', 'job_position',
      ↪ 'total_experience', 'Hike',
      'Hike_From_Same_Company', 'Is_Job_Position_Changed', 'Is_CTC_Changed',
      'Working_in_multiple_companies', 'tenure', 'Designation', 'Class',
      'Tier']])
      outlier_test_umap = outlier_umap.transform(X_test_outlier[['company_hash',
      ↪ 'ctc', 'job_position', 'total_experience', 'Hike',
      'Hike_From_Same_Company', 'Is_Job_Position_Changed', 'Is_CTC_Changed',
      'Working_in_multiple_companies', 'tenure', 'Designation', 'Class',
      'Tier']])
```

C:\Users\saina\AppData\Roaming\Python\Python312\site-packages\umap\umap.py:1945: UserWarning:

n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

```
[161]: def
      ↪ create_outlier_umap(outlier_train_umap, y_train_pred, outlier_test_umap, y_test_pred):
      ↪
      # Create a figure with 1 row and 2 columns
      fig, axs = plt.subplots(1, 2, figsize=(12, 6))
      train_cmap = ListedColormap(sns.husl_palette(len(np.unique(y_train_pred))))

      # Plot the training data
      im_train = axs[0].scatter(outlier_train_umap[:, 0],
                                outlier_train_umap[:, 1],
                                s=25,
                                c=np.array(y_train_pred),
```

```

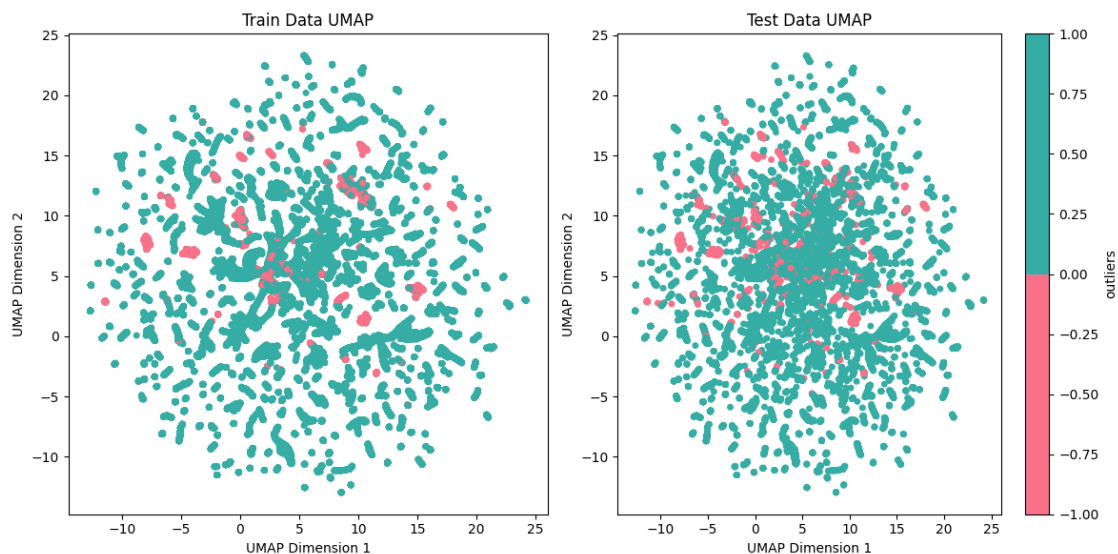
        cmap=train_cmap,
        edgecolor='none')
    axs[0].set_title('Train Data UMAP')
    axs[0].set_xlabel('UMAP Dimension 1')
    axs[0].set_ylabel('UMAP Dimension 2')

    # Plot the test data
    im_test = axs[1].scatter(outlier_test_umap[:, 0],
                             outlier_test_umap[:, 1],
                             s=25,
                             c=np.array(y_test_pred),
                             cmap= train_cmap,
                             edgecolor='none')

    axs[1].set_title('Test Data UMAP')
    axs[1].set_xlabel('UMAP Dimension 1')
    axs[1].set_ylabel('UMAP Dimension 2')
    # Add colorbar for training data
    cbar_train = fig.colorbar(im_train, ax=axs[1], label='outliers')

    # Display the plots
    plt.tight_layout()
    plt.show()
create_outlier_umap(outlier_train_umap,X_train_outlier["is_outlier"],outlier_test_umap,X_test_

```



6 CHAPTER 6: ML CLUSTERING

```
[2]: X_train = pd.read_csv("X_train_final.csv")
     X_test = pd.read_csv("X_test_final.csv")
```

6.1 KMeans

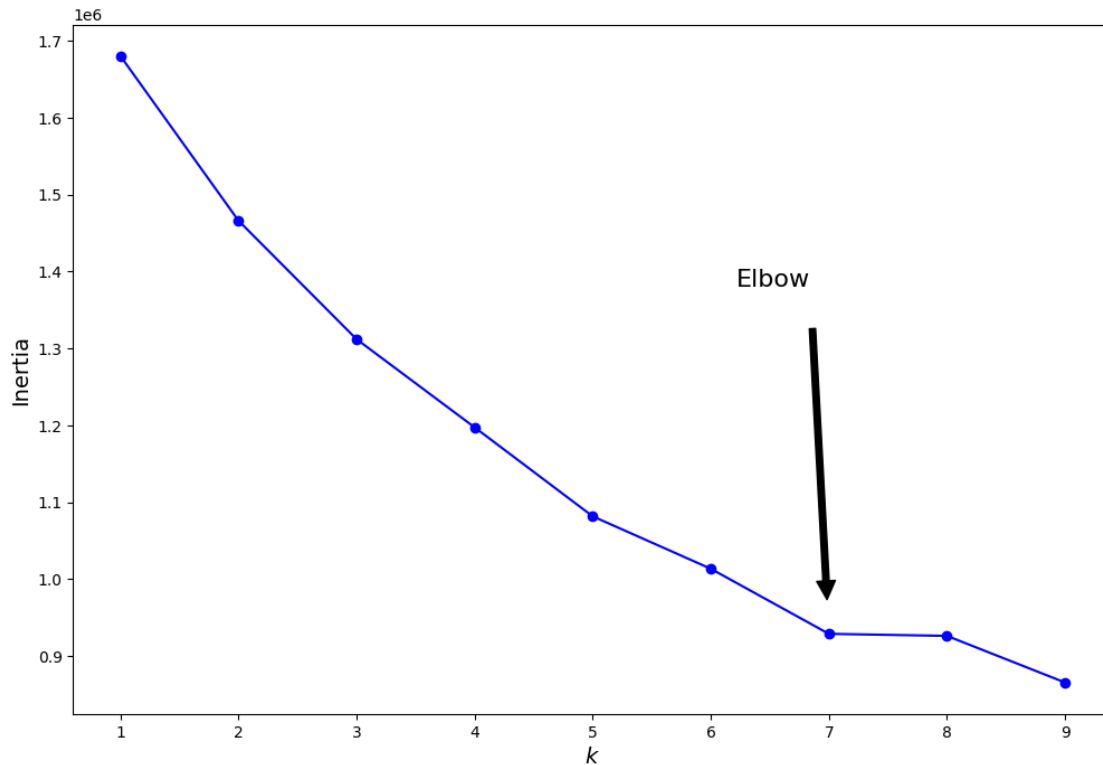
```
[7]: # Inertia = Within Cluster Sum of Squares
     kmeans_per_k = [KMeans(n_clusters=k, random_state=42).fit(X_train)
                     for k in range(1, 10)]

     inertias = [model.inertia_ for model in kmeans_per_k]
```

```
[8]: with open('kmeans_per_k.pkl', 'wb') as f:
     pickle.dump(kmeans_per_k, f)
     with open('inertias.pkl', 'wb') as f:
     pickle.dump(inertias, f)
```

```
[9]: with open('kmeans_per_k.pkl', 'rb') as file:
     kmeans_per_k = pickle.load(file)
     with open('inertias.pkl', 'rb') as file:
     inertias = pickle.load(file)
```

```
[16]: plt.figure(figsize=(12, 8))
     plt.plot(range(1, 10), inertias, "bo-")
     plt.xlabel("$k$", fontsize=14)
     plt.ylabel("Inertia", fontsize=14)
     plt.annotate('Elbow',
                  xy=(7, inertias[6]),
                  xytext=(0.55, 0.55),
                  textcoords='figure fraction',
                  fontsize=16,
                  arrowprops=dict(facecolor='black', shrink=0.1)
                  )
     plt.show()
```



Observation: > The key point of interest in this graph is the “elbow” method for determining the optimal number of clusters. The “elbow” point is indicated by a black arrow pointing to $k = 7$. This point suggests the optimal number of clusters, as it represents the point where the rate of decrease in inertia slows down significantly.

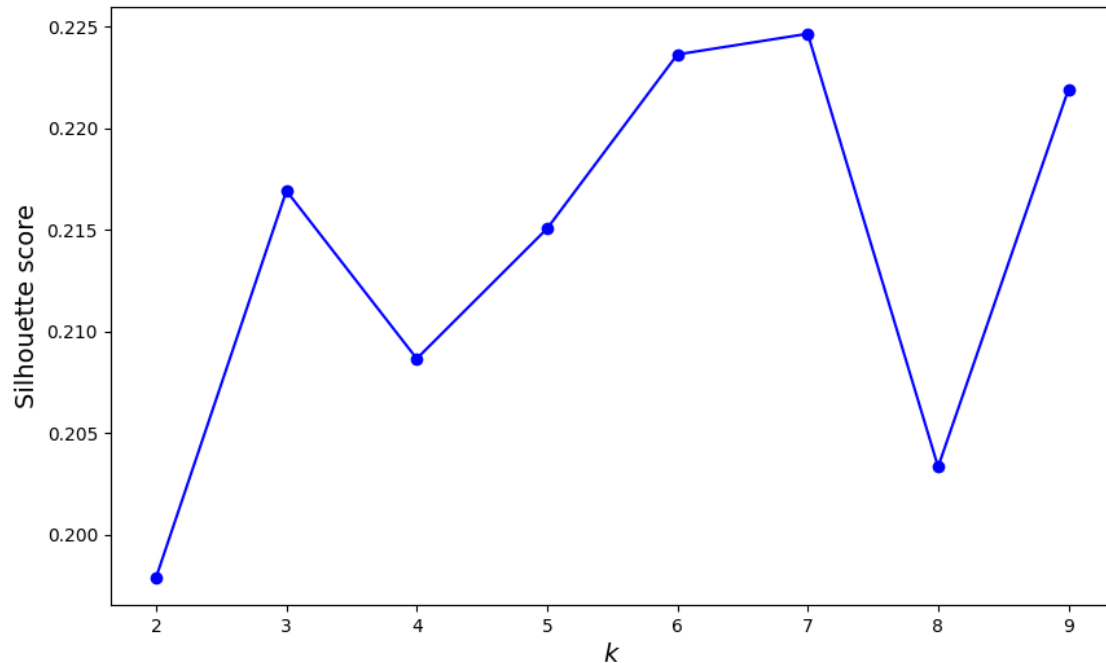
In summary, the optimal number of clusters, according to this graph, is $k = 7$.

```
[11]: ## plot for different values of K
silhouette_scores = [silhouette_score(X_train, model.labels_)
                      for model in kmeans_per_k[1:]]
```

```
[12]: with (open('silhouette_scores.pkl', 'wb')) as f:
        pickle.dump(silhouette_scores, f)
```

```
[ ]: with open('silhouette_scores.pkl', 'rb') as file:
        silhouette_scores = pickle.load(file)
```

```
[13]: plt.figure(figsize=(10, 6))
plt.plot(range(2, 10), silhouette_scores, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Silhouette score", fontsize=14)
plt.show()
```



Observations: > This graph helps identify the optimal number of clusters, which would be around $k = 3$ or $k = 7$, as these values show higher Silhouette scores.

```
[17]: k = 7
      kmeans = KMeans(n_clusters=k, random_state = 42)
      y_train_pred = kmeans.fit_predict(X_train)
      y_test_pred = kmeans.predict(X_test)
```

```
[18]: kmeans.inertia_
```

```
[18]: 928701.8163173792
```

```
[19]: with open('kmeans.pkl', 'wb') as f:
      pickle.dump(kmeans, f)
```

```
[20]: print(y_train_pred)
      print(y_test_pred)
```

```
[0 1 1 ... 1 1 1]
[0 3 3 ... 0 1 2]
```

```
[21]: # Get unique values and their counts
      unique_values, counts = np.unique(y_train_pred, return_counts=True)

      # Display the results
```

```
print("Unique Values:", unique_values)
print("Counts:", counts)
```

```
Unique Values: [0 1 2 3 4 5 6]
Counts: [50481 46060 5459 19702 1004 3034 20]
```

Observation > value counts of each kmeans_label is so uneven and Imbalanced. So it is not a good clustering.

```
[22]: # Get unique values and their counts
unique_values, counts = np.unique(y_test_pred, return_counts=True)

# Display the results
print("Unique Values:", unique_values)
print("Counts:", counts)
```

```
Unique Values: [0 1 2 3 4 5 6]
Counts: [12700 11400 1436 4905 250 737 12]
```

```
[23]: ##coordinates of the cluster centers
kmeans.cluster_centers_
```

```
[23]: array([[ -2.80499272e-02, -6.26374691e-02, -6.05903863e-02,
        -2.81018204e-01, -1.08537783e-02, -3.28744411e-03,
        -2.08742686e-01, -1.04598871e-01, -1.00987532e-01,
        -2.81327131e-01, -5.43892685e-01, -5.92475354e-01,
        -5.75540001e-01,  9.59197417e-01],
       [-1.82702878e-02, -1.33329225e-01, -4.11015072e-02,
        -3.51020101e-01, -2.00439791e-02, -5.22218199e-03,
        -2.08742686e-01, -1.33712848e-01, -7.27797088e-02,
        -3.48230276e-01,  6.88239844e-01,  8.91691324e-01,
        9.62371850e-01,  9.52880944e-01],
       [-6.48954319e-02, -5.32147475e-02, -1.70248674e-01,
        -3.66116557e-01,  2.47649643e-01,  1.92034441e-01,
        2.18881092e+00,  3.84651832e+00,  1.40799131e+00,
        -3.77606519e-01, -5.52402014e-01, -3.37767173e-01,
        -3.06794351e-01, -9.08408133e-01],
       [-2.03402245e-02, -1.51684138e-02,  2.54960540e-01,
        1.73206973e+00, -1.35855847e-02, -2.36743039e-03,
        -2.07725529e-01, -1.01123583e-01, -2.57253144e-01,
        1.73333179e+00, -5.96220413e-02, -4.75132101e-01,
        -6.91649082e-01,  8.29501526e-01],
       [ 3.01959177e+00,  9.99870973e+00,  9.41486349e-01,
        -1.56414583e-01,  6.77312112e-02, -1.83899405e-01,
        -1.68907390e-01, -8.63271952e-02, -2.36280778e-01,
        -1.51022253e-01, -7.06048505e-01, -9.45744254e-01,
        -1.07789584e+00, -6.63346614e-01],
       [-4.03730948e-02, -1.14127065e-01, -2.47163719e-02,
```

```

-5.02433917e-01, -2.93871256e-01, -3.04226848e-01,
 4.10511530e+00, -2.47867481e+00,  1.99517531e+00,
-5.28857925e-01,  2.16757153e-01,  3.19946338e-01,
 3.55903887e-01, -7.93012525e-01],
[ 5.16593337e+00,  1.02294576e+01,  1.21196935e-02,
-2.02354793e-01,  6.05349209e+01,  2.56290911e+01,
 5.41156768e-01,  2.07143380e+00,  4.32360609e-01,
-2.19106397e-01, -1.33152041e+00, -1.36609574e+00,
-1.45992145e+00, -1.00000000e+00]])

```

6.1.1 Visualizing clusters

```

[24]: X_train_clusters = X_train.copy(deep = True)
      X_test_clusters = X_test.copy(deep = True)

```

```

[25]: X_train_clusters.to_csv("X_train_clusters.csv", index=False)
      X_test_clusters.to_csv("X_test_clusters.csv", index=False)

```

```

[26]: X_train_clusters['k_means_label'] = y_train_pred
      X_train_clusters.head()

```

```

[26]:  company_hash      ctc  job_position  total_experience      Hike  \
0      0.028608 -0.122791    -0.217902    -0.514316 -0.015194
1      1.611623 -0.138620     1.936612    -0.283234 -0.015194
2      0.052871 -0.165372    -0.479036    -0.745398 -0.015194
3     -0.292440 -0.078469    -0.479036    -0.283234 -0.015194
4      0.767031 -0.177402     0.224742    -1.207562 -0.015194

      Hike_From_Same_Company  Is_Job_Position_Changed  Is_CTC_Changed  \
0                -0.002777                -0.208743        -0.098114
1                -0.002777                -0.208743        -0.098114
2                -0.002777                -0.208743        -0.098114
3                -0.002777                -0.208743        -0.098114
4                -0.002777                -0.208743        -0.098114

      Working_in_multiple_companies  tenure  Designation      Class      Tier  \
0                -0.298125 -0.507774    -0.098588 -0.163924 -0.211203
1                -0.298125 -0.276840    -0.098588  1.250395  1.103237
2                -0.298125 -0.738708     1.545323  1.250395  1.103237
3                -0.298125 -0.276840    -0.098588 -0.163924 -0.211203
4                -0.298125 -1.200576     1.545323  1.250395  1.103237

      is_outlier  k_means_label
0              1              0
1              1              1
2              1              1
3              1              0

```


4 1 1

```
[27]: X_test_clusters['k_means_label'] = y_test_pred
X_test_clusters.head()
```

```
[27]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
0	0.028608	-0.193232	-0.551260	-0.514316	-0.015194	
1	0.028608	0.078242	1.421846	2.489751	-0.015194	
2	-0.248843	0.032337	1.421846	2.027587	-0.015194	
3	0.028608	-0.189274	-0.974696	0.641095	-0.015194	
4	-0.178556	-0.064222	0.224742	-0.052151	-0.015194	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
0	-0.002777	-0.208743	-0.098114	
1	-0.002777	-0.208743	-0.098114	
2	-0.002777	-0.208743	-0.098114	
3	-0.002777	-0.208743	-0.098114	
4	-0.002777	-0.208743	-0.098114	

	Working_in_multiple_companies	tenure	Designation	Class	Tier	\
0	-0.298125	-0.507774	-0.098588	-0.163924	-0.211203	
1	-0.298125	2.494369	-0.098588	-0.163924	-0.211203	
2	-0.298125	2.032501	-0.098588	-0.163924	-1.525643	
3	-0.298125	0.646896	-0.098588	-0.163924	-0.211203	
4	-0.298125	-0.045906	-0.098588	-0.163924	1.103237	

	is_outlier	k_means_label
0	1	0
1	1	3
2	1	3
3	1	0
4	1	1

total_experience vs ctc plot

```
[28]: def viz_clusters(kmeans, clusters):
# Plot the clusters with colors based on labels
scatter = plt.scatter(
    X_train_clusters['total_experience'],
    X_train_clusters['ctc'],
    c=X_train_clusters['k_means_label'],
    cmap='viridis',
    alpha=0.7
)

# Plot the centroids with different colors
centroid_handles = []
```

```

centroid_labels = []
for i, centroid in enumerate(kmeans.cluster_centers_):
    color = plt.cm.viridis(i / kmeans.n_clusters) # Use the same colormap
    for centroids
        centroid_handles.append(plt.scatter(centroid[0], centroid[1],
        color=color, marker="X", s=100, edgecolors="white"))
        centroid_labels.append(f"Centroid {i+1}")

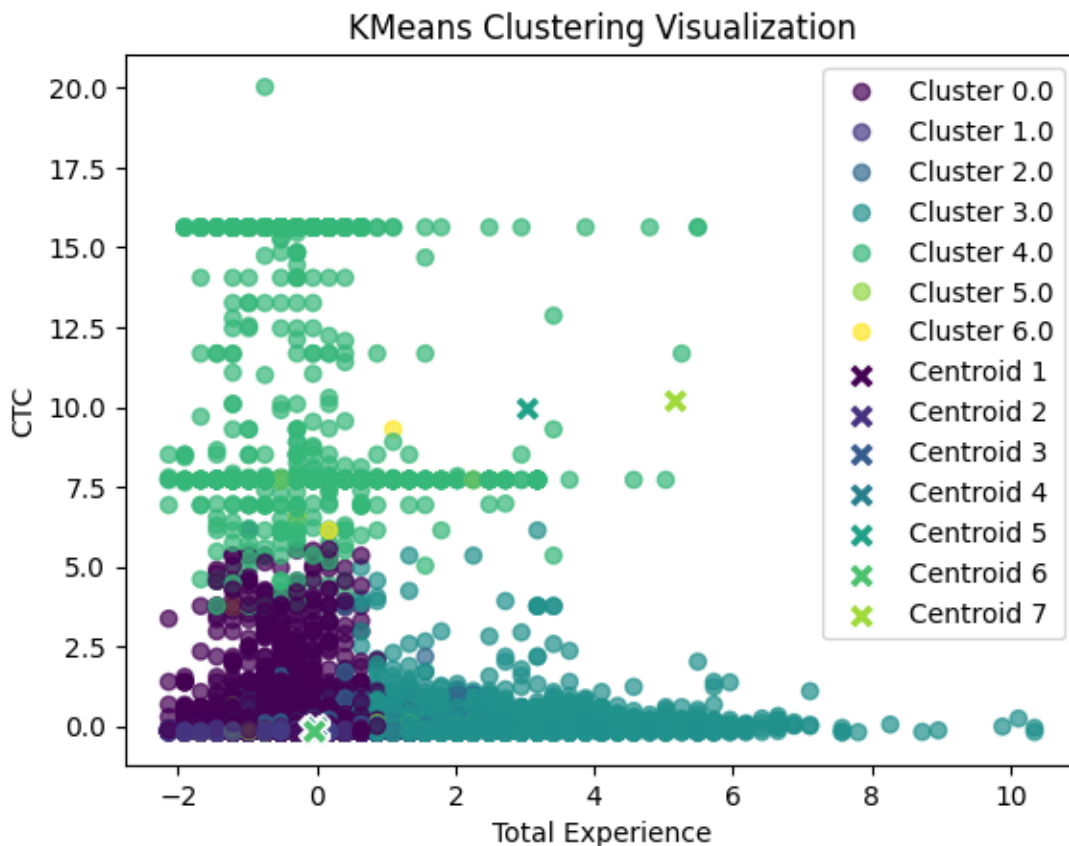
# Create legend elements for the clusters
handles, labels = scatter.legend_elements(prop="colors", fmt="Cluster {x}")

# Add legend for the centroids
plt.legend(handles + centroid_handles, labels + centroid_labels, loc='best')

# Add labels and title
plt.xlabel('Total Experience')
plt.ylabel('CTC')
plt.title('KMeans Clustering Visualization')
plt.show()

# Usage
viz_clusters(kmeans, X_train_clusters)

```



polar line plot

```
[29]: polar = X_train_clusters.groupby("k_means_label").mean().reset_index()
      polar = pd.melt(polar, id_vars=["k_means_label"])
      polar.head()
```

```
[29]:   k_means_label  variable  value
      0           0  company_hash -0.028047
      1           1  company_hash -0.018197
      2           2  company_hash -0.064895
      3           3  company_hash -0.020517
      4           4  company_hash  3.019592
```

```
[30]: fig = px.line_polar(polar, r="value", theta="variable", color="k_means_label",
      ↪line_close=True,height=700,width=800)
      fig.show()
```

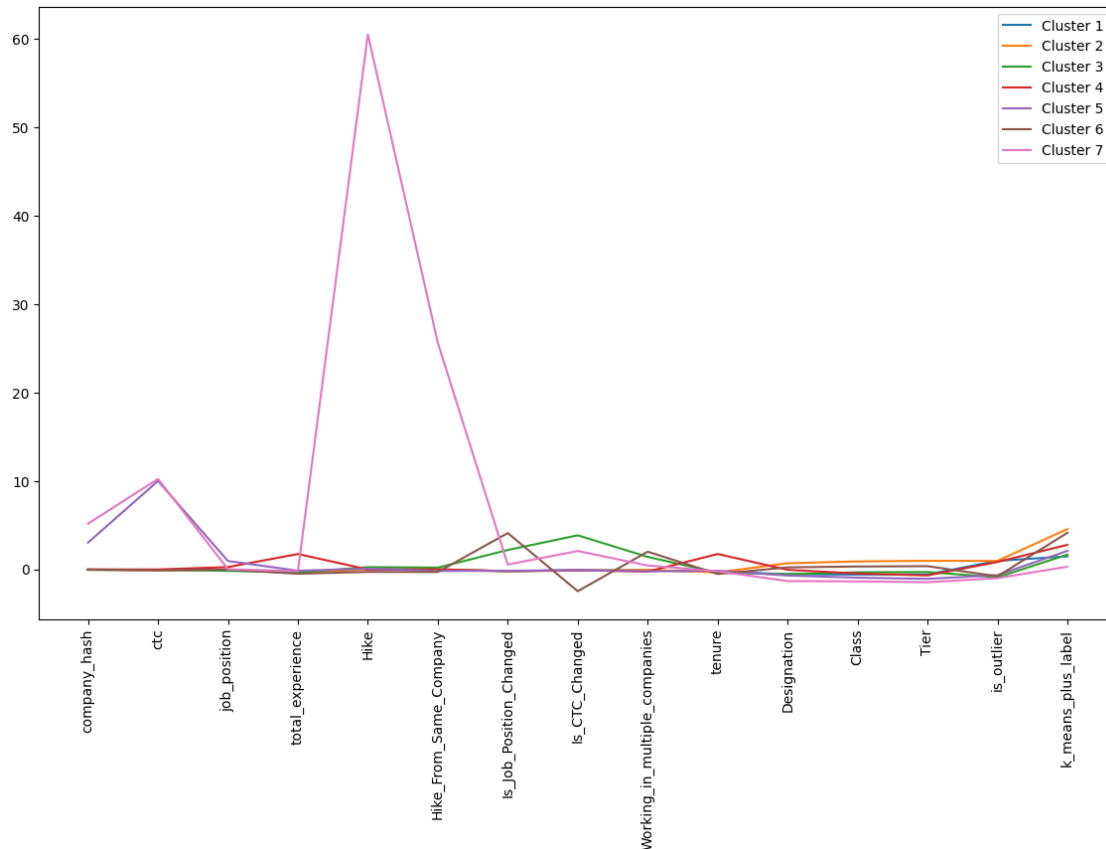
6.1.2 Visualising the mean feature values of clusters

```
[91]: clustered_df = X_train_clusters.groupby('k_means_label').mean()

      labels = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4', 'Cluster 5',
      ↪'Cluster 6', 'Cluster 7']

      plt.figure(figsize=(14,8))
      plt.plot(clustered_df.T, label=labels)
      plt.xticks(rotation=90)
      plt.legend(labels)
```

```
[91]: <matplotlib.legend.Legend at 0x182ba43cf20>
```



k_means_label UMAP Visualisation

```
[157]: train_umap = UMAP(init = "random",n_neighbors = 200, min_dist = 0.1,
    ↪random_state = 42,n_jobs = -1)
binary_train_umap = train_umap.fit_transform(X_train)
binary_test_umap = train_umap.transform(X_test)
```

C:\Users\saina\AppData\Roaming\Python\Python312\site-packages\umap\umap_.py:1945: UserWarning:

n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

```
[32]: def create_umap(binary_train_umap,y_train_pred,binary_test_umap,y_test_pred):
    # Create a figure with 1 row and 2 columns
    fig, axs = plt.subplots(1, 2, figsize=(12, 6))
    train_cmap = ListedColormap(sns.husl_palette(len(np.unique(y_train_pred))))

    # Plot the training data
```

```

im_train = axs[0].scatter(binary_train_umap[:, 0],
                          binary_train_umap[:, 1],
                          s=25,
                          c=np.array(y_train_pred),
                          cmap=train_cmap,
                          edgecolor='none')

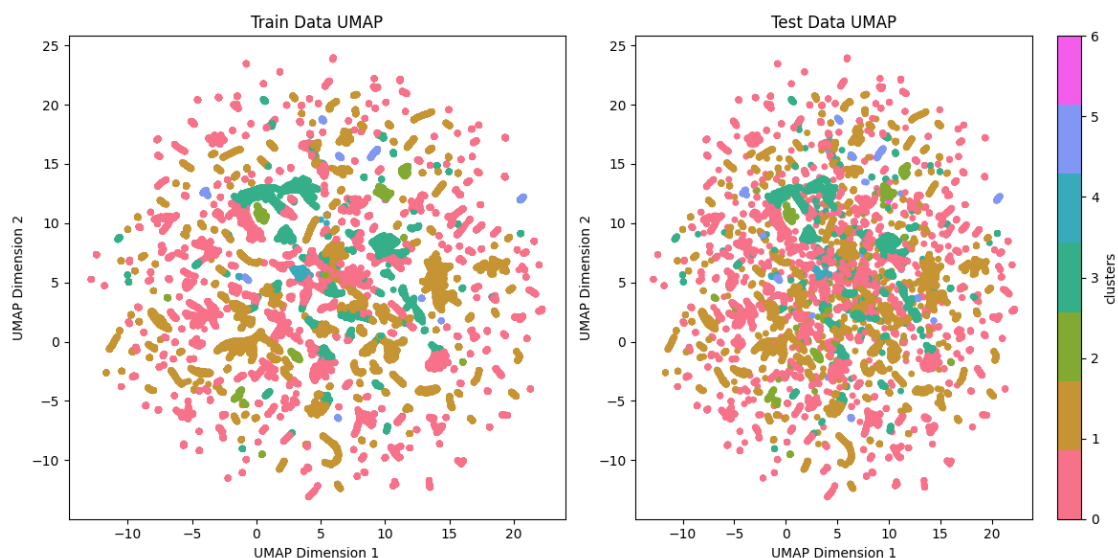
axs[0].set_title('Train Data UMAP')
axs[0].set_xlabel('UMAP Dimension 1')
axs[0].set_ylabel('UMAP Dimension 2')

# Plot the test data
im_test = axs[1].scatter(binary_test_umap[:, 0],
                          binary_test_umap[:, 1],
                          s=25,
                          c=np.array(y_test_pred),
                          cmap= train_cmap,
                          edgecolor='none')

axs[1].set_title('Test Data UMAP')
axs[1].set_xlabel('UMAP Dimension 1')
axs[1].set_ylabel('UMAP Dimension 2')
# Add colorbar for training data
cbar_train = fig.colorbar(im_train, ax=axs[1], label='clusters')

# Display the plots
plt.tight_layout()
plt.show()
create_umap(binary_train_umap,X_train_clusters["k_means_label"],binary_test_umap,X_test_clusters["k_means_label"])

```



```
[205]: with open('binary_train_umap.pkl', 'wb') as f:
        pickle.dump(binary_train_umap, f)

        with open('binary_test_umap.pkl', 'wb') as f:
            pickle.dump(binary_test_umap, f)
```

```
[3]: # To load the model later
      with open('binary_train_umap.pkl', 'rb') as file:
          binary_train_umap = pickle.load(file)
      with open('binary_test_umap.pkl', 'rb') as file:
          binary_test_umap = pickle.load(file)
```

6.1.3 3D plot total_experience vs etc vs Hike

```
[54]: fig = px.scatter_3d(X_train_clusters, x='total_experience', y='etc', z='Hike',
    ↪color='k_means_label', width=600, height=600)
      fig.update_traces(marker=dict(size=2), selector=dict(mode='markers'))
      fig.show()
```

6.1.4 INSIGHTS

6.1.5 Cluster Interpretations and Names

1. **Cluster 0:**
 - **Interpretation:** This cluster has higher values for “Is_Job_Position_Changed” and “Is_CTC_Changed.”
 - **Name:** “Dynamic Movers”
2. **Cluster 1:**
 - **Interpretation:** This cluster shows higher “CTC” and “Hike” values, suggesting well-compensated individuals.
 - **Name:** “High Earners”
3. **Cluster 2:**
 - **Interpretation:** Higher values in “total_experience” and “tenure” indicate experienced professionals.
 - **Name:** “Veterans”
4. **Cluster 3:**
 - **Interpretation:** Elevated “Hike_From_Same_Company” values suggest loyalty and growth within the same company.
 - **Name:** “Loyal Climbers”
5. **Cluster 4:**
 - **Interpretation:** Balanced features with no significant spikes, indicating steady and moderate profiles.
 - **Name:** “Steady Achievers”
6. **Cluster 5:**
 - **Interpretation:** Higher values for “Working_in_multiple_companies” indicate frequent job changes.

- **Name:** “Frequent Movers”

7. Cluster 6:

- **Interpretation:** High “Designation” and “Class” values suggest senior-level positions and higher professional classes.
- **Name:** “Senior Leaders”

6.2 K Means ++

```
[68]: kmeansplus = KMeans(n_clusters=7, init = 'k-means++', random_state = 50)
y_train_pred = kmeansplus.fit_predict(X_train)
y_test_pred = kmeansplus.predict(X_test)
```

```
[69]: kmeansplus.inertia_
```

```
[69]: 1012396.369653316
```

```
[71]: with open('kmeansplus.pkl', 'wb') as f:
      pickle.dump(kmeansplus, f)
```

```
[72]: print(y_train_pred)
      print(y_test_pred)
```

```
[1 6 6 ... 5 6 1]
[1 3 3 ... 2 1 0]
```

```
[73]: # Get unique values and their counts
unique_values, counts = np.unique(y_train_pred, return_counts=True)

# Display the results
print("Unique Values:", unique_values)
print("Counts:", counts)
```

```
Unique Values: [0 1 2 3 4 5 6]
Counts: [ 2640 49970 15995 16660  2631  7196 30668]
```

Observation > Better balance in value counts in Kmeans ++ than kmeans

```
[74]: # Get unique values and their counts
unique_values, counts = np.unique(y_test_pred, return_counts=True)

# Display the results
print("Unique Values:", unique_values)
print("Counts:", counts)
```

```
Unique Values: [0 1 2 3 4 5 6]
Counts: [ 677 12584  4042  4179   627  1812  7519]
```

```
[75]: ##coordinates of the cluster centers
kmeansplus.cluster_centers_
```

```
[75]: array([[ 3.33834873e-03,  2.13122589e-02, -5.52870343e-02,
             -3.77154339e-01,  6.52375867e-01,  3.34750214e-01,
             4.76028804e+00,  3.83755324e+00,  1.79233600e+00,
             -3.95193613e-01, -2.76055175e-01, -1.95532186e-01,
             -2.24646204e-01, -1.00000000e+00],
            [-7.60498611e-02, -1.06825480e-01, -5.74143164e-02,
             -2.46298298e-01, -1.29468300e-02, -2.05175394e-03,
             -2.08742686e-01, -7.29320820e-02, -2.98124649e-01,
             -2.45866926e-01, -2.01985819e-01, -1.58178708e-01,
             -1.07883857e-02,  9.85711427e-01],
            [ 1.95565411e-01,  6.37884302e-01, -1.43611715e-02,
             -1.56886704e-01,  4.26162473e-02,  3.18499263e-02,
             -2.08430150e-01,  1.84490992e-01, -2.95841312e-01,
             -1.58990780e-01, -1.23717746e+00, -1.45631652e+00,
             -1.31519824e+00,  7.34058515e-01],
            [-1.28691248e-02,  1.31490319e-02,  3.35606032e-01,
             1.88412972e+00, -1.18180020e-02, -6.75423630e-04,
             -2.07542295e-01, -9.29046341e-02, -2.87381574e-01,
             1.88496290e+00, -2.67058925e-03, -4.29401008e-01,
             -7.12235331e-01,  8.06711087e-01],
            [-1.98576615e-02, -1.05826818e-01,  9.65532996e-03,
             -5.13700711e-01, -2.34189593e-01, -4.19518060e-01,
             4.77348554e+00, -2.24209879e+00,  1.78282822e+00,
             -5.45429119e-01,  2.08825561e-01,  3.19341805e-01,
             3.71826944e-01, -7.68909160e-01],
            [-4.53037768e-02, -9.18253141e-02, -1.51994863e-01,
             -4.42447602e-01, -6.96616000e-02, -2.77700269e-03,
             -2.08742686e-01,  2.46685067e-01,  3.35430164e+00,
             -4.35952062e-01,  4.51061050e-02,  1.40323396e-01,
             1.30741411e-01,  1.97609783e-01],
            [ 4.09473143e-02, -1.37003941e-01, -4.16667724e-02,
             -3.59969491e-01, -1.44350756e-02, -5.07690856e-03,
             -2.08742686e-01, -1.22809699e-01, -2.98124649e-01,
             -3.57276769e-01,  9.71122783e-01,  1.20709110e+00,
             1.04721890e+00,  9.59045259e-01]])
```

6.2.1 Visualizing clusters

```
[76]: X_train_clusters['k_means_plus_label'] = y_train_pred
      X_train_clusters.head()
```

```
[76]:
```

	company_hash	ctc	job_position	total_experience	Hike \
0	0.028608	-0.122791	-0.217902	-0.514316	-0.015194
1	1.611623	-0.138620	1.936612	-0.283234	-0.015194
2	0.052871	-0.165372	-0.479036	-0.745398	-0.015194
3	-0.292440	-0.078469	-0.479036	-0.283234	-0.015194
4	0.767031	-0.177402	0.224742	-1.207562	-0.015194

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
0	-0.002777	-0.208743	-0.098114	
1	-0.002777	-0.208743	-0.098114	
2	-0.002777	-0.208743	-0.098114	
3	-0.002777	-0.208743	-0.098114	
4	-0.002777	-0.208743	-0.098114	

	Working_in_multiple_companies	tenure	Designation	Class	Tier	\
0	-0.298125	-0.507774	-0.098588	-0.163924	-0.211203	
1	-0.298125	-0.276840	-0.098588	1.250395	1.103237	
2	-0.298125	-0.738708	1.545323	1.250395	1.103237	
3	-0.298125	-0.276840	-0.098588	-0.163924	-0.211203	
4	-0.298125	-1.200576	1.545323	1.250395	1.103237	

	is_outlier	k_means_label	k_means_plus_label
0	1	0	1
1	1	1	6
2	1	1	6
3	1	0	1
4	1	1	6

```
[77]: X_test_clusters['k_means_plus_label'] = y_test_pred
X_test_clusters.head()
```

```
[77]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
0	0.028608	-0.193232	-0.551260	-0.514316	-0.015194	
1	0.028608	0.078242	1.421846	2.489751	-0.015194	
2	-0.248843	0.032337	1.421846	2.027587	-0.015194	
3	0.028608	-0.189274	-0.974696	0.641095	-0.015194	
4	-0.178556	-0.064222	0.224742	-0.052151	-0.015194	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
0	-0.002777	-0.208743	-0.098114	
1	-0.002777	-0.208743	-0.098114	
2	-0.002777	-0.208743	-0.098114	
3	-0.002777	-0.208743	-0.098114	
4	-0.002777	-0.208743	-0.098114	

	Working_in_multiple_companies	tenure	Designation	Class	Tier	\
0	-0.298125	-0.507774	-0.098588	-0.163924	-0.211203	
1	-0.298125	2.494369	-0.098588	-0.163924	-0.211203	
2	-0.298125	2.032501	-0.098588	-0.163924	-1.525643	
3	-0.298125	0.646896	-0.098588	-0.163924	-0.211203	
4	-0.298125	-0.045906	-0.098588	-0.163924	1.103237	

	is_outlier	k_means_label	k_means_plus_label
0	1	0	1
1	1	1	6
2	1	1	6
3	1	0	1
4	1	1	6

0	1	0	1
1	1	3	3
2	1	3	3
3	1	0	1
4	1	1	1

```
[78]: X_train_clusters.to_csv("X_train_clusters.csv", index=False)
      X_test_clusters.to_csv("X_test_clusters.csv", index=False)
```

total_experience vs ctc plot

```
[79]: def viz_clusters(kmeansplus, clusters):
      # Plot the clusters with colors based on labels
      scatter = plt.scatter(
          X_train_clusters['total_experience'],
          X_train_clusters['ctc'],
          c=X_train_clusters['k_means_plus_label'],
          cmap='viridis',
          alpha=0.7
      )

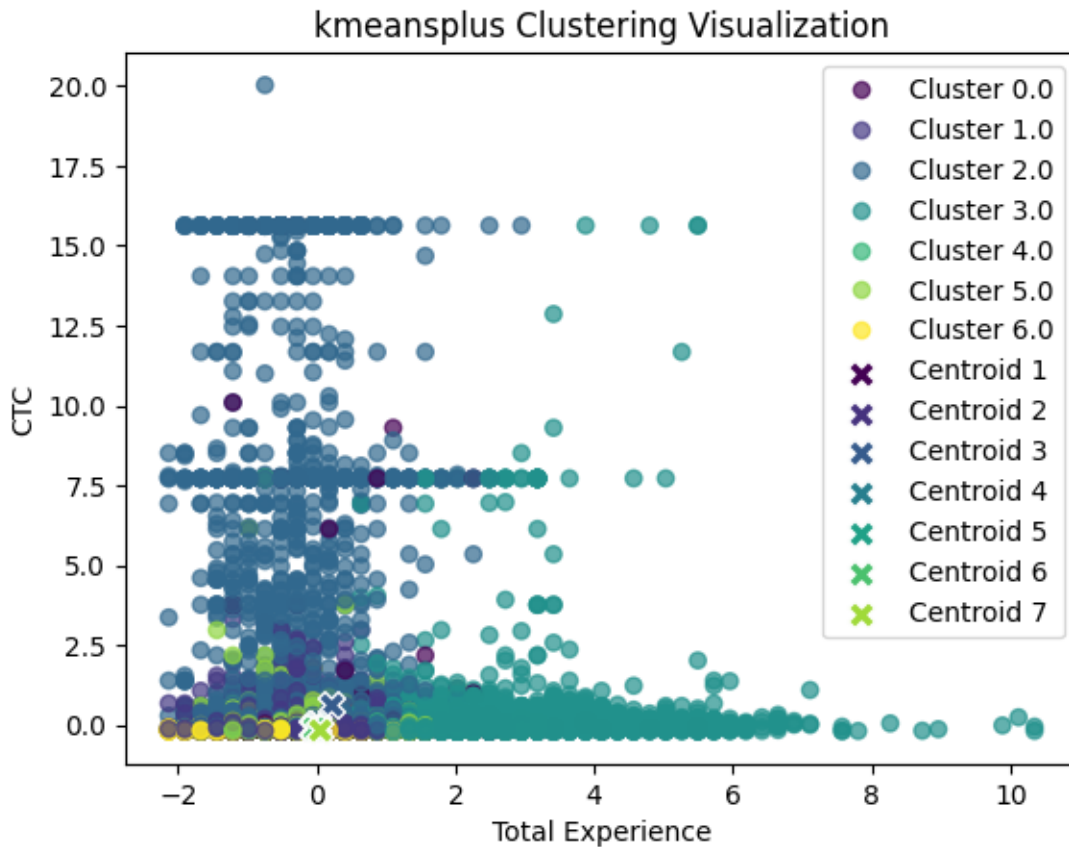
      # Plot the centroids with different colors
      centroid_handles = []
      centroid_labels = []
      for i, centroid in enumerate(kmeansplus.cluster_centers_):
          color = plt.cm.viridis(i / kmeansplus.n_clusters) # Use the same
          colormap for centroids
          centroid_handles.append(plt.scatter(centroid[0], centroid[1],
          color=color, marker="X", s=100, edgecolors="white"))
          centroid_labels.append(f"Centroid {i+1}")

      # Create legend elements for the clusters
      handles, labels = scatter.legend_elements(prop="colors", fmt="Cluster {x}")

      # Add legend for the centroids
      plt.legend(handles + centroid_handles, labels + centroid_labels, loc='best')

      # Add labels and title
      plt.xlabel('Total Experience')
      plt.ylabel('CTC')
      plt.title('kmeansplus Clustering Visualization')
      plt.show()

      # Usage
      viz_clusters(kmeansplus, X_train_clusters)
```



polar line plot

```
[80]: polar = X_train_clusters.groupby("k_means_plus_label").mean().reset_index()
      polar = pd.melt(polar, id_vars=["k_means_plus_label"])
      polar.head()
```

```
[80]:   k_means_plus_label  variable  value
0         0         0  company_hash  0.003338
1         1         1  company_hash -0.076050
2         2         2  company_hash  0.195618
3         3         3  company_hash -0.012907
4         4         4  company_hash -0.019858
```

```
[81]: fig = px.line_polar(polar, r="value", theta="variable",
      ↪color="k_means_plus_label", line_close=True,height=700,width=800)
      fig.show()
```

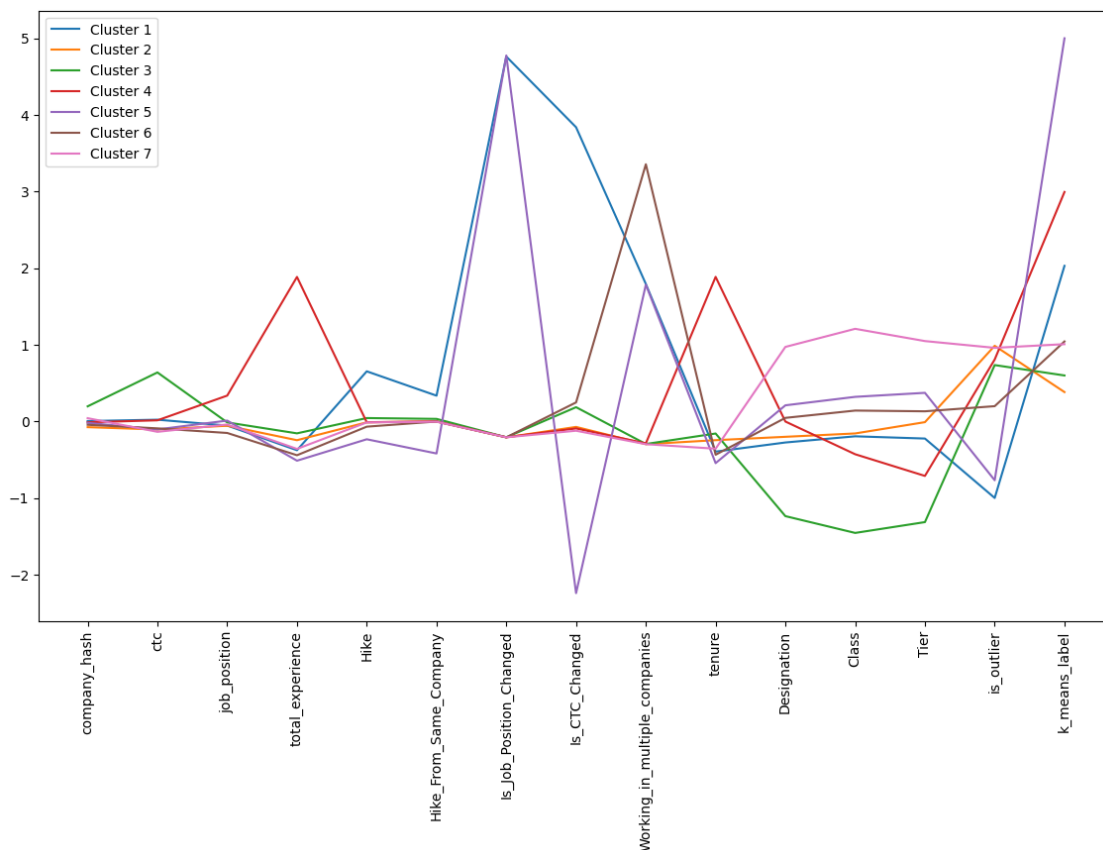
6.2.2 Visualising the mean feature values of clusters

```
[90]: clustered_df = X_train_clusters.groupby('k_means_plus_label').mean()

labels = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4', 'Cluster 5', 'Cluster 6', 'Cluster 7']

plt.figure(figsize=(14,8))
plt.plot(clustered_df.T, label=labels)
plt.xticks(rotation=90)
plt.legend(labels)
```

[90]: <matplotlib.legend.Legend at 0x182ba0d7200>



k_means_label UMAP Visualisation

```
[ ]: train_umap = UMAP(init = "random",n_neighbors = 200, min_dist = 0.1,
    random_state = 42,n_jobs = -1)
binary_train_umap = train_umap.fit_transform(X_train)
binary_test_umap = train_umap.transform(X_test)
```

C:\Users\saina\AppData\Roaming\Python\Python312\site-packages\umap\umap_.py:1945: UserWarning:

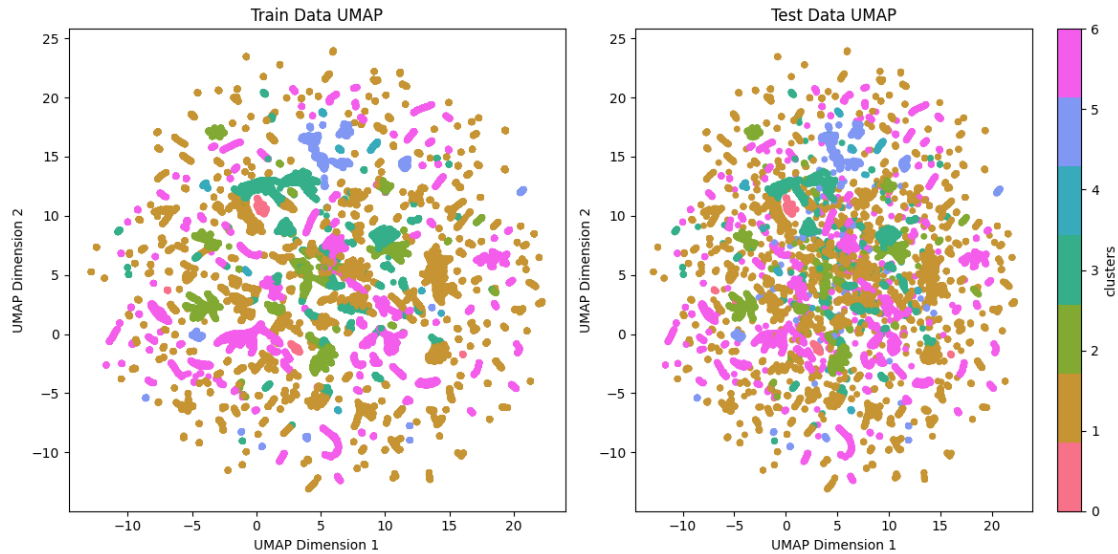
n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

```
[82]: def create_umap(binary_train_umap,y_train_pred,binary_test_umap,y_test_pred):
    # Create a figure with 1 row and 2 columns
    fig, axs = plt.subplots(1, 2, figsize=(12, 6))
    train_cmap = ListedColormap(sns.husl_palette(len(np.unique(y_train_pred))))

    # Plot the training data
    im_train = axs[0].scatter(binary_train_umap[:, 0],
                              binary_train_umap[:, 1],
                              s=25,
                              c=np.array(y_train_pred),
                              cmap=train_cmap,
                              edgecolor='none')
    axs[0].set_title('Train Data UMAP')
    axs[0].set_xlabel('UMAP Dimension 1')
    axs[0].set_ylabel('UMAP Dimension 2')

    # Plot the test data
    im_test = axs[1].scatter(binary_test_umap[:, 0],
                              binary_test_umap[:, 1],
                              s=25,
                              c=np.array(y_test_pred),
                              cmap= train_cmap,
                              edgecolor='none')
    axs[1].set_title('Test Data UMAP')
    axs[1].set_xlabel('UMAP Dimension 1')
    axs[1].set_ylabel('UMAP Dimension 2')
    # Add colorbar for training data
    cbar_train = fig.colorbar(im_train, ax=axs[1], label='clusters')

    # Display the plots
    plt.tight_layout()
    plt.show()
    create_umap(binary_train_umap,X_train_clusters["k_means_plus_label"],binary_test_umap,X_test_clusters["k_means_plus_label"])
```



6.2.3 3D plot total_experience vs ctc vs Hike

```
[55]: fig = px.scatter_3d(X_train_clusters, x='total_experience', y='ctc', z='Hike',
    ↪color='k_means_plus_label', width=600, height=600)
fig.update_traces(marker=dict(size=2), selector=dict(mode='markers'))
fig.show()
```

6.3 INSIGHTS

6.3.1 Cluster Interpretations and Names

1. **Cluster 0:**
 - **Interpretation:** This cluster has higher values for “Is_Job_Position_Changed” and “Is_CTC_Changed.”
 - **Name:** “Dynamic Movers”
2. **Cluster 1:**
 - **Interpretation:** This cluster shows higher outliers
 - **Name:** “Outlier”
3. **Cluster 2:**
 - **Interpretation:** Higher values in “ctc”, “Hike”, “Hike from same company” and Medium values in “Is_CTC_Changed”, “Is_job_position_Changed”, “tenure”.
 - **Name:** “High Earners & Medium Movers”
4. **Cluster 3:**
 - **Interpretation:** Higher values in “total_experience” and “tenure” indicate experienced professionals.
 - **Name:** “Veterans”
5. **Cluster 4:**
 - **Interpretation:** Higher values for “Working_in_multiple_companies” indicate frequent job changes.

- **Name:** “Frequent Movers”
6. **Cluster 5:**
- **Interpretation:** Higher values for “Working_in_multiple_companies” only. Indicates he works simultaneously in two companies
 - **Name:** “Parallel Jobs”
7. **Cluster 6:**
- **Interpretation:** High “Designation” and “Class” values suggest senior-level positions and higher professional classes.
 - **Name:** “Senior Leaders”

6.4 Agglomerative clustering

```
[39]: # import hierarchical clustering libraries
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import pairwise_distances_argmin
df_train = X_train.sample(n=10000, random_state=42)
df_test = X_test.sample(n=10000, random_state=42)

# create clusters
hc= AgglomerativeClustering(n_clusters=7, metric = 'euclidean', linkage = 'ward')
y_train_pred = hc.fit_predict(df_train)

# Compute cluster centroids
centroids = np.array([df_train[y_train_pred == i].mean(axis=0) for i in range(hc.n_clusters)])

# Assign each point in X_test to the nearest centroid
y_test_pred = pairwise_distances_argmin(df_test, centroids)
```

```
[40]: df_train_clusters = df_train.copy(deep = True)
df_test_clusters = df_test.copy(deep = True)
```

```
[41]: df_train.to_csv("df_train.csv", index=False)
df_test.to_csv("df_test.csv", index=False)
```

```
[42]: df_train_clusters['hc_label'] = y_train_pred
df_train_clusters.head()
```

```
[42]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
16615	-0.116385	-0.030981	-0.479036	-1.207562	-0.689101	
68223	-0.397989	-0.058682	-0.495886	0.641095	-0.015194	
120736	0.011208	-0.149701	-0.551260	-0.976480	-0.015194	
45755	0.028608	-0.070554	-0.479036	-0.745398	-0.015194	
106270	0.028608	0.198545	0.596984	1.334341	-0.015194	

```

Hike_From_Same_Company  Is_Job_Position_Changed  Is_CTC_Changed  \

```

16615	-0.002777	-0.208743	-0.098114
68223	-0.002777	-0.208743	-0.098114
120736	-0.002777	-0.208743	-0.098114
45755	-0.002777	-0.208743	-0.098114
106270	-0.002777	-0.208743	-0.098114

	Working_in_multiple_companies	tenure	Designation	Class \
16615	3.354302	-1.200576	-0.098588	-1.578244
68223	-0.298125	0.646896	-0.098588	-0.163924
120736	3.354302	-0.969642	1.545323	1.250395
45755	-0.298125	-0.738708	-0.098588	-0.163924
106270	-0.298125	1.339698	-0.098588	-0.163924

	Tier	is_outlier	hc_label
16615	-0.211203	-1	1
68223	-1.525643	1	3
120736	1.103237	1	1
45755	-0.211203	1	4
106270	-0.211203	1	6

```
[43]: df_test_clusters['hc_label'] = y_test_pred
df_test_clusters.head()
```

	company_hash	ctc	job_position	total_experience	Hike \
1188	0.028608	-0.196833	-0.748503	-0.514316	-0.015194
2157	0.052871	-0.102213	-0.551260	-0.052151	-0.015194
28339	-0.450090	-0.118042	-0.479036	0.178931	-0.015194
11270	0.028608	-0.023066	0.224742	2.258669	-0.015194
18192	-0.581748	0.000678	0.224742	2.720833	-0.015194

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed \
1188	-0.002777	-0.208743	-0.098114
2157	-0.002777	-0.208743	-0.098114
28339	-0.002777	-0.208743	-0.098114
11270	-0.002777	-0.208743	-0.098114
18192	-0.002777	-0.208743	-0.098114

	Working_in_multiple_companies	tenure	Designation	Class \
1188	-0.298125	-0.507774	-0.098588	-0.163924
2157	-0.298125	-0.045906	-1.742498	-1.578244
28339	-0.298125	0.185028	-0.098588	-0.163924
11270	-0.298125	2.263435	-0.098588	-0.163924
18192	-0.298125	2.725303	-0.098588	-1.578244

	Tier	is_outlier	hc_label
1188	-0.211203	1	3
2157	-1.525643	1	3

28339	-0.211203	1	3
11270	-0.211203	1	5
18192	-1.525643	1	5

```
[47]: df_train_clusters.to_csv("df_train_clusters.csv", index=False)
df_test_clusters.to_csv("df_test_clusters.csv", index=False)
```

```
[37]: df_train_clusters = pd.read_csv("df_train_clusters.csv")
df_test_clusters = pd.read_csv("df_test_clusters.csv")
```

```
[44]: hc
```

```
[44]: AgglomerativeClustering(n_clusters=7)
```

```
[45]: import pickle

with open("hc.pkl", "wb") as f:
    pickle.dump(hc, f)
```

6.4.1 total_experience vs ctc

```
[46]: def viz_agglomerative_clusters(df_train, y_train_pred, df_test, y_test_pred,
    ↪ centroids):
    import matplotlib.pyplot as plt
    from matplotlib.colors import ListedColormap
    import seaborn as sns
    import numpy as np

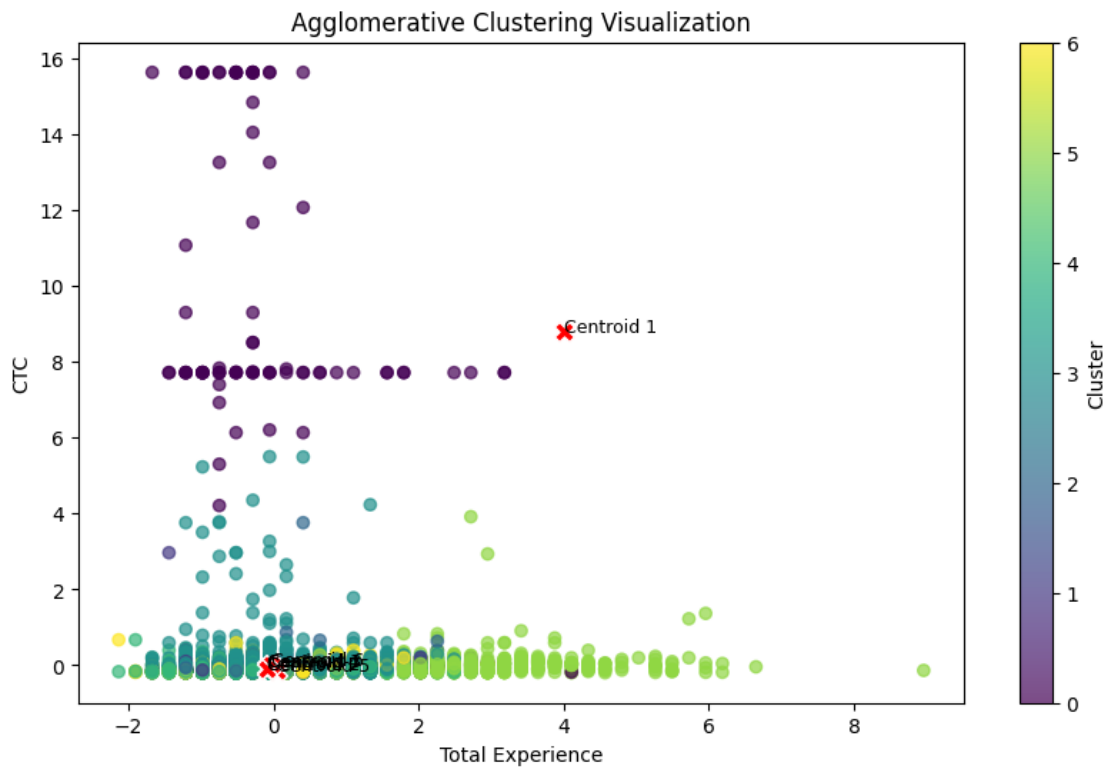
    # Plot the clusters for train data
    plt.figure(figsize=(10, 6))
    scatter = plt.scatter(
        df_train['total_experience'],
        df_train['ctc'],
        c=y_train_pred,
        cmap='viridis',
        alpha=0.7
    )

    # Plot the centroids
    for i, centroid in enumerate(centroids):
        plt.scatter(centroid[0], centroid[1], color='red', marker='X', s=100,
    ↪ edgecolors='white')
        plt.text(centroid[0], centroid[1], f'Centroid {i+1}', fontsize=9,
    ↪ color='black')

    plt.colorbar(scatter, label='Cluster')
```

```
plt.xlabel('Total Experience')
plt.ylabel('CTC')
plt.title('Agglomerative Clustering Visualization')
plt.show()

# Visualize
viz_agglomerative_clusters(df_train, y_train_pred, df_test, y_test_pred,
    ↪ centroids)
```



```
[ ]: viz_clusters(hc, df_train_clusters)
```

```
[55]: # Ensure UMAP is fitted to the same subset used for clustering
hc_umap = UMAP(init="random", n_neighbors=200, min_dist=0.1, random_state=42,
    ↪ n_jobs=-1)
binary_train_hc_umap = hc_umap.fit_transform(df_train) # Use the same
    ↪ `df_train` used for clustering
binary_test_hc_umap = hc_umap.transform(df_test) # Use the same `df_test`
    ↪ used for clustering
```

C:\Users\saina\AppData\Roaming\Python\Python312\site-packages\umap\umap_.py:1945: UserWarning:

n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

6.4.2 Umap visualisation

```
[56]: def create_umap_agglomerative(binary_train_hc_umap, y_train_pred,
    ↪binary_test_hc_umap, y_test_pred):
    # Create a figure with 1 row and 2 columns
    fig, axs = plt.subplots(1, 2, figsize=(12, 6))
    train_cmap = ListedColormap(sns.husl_palette(len(np.unique(y_train_pred))))

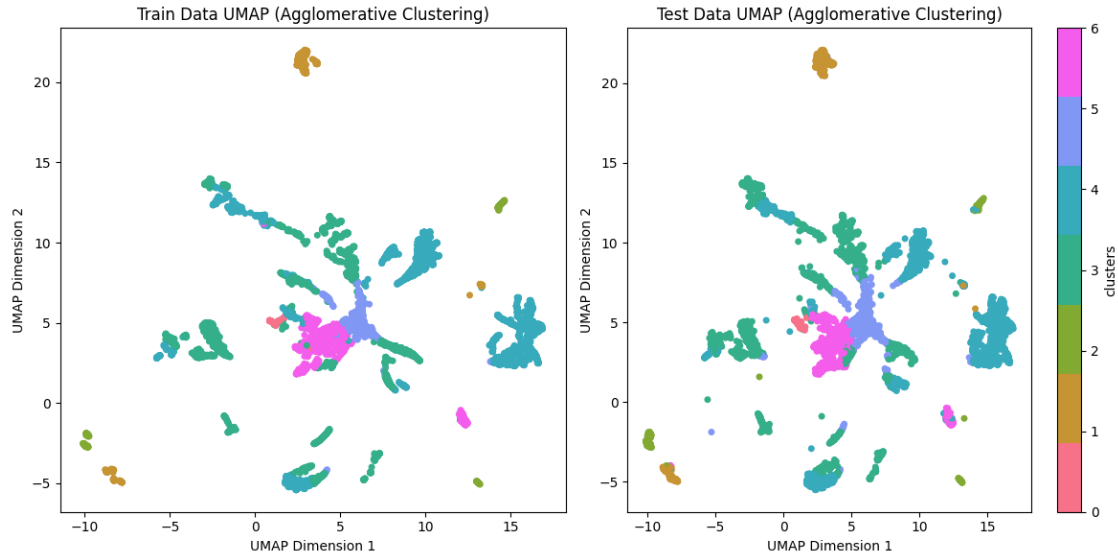
    # Plot the training data
    im_train = axs[0].scatter(binary_train_hc_umap[:, 0],
                              binary_train_hc_umap[:, 1],
                              s=25,
                              c=np.array(y_train_pred),
                              cmap=train_cmap,
                              edgecolor='none')
    axs[0].set_title('Train Data UMAP (Agglomerative Clustering)')
    axs[0].set_xlabel('UMAP Dimension 1')
    axs[0].set_ylabel('UMAP Dimension 2')

    # Plot the test data
    im_test = axs[1].scatter(binary_test_hc_umap[:, 0],
                              binary_test_hc_umap[:, 1],
                              s=25,
                              c=np.array(y_test_pred),
                              cmap=train_cmap,
                              edgecolor='none')
    axs[1].set_title('Test Data UMAP (Agglomerative Clustering)')
    axs[1].set_xlabel('UMAP Dimension 1')
    axs[1].set_ylabel('UMAP Dimension 2')

    # Add colorbar for clusters
    fig.colorbar(im_train, ax=axs[1], label='clusters')

    # Display the plots
    plt.tight_layout()
    plt.show()

    # Visualize
    create_umap_agglomerative(binary_train_hc_umap, y_train_pred,
    ↪binary_test_hc_umap, y_test_pred)
```



6.4.3 polar line plot

```
[57]: # Compute the mean of each cluster
polar_hc = df_train.groupby(y_train_pred).mean().reset_index()

# Melt the DataFrame for polar plotting
polar_hc_melted = pd.melt(polar_hc, id_vars=[polar_hc.columns[0]],
    ↪ var_name="variable", value_name="value")
polar_hc_melted.rename(columns={polar_hc.columns[0]: "hc_label"}, inplace=True)

# Plot the polar chart
fig = px.line_polar(polar_hc_melted,
                    r="value",
                    theta="variable",
                    color="hc_label",
                    line_close=True,
                    height=700,
                    width=800)
fig.update_traces(marker=dict(size=4))
fig.show()
```

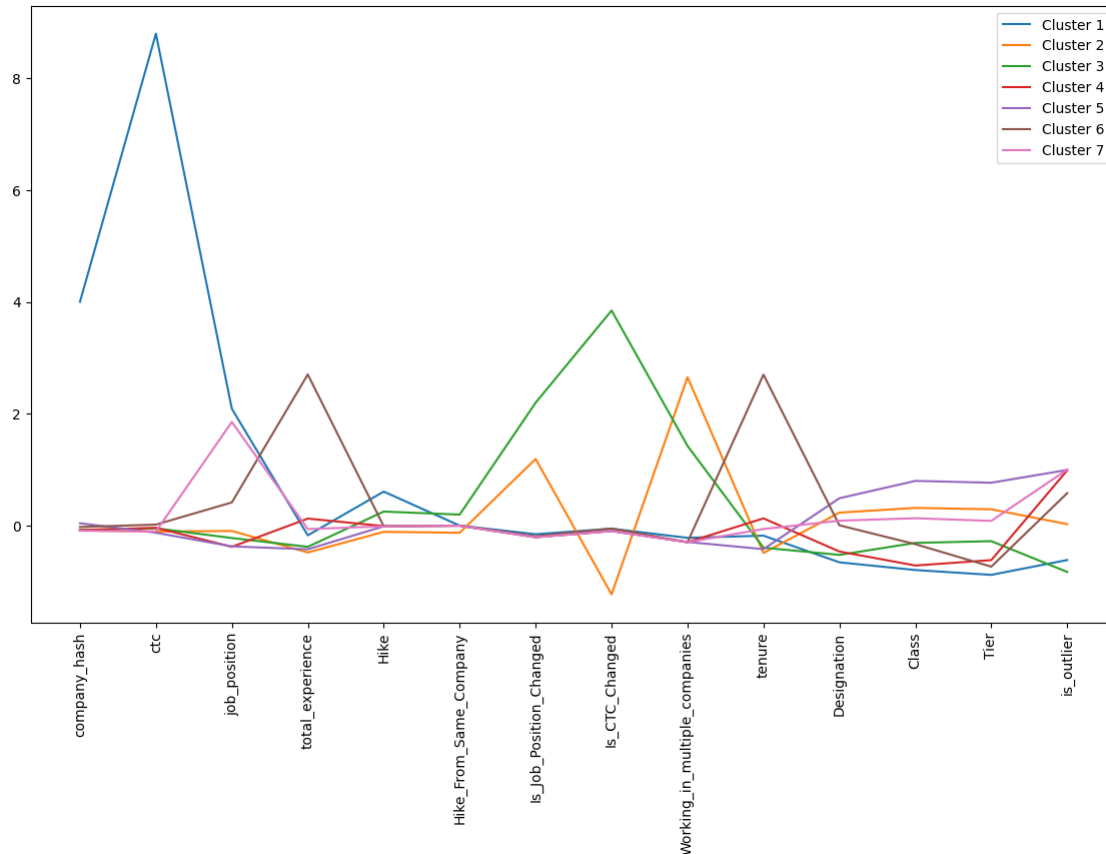
6.4.4 Visualising the mean feature values of clusters

```
[63]: clustered_df = df_train_clusters.groupby('hc_label').mean()

labels = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4', 'Cluster 5',
    ↪ 'Cluster 6', 'Cluster 7']
```

```
plt.figure(figsize=(14,8))
plt.plot(clustered_df.T, label=labels)
plt.xticks(rotation=90)
plt.legend(labels)
```

[63]: <matplotlib.legend.Legend at 0x18237db9ac0>



6.4.5 3D plot total_experience vs ctc vs Hike

[89]: df_train_clusters.head()

```
[89]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
16615	-0.116385	-0.030981	-0.479036	-1.207562	-0.689101	
68223	-0.397989	-0.058682	-0.495886	0.641095	-0.015194	
120736	0.011208	-0.149701	-0.551260	-0.976480	-0.015194	
45755	0.028608	-0.070554	-0.479036	-0.745398	-0.015194	
106270	0.028608	0.198545	0.596984	1.334341	-0.015194	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
16615	-0.002777	-0.208743	-0.098114	

68223	-0.002777	-0.208743	-0.098114
120736	-0.002777	-0.208743	-0.098114
45755	-0.002777	-0.208743	-0.098114
106270	-0.002777	-0.208743	-0.098114

	Working_in_multiple_companies	tenure	Designation	Class \
16615	3.354302	-1.200576	-0.098588	-1.578244
68223	-0.298125	0.646896	-0.098588	-0.163924
120736	3.354302	-0.969642	1.545323	1.250395
45755	-0.298125	-0.738708	-0.098588	-0.163924
106270	-0.298125	1.339698	-0.098588	-0.163924

	Tier	is_outlier	hc_label
16615	-0.211203	-1	1
68223	-1.525643	1	3
120736	1.103237	1	1
45755	-0.211203	1	4
106270	-0.211203	1	6

```
[52]: fig = px.scatter_3d(df_train_clusters, x='total_experience', y='ctc', z='Hike',
    ↪color='hc_label', width=600, height=600)
fig.update_traces(marker=dict(size=2), selector=dict(mode='markers'))
fig.show()
```

6.5 Agglomerative clustering using scipy

```
[58]: # import hierarchical clustering libraries
import scipy.cluster.hierarchy as sch

# Refer https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.
    ↪hierarchy.linkage.html#scipy.cluster.hierarchy.linkage
Z = sch.linkage(df_train, method='ward') #linkage = ward
```

```
[59]: Z.shape
```

```
[59]: (9999, 4)
```

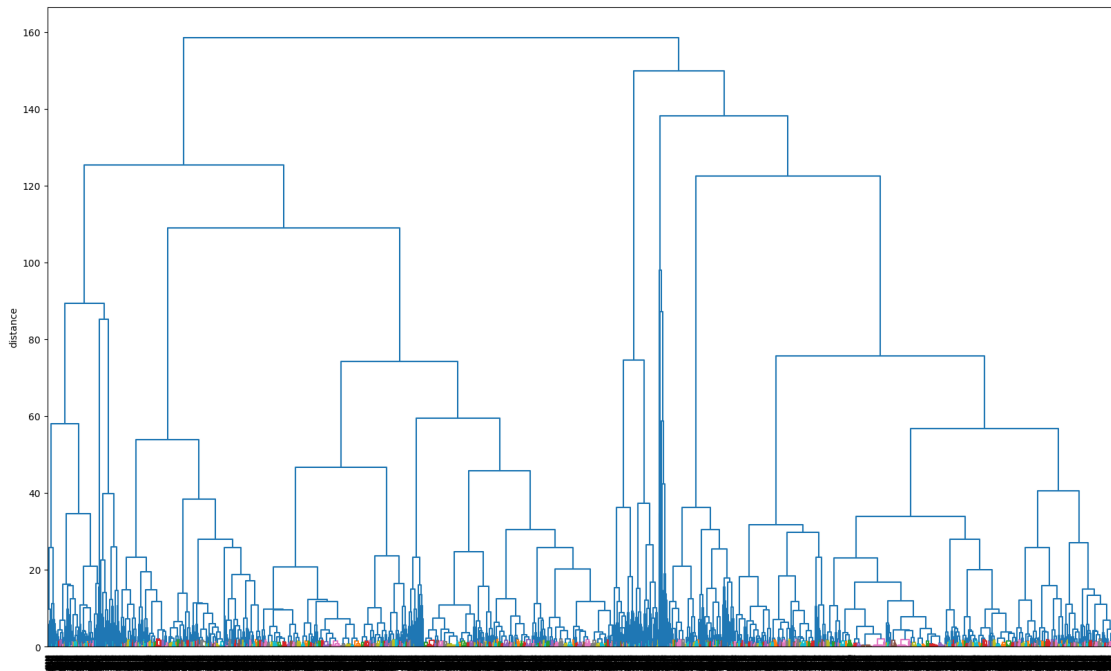
```
[60]: Z
```

```
[60]: array([[3.63700000e+03, 6.19900000e+03, 0.00000000e+00, 2.00000000e+00],
    [2.81300000e+03, 4.94600000e+03, 0.00000000e+00, 2.00000000e+00],
    [2.50000000e+01, 2.56500000e+03, 0.00000000e+00, 2.00000000e+00],
    ...,
    [1.99920000e+04, 1.99940000e+04, 1.38265912e+02, 4.28600000e+03],
    [1.99870000e+04, 1.99960000e+04, 1.49996071e+02, 4.71000000e+03],
    [1.99950000e+04, 1.99970000e+04, 1.58535499e+02, 1.00000000e+04]])
```

6.5.1 Dendrogram plot

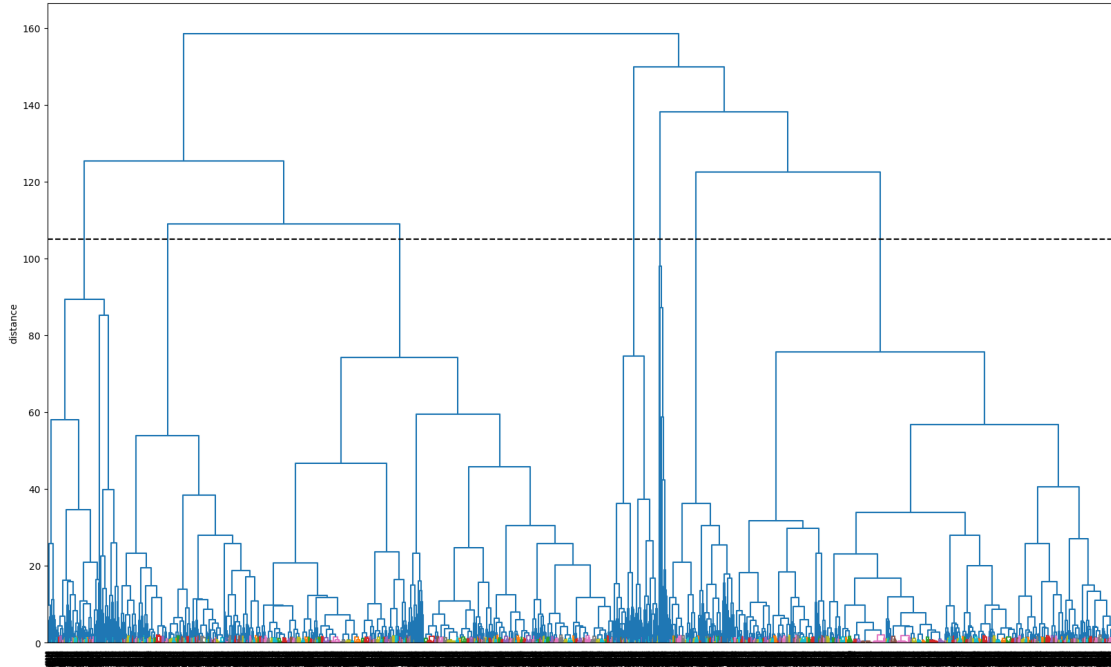
```
[61]: fig, ax = plt.subplots(figsize=(20, 12))
sch.dendrogram(Z, labels=df_train.index, ax=ax, color_threshold=2)
plt.xticks(rotation=90)
ax.set_ylabel('distance')
```

```
[61]: Text(0, 0.5, 'distance')
```



```
[64]: fig, ax = plt.subplots(figsize=(20, 12))
sch.dendrogram(Z, labels=df_train.index, ax=ax, color_threshold=2)
plt.xticks(rotation=90)
plt.axhline(y=105, color='k', linestyle='--')
ax.set_ylabel('distance')
```

```
[64]: Text(0, 0.5, 'distance')
```



Observation > The dendrogram representation shows how clusters are formed based on the similarity of features. By analyzing the ‘total_experience’ feature in mean feature values plot, we can see that clusters with similar years of experience are grouped together in the dendrogram. This indicates that ‘Years of Experience’ is a significant factor in the clustering process.

6.6 INSIGHTS

6.6.1 Cluster Interpretations and Names

1. **Cluster 0:**
 - **Interpretation:** Higher values in “ctc”
 - **Name:** “High Earners”
2. **Cluster 1:**
 - **Interpretation:** Higher values for “Working_in_multiple_companies” indicate frequent job changes.
 - **Name:** “Frequent Movers”
3. **Cluster 2:**
 - **Interpretation:** Higher values in “Is_CTC_changed” and Medium “Working in multiple companies” but low “ctc” and
 - **Name:** “Free Lancers with variable income”
4. **Cluster 3:**
 - **Interpretation:** Higher values in “total_experience” and “tenure” and “Outliers” indicate experienced professionals.
 - **Name:** “Veteran Outliers”
5. **Cluster 4:**
 - **Interpretation:** High “Designation” and “Class” values suggest senior-level positions

and higher professional classes.

- **Name:** “Senior Leaders”

6. **Cluster 5:**

- **Interpretation:** Higher values in “total_experience” and “tenure” indicate experienced professionals.
- **Name:** “Veterans”
-

7. **Cluster 6:**

- **Interpretation:** Balanced features with no significant spikes, indicating steady and moderate profiles.
- **Name:** “Steady Achievers”

6.7 Gaussian Mixture Models

```
[4]: k = 7
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=k, random_state=42).fit(X_train)
y_train_pred = gmm.fit_predict(X_train)
y_test_pred = gmm.predict(X_test)
```

```
[5]: gmm.score(X_train)
```

```
[5]: 28.885973159228698
```

```
[6]: gmm.aic(X_train)
```

```
[6]: -7263721.969009202
```

```
[7]: gmm.bic(X_train)
```

```
[7]: -7255548.321429373
```

```
[8]: with open('gmm.pkl', 'wb') as f:
    pickle.dump(gmm, f)
```

```
[8]: with open('gmm.pkl', 'rb') as file:
    gmm = pickle.load(file)
```

```
[9]: print(y_train_pred)
    print(y_test_pred)
```

```
[3 1 1 ... 0 1 1]
```

```
[3 3 3 ... 3 1 2]
```

```
[10]: # Get unique values and their counts
unique_values, counts = np.unique(y_train_pred, return_counts=True)

# Display the results
```

```
print("Unique Values:", unique_values)
print("Counts:", counts)
```

```
Unique Values: [0 1 2 3 4 5 6]
Counts: [ 5330 40636  5115 66675  1740  4064  2200]
```

```
[11]: # Get unique values and their counts
unique_values, counts = np.unique(y_test_pred, return_counts=True)

# Display the results
print("Unique Values:", unique_values)
print("Counts:", counts)
```

```
Unique Values: [0 1 2 3 4 5 6]
Counts: [ 1343  9927 1357 16844   436   970   563]
```

6.7.1 Visualizing clusters

```
[5]: X_train_clusters = pd.read_csv("X_train_clusters.csv")
X_test_clusters = pd.read_csv("X_test_clusters.csv")
```

```
[15]: X_train_clusters['gmm_label'] = y_train_pred
X_train_clusters.head()
```

```
[15]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
0	0.028608	-0.122791	-0.217902	-0.514316	-0.015194	
1	1.611623	-0.138620	1.936612	-0.283234	-0.015194	
2	0.052871	-0.165372	-0.479036	-0.745398	-0.015194	
3	-0.292440	-0.078469	-0.479036	-0.283234	-0.015194	
4	0.767031	-0.177402	0.224742	-1.207562	-0.015194	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
0	-0.002777	-0.208743	-0.098114	
1	-0.002777	-0.208743	-0.098114	
2	-0.002777	-0.208743	-0.098114	
3	-0.002777	-0.208743	-0.098114	
4	-0.002777	-0.208743	-0.098114	

	Working_in_multiple_companies	tenure	Designation	Class	Tier	\
0	-0.298125	-0.507774	-0.098588	-0.163924	-0.211203	
1	-0.298125	-0.276840	-0.098588	1.250395	1.103237	
2	-0.298125	-0.738708	1.545323	1.250395	1.103237	
3	-0.298125	-0.276840	-0.098588	-0.163924	-0.211203	
4	-0.298125	-1.200576	1.545323	1.250395	1.103237	

	is_outlier	k_means_label	k_means_plus_label	gmm_label
0	1	0	1	3

1	1	1	6	1
2	1	1	6	1
3	1	0	1	3
4	1	1	6	1

```
[16]: X_test_clusters['gmm_label'] = y_test_pred
X_test_clusters.head()
```

```
[16]:
```

	company_hash	ctc	job_position	total_experience	Hike	\
0	0.028608	-0.193232	-0.551260	-0.514316	-0.015194	
1	0.028608	0.078242	1.421846	2.489751	-0.015194	
2	-0.248843	0.032337	1.421846	2.027587	-0.015194	
3	0.028608	-0.189274	-0.974696	0.641095	-0.015194	
4	-0.178556	-0.064222	0.224742	-0.052151	-0.015194	

	Hike_From_Same_Company	Is_Job_Position_Changed	Is_CTC_Changed	\
0	-0.002777	-0.208743	-0.098114	
1	-0.002777	-0.208743	-0.098114	
2	-0.002777	-0.208743	-0.098114	
3	-0.002777	-0.208743	-0.098114	
4	-0.002777	-0.208743	-0.098114	

	Working_in_multiple_companies	tenure	Designation	Class	Tier	\
0	-0.298125	-0.507774	-0.098588	-0.163924	-0.211203	
1	-0.298125	2.494369	-0.098588	-0.163924	-0.211203	
2	-0.298125	2.032501	-0.098588	-0.163924	-1.525643	
3	-0.298125	0.646896	-0.098588	-0.163924	-0.211203	
4	-0.298125	-0.045906	-0.098588	-0.163924	1.103237	

	is_outlier	k_means_label	k_means_plus_label	gmm_label
0	1	0	1	3
1	1	3	3	3
2	1	3	3	3
3	1	0	1	3
4	1	1	1	1

```
[ ]: X_train_clusters.to_csv("X_train_clusters.csv", index=False)
X_test_clusters.to_csv("X_test_clusters.csv", index=False)
```

total_experience vs ctc plot

```
[9]: from matplotlib.patches import Ellipse
import numpy as np

def plot_gmm_clusters(gmm, X_train_clusters):
    # Scatter plot of the data points with cluster labels
    scatter = plt.scatter(
```

```

X_train_clusters['total_experience'],
X_train_clusters['ctc'],
c=X_train_clusters['gmm_label'],
cmap='viridis',
alpha=0.7
)

# Plot the mean vectors as centroids
centroid_handles = []
centroid_labels = []
for i, mean in enumerate(gmm.means_):
    # Extract mean values for the two features
    mean_2d = mean[:2]
    color = plt.cm.viridis(i / gmm.n_components) # Use the same colormap
    centroid_handles.append(plt.scatter(mean_2d[0], mean_2d[1],
    ↪color=color, marker="X", s=100, edgecolors="white"))
    centroid_labels.append(f"Centroid {i+1}")

# Plot covariance ellipses for the two features
for i, (mean, covar) in enumerate(zip(gmm.means_, gmm.covariances_)):
    # Extract the 2D covariance matrix
    if covar.ndim == 2: # Full covariance
        covar_2d = covar[:2, :2]
    else: # Diagonal covariance
        covar_2d = np.diag(covar[:2])

    # Compute eigenvalues and eigenvectors for ellipse orientation
    eigenvalues, eigenvectors = np.linalg.eigh(covar_2d)

    # Width and height of the ellipse
    width, height = 2 * np.sqrt(eigenvalues)
    angle = np.degrees(np.arctan2(eigenvectors[0, 1], eigenvectors[0, 0]))

    # Create an ellipse
    color = plt.cm.viridis(i / gmm.n_components)
    ellipse = Ellipse(xy=mean[:2], width=width, height=height, angle=angle,
    ↪alpha=0.2, color=color)
    plt.gca().add_patch(ellipse)

# Create legend for clusters
handles, labels = scatter.legend_elements(prop="colors", fmt="Cluster {x}")

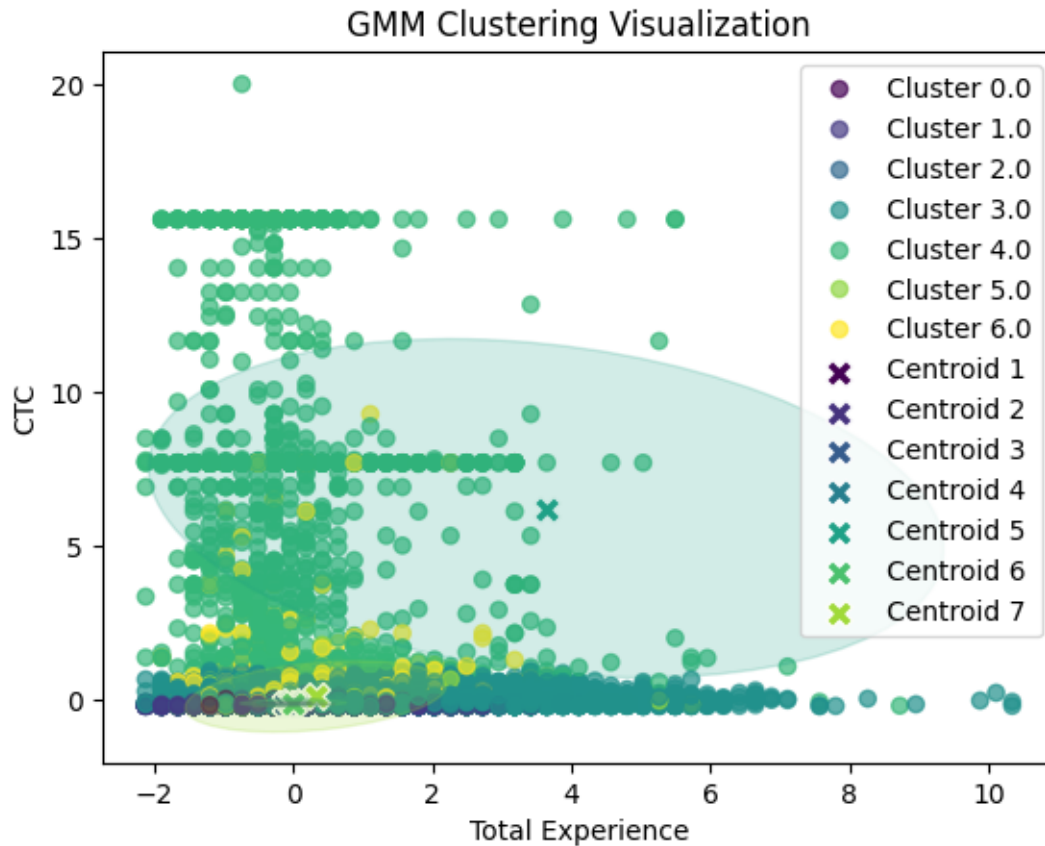
# Add legend for centroids
plt.legend(handles + centroid_handles, labels + centroid_labels, loc='best')

# Add labels and title
plt.xlabel('Total Experience')

```

```
plt.ylabel('CTC')
plt.title('GMM Clustering Visualization')
plt.show()

# Usage Example
plot_gmm_clusters(gmm, X_train_clusters)
```



polar line plot

```
[10]: polar = X_train_clusters.groupby("gmm_label").mean().reset_index()
polar = pd.melt(polar, id_vars=["gmm_label"])
polar.head()
```

```
[10]:
```

	gmm_label	variable	value
0	0	company_hash	-0.136554
1	1	company_hash	-0.056950
2	2	company_hash	-0.134163
3	3	company_hash	-0.048879
4	4	company_hash	3.643655

```
[12]: fig = px.line_polar(polar, r="value", theta="variable", color="gmm_label",
    ↪line_close=True,height=700,width=800)
fig.show()
```

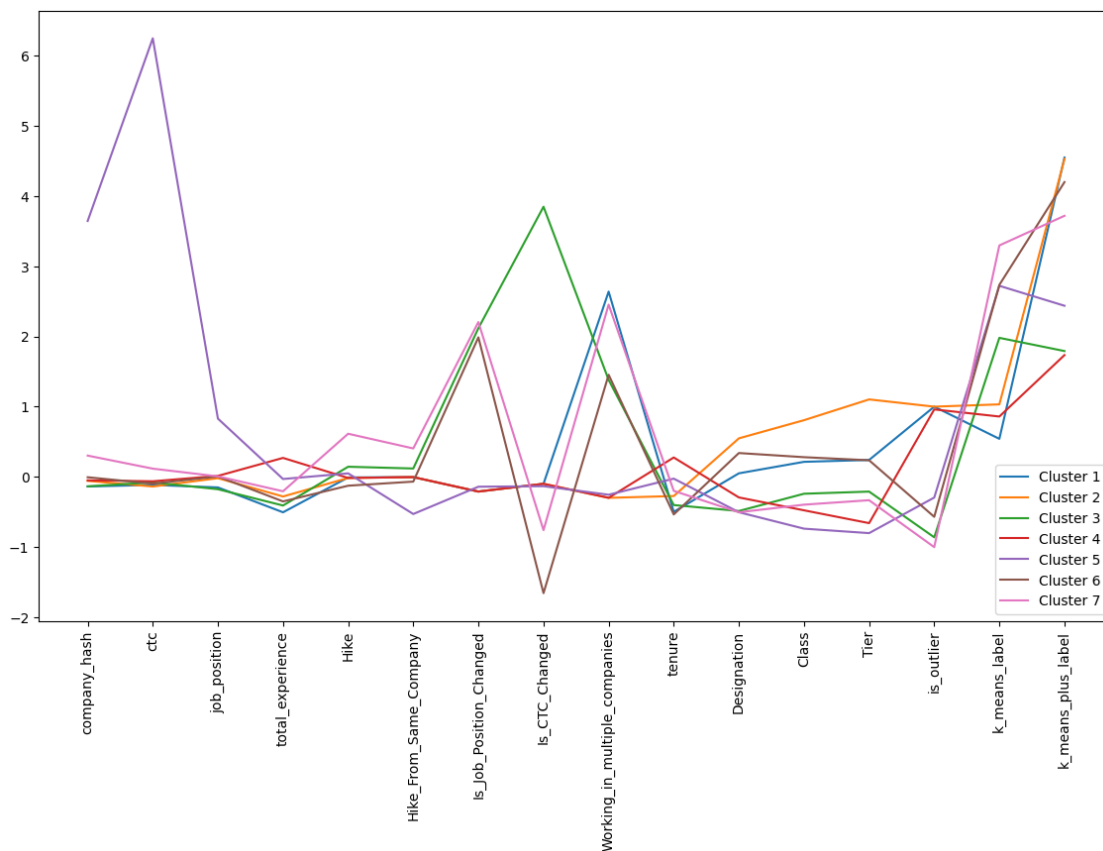
6.7.2 Visualising the mean feature values of clusters

```
[13]: clustered_df = X_train_clusters.groupby('gmm_label').mean()

labels = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4', 'Cluster 5',
    ↪'Cluster 6', 'Cluster 7']

plt.figure(figsize=(14,8))
plt.plot(clustered_df.T, label=labels)
plt.xticks(rotation=90)
plt.legend(labels)
```

[13]: <matplotlib.legend.Legend at 0x22e51bf3f80>



gmm_label UMAP Visualisation

```
[ ]: train_umap = UMAP(init = "random",n_neighbors = 200, min_dist = 0.1,
    ↪random_state = 42,n_jobs = -1)
binary_train_umap = train_umap.fit_transform(X_train)
binary_test_umap = train_umap.transform(X_test)
```

C:\Users\saina\AppData\Roaming\Python\Python312\site-packages\umap\umap_.py:1945: UserWarning:

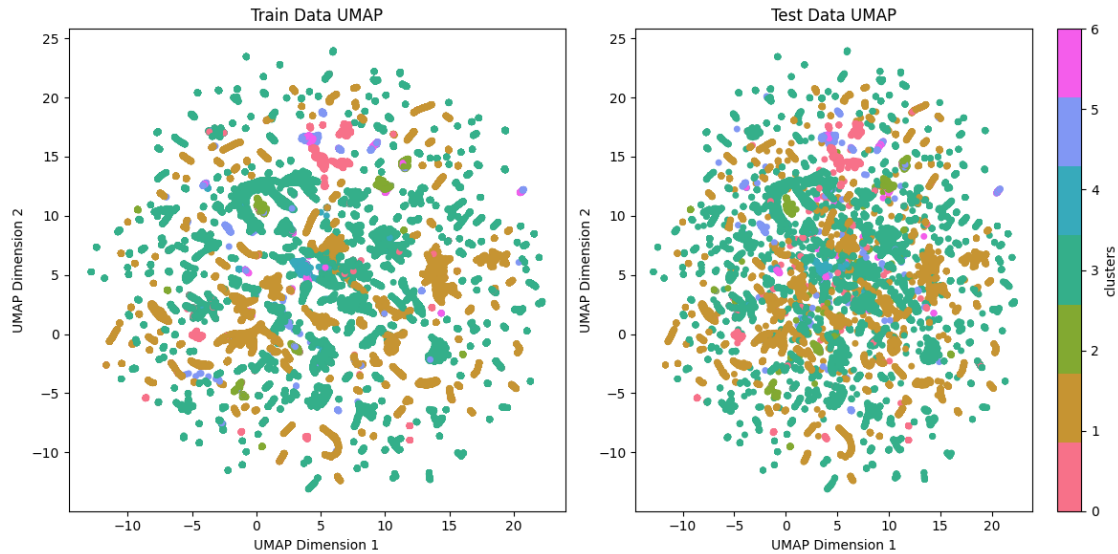
n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

```
[15]: def create_umap(binary_train_umap,y_train_pred,binary_test_umap,y_test_pred):
    # Create a figure with 1 row and 2 columns
    fig, axs = plt.subplots(1, 2, figsize=(12, 6))
    train_cmap = ListedColormap(sns.husl_palette(len(np.unique(y_train_pred))))

    # Plot the training data
    im_train = axs[0].scatter(binary_train_umap[:, 0],
                             binary_train_umap[:, 1],
                             s=25,
                             c=np.array(y_train_pred),
                             cmap=train_cmap,
                             edgecolor='none')
    axs[0].set_title('Train Data UMAP')
    axs[0].set_xlabel('UMAP Dimension 1')
    axs[0].set_ylabel('UMAP Dimension 2')

    # Plot the test data
    im_test = axs[1].scatter(binary_test_umap[:, 0],
                             binary_test_umap[:, 1],
                             s=25,
                             c=np.array(y_test_pred),
                             cmap= train_cmap,
                             edgecolor='none')
    axs[1].set_title('Test Data UMAP')
    axs[1].set_xlabel('UMAP Dimension 1')
    axs[1].set_ylabel('UMAP Dimension 2')
    # Add colorbar for training data
    cbar_train = fig.colorbar(im_train, ax=axs[1], label='clusters')

    # Display the plots
    plt.tight_layout()
    plt.show()
create_umap(binary_train_umap,X_train_clusters["gmm_label"],binary_test_umap,X_test_clusters["
```



6.7.3 3D plot

```
[51]: fig = px.scatter_3d(X_train_clusters, x='total_experience', y='ctc', z='Hike',
    ↪color='gmm_label', width=600, height=600)
fig.update_traces(marker=dict(size=2), selector=dict(mode='markers'))
fig.show()
```

6.7.4 INSIGHTS

6.7.5 Cluster Interpretations and Names

1. **Cluster 0:**
 - **Interpretation:** Higher values for “Working_in_multiple_companies” indicate frequent job changes.
 - **Name:** “Frequent Movers”
2. **Cluster 1:**
 - **Interpretation:** Higher values for “Working_in_multiple_companies” only. Indicates he works simultaneously in two companies
 - **Name:** “Parallel Jobs”
3. **Cluster 2:**
 - **Interpretation:** Higher values in “Is_CTC_changed” and Medium “Working in multiple companies” but low “ctc” and
 - **Name:** “Free Lancers with variable income”
4. **Cluster 3:**
 - **Interpretation:** Higher values in “Outliers” indicate
 - **Name:** “Outliers”
5. **Cluster 4:**
 - **Interpretation:** Higher values in “ctc”
 - **Name:** “High Earners”

6. Cluster 5:

- **Interpretation:** Higher values for “Working_in_multiple_companies” indicate frequent job changes but has high “designation,” class” and “Tier”
- **Name:** “Senior Frequent Movers”

7. Cluster 6:

- **Interpretation:** Higher values for “Working_in_multiple_companies” indicate frequent job changes but has low “designation,” class” and “Tier”
- **Name:** “Junior Frequent Movers”

6.8 DBSCAN

```
[11]: from sklearn.cluster import DBSCAN
dbsc = DBSCAN(eps = 1, min_samples = 30).fit(X_train)
```

```
[12]: y_train_pred = dbsc.labels_
```

```
[14]: from sklearn.metrics import silhouette_score

# Assuming dbscan labels are in train_labels (for X_train)
sil_score = silhouette_score(X_train, y_train_pred)
print("Silhouette Score:", sil_score)
```

Silhouette Score: 0.11320869390988435

```
[16]: X_train_clusters['dbsc_label'] = y_train_pred
X_train_clusters.head()
```

```
[16]:  company_hash      ctc  job_position  total_experience      Hike  \
0      0.028608 -0.122791   -0.217902      -0.514316 -0.015194
1      1.611623 -0.138620    1.936612      -0.283234 -0.015194
2      0.052871 -0.165372   -0.479036      -0.745398 -0.015194
3     -0.292440 -0.078469   -0.479036      -0.283234 -0.015194
4      0.767031 -0.177402    0.224742      -1.207562 -0.015194

      Hike_From_Same_Company  Is_Job_Position_Changed  Is_CTC_Changed  \
0             -0.002777             -0.208743             -0.098114
1             -0.002777             -0.208743             -0.098114
2             -0.002777             -0.208743             -0.098114
3             -0.002777             -0.208743             -0.098114
4             -0.002777             -0.208743             -0.098114

      Working_in_multiple_companies  tenure  Designation      Class      Tier  \
0             -0.298125 -0.507774      -0.098588 -0.163924 -0.211203
1             -0.298125 -0.276840      -0.098588  1.250395  1.103237
2             -0.298125 -0.738708      1.545323  1.250395  1.103237
3             -0.298125 -0.276840      -0.098588 -0.163924 -0.211203
4             -0.298125 -1.200576      1.545323  1.250395  1.103237
```

	is_outlier	k_means_label	k_means_plus_label	gmm_label	dbsc_label
0	1	0	1	3	0
1	1	1	6	1	1
2	1	1	6	1	2
3	1	0	1	3	0
4	1	1	6	1	2

```
[17]: X_train_clusters.to_csv("X_train_clusters.csv", index=False)
```

```
[18]: with open('dbsc.pkl', 'wb') as f:
      pickle.dump(dbsc, f)
```

```
[19]: # Get unique values and their counts
      unique_values, counts = np.unique(y_train_pred, return_counts=True)

      # Display the results
      print("Unique Values:", unique_values)
      print("Counts:", counts)
```

```
Unique Values: [ -1  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14
15 16
17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34
35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52
53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70
71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88
89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106
107 108 109 110 111 112 113 114 115 116 117]
Counts: [ 7582 30599 10734 17470   235  6196 1642    93  6741  7581   915   602
247  8936 1153   318 2897   94 1038 2214 2363  532  361   58
527   220 1865 1376  162  294  248   61  577   39  136   81
373  109  122  779   40  274  175  133   93  444   98   78
355  130  146  196   80   91  757   60  106  155   82  220
59  239   59  149   65   66   63  160  140   92   48   98
68   84  113   62   53  172   53   75   77  154   63  128
63   94   44   90  100   58  100   99   37  110   50   52
66  115  115   62   55   49   64  110   44   42   44   38
33   41   48   53   41   40   47   43   30   30   30]
```

6.8.1 Visualizing clusters

total_experience vs ctc plot

```
[23]: import numpy as np
      import matplotlib.pyplot as plt

      def plot_dbsc_clusters(dbsc, X_train_clusters):
          # Extract the dbsc labels from the X_train_clusters DataFrame
          y_train_pred = X_train_clusters['dbsc_label']
```

```

# Scatter plot of the data points with cluster labels
scatter = plt.scatter(
    X_train_clusters['total_experience'],
    X_train_clusters['ctc'],
    c=y_train_pred,
    cmap='viridis',
    alpha=0.7
)

# Plot the core points as centroids (Core points are the ones with label >= 0)
core_points = X_train_clusters[y_train_pred != -1] # Exclude noise points
core_labels = y_train_pred[y_train_pred != -1]

# Plot core points (cluster centers)
centroid_handles = []
centroid_labels = []
for i in np.unique(core_labels):
    cluster_points = core_points[core_labels == i]
    centroid = np.mean(cluster_points[['total_experience', 'ctc']], axis=0)
    # Mean of the cluster points (centroid)
    color = plt.cm.viridis(i / len(np.unique(core_labels))) # Use the same
    centroid_handles.append(plt.scatter(centroid[0], centroid[1],
    color=color, marker="X", s=100, edgecolors="white"))
    centroid_labels.append(f"Centroid {i+1}")

# Plot noise points (label -1)
noise_points = X_train_clusters[y_train_pred == -1]
plt.scatter(noise_points['total_experience'], noise_points['ctc'],
color='red', label='Noise', alpha=0.5)

# Create legend for clusters
handles, labels = scatter.legend_elements(prop="colors", fmt="Cluster {x}")

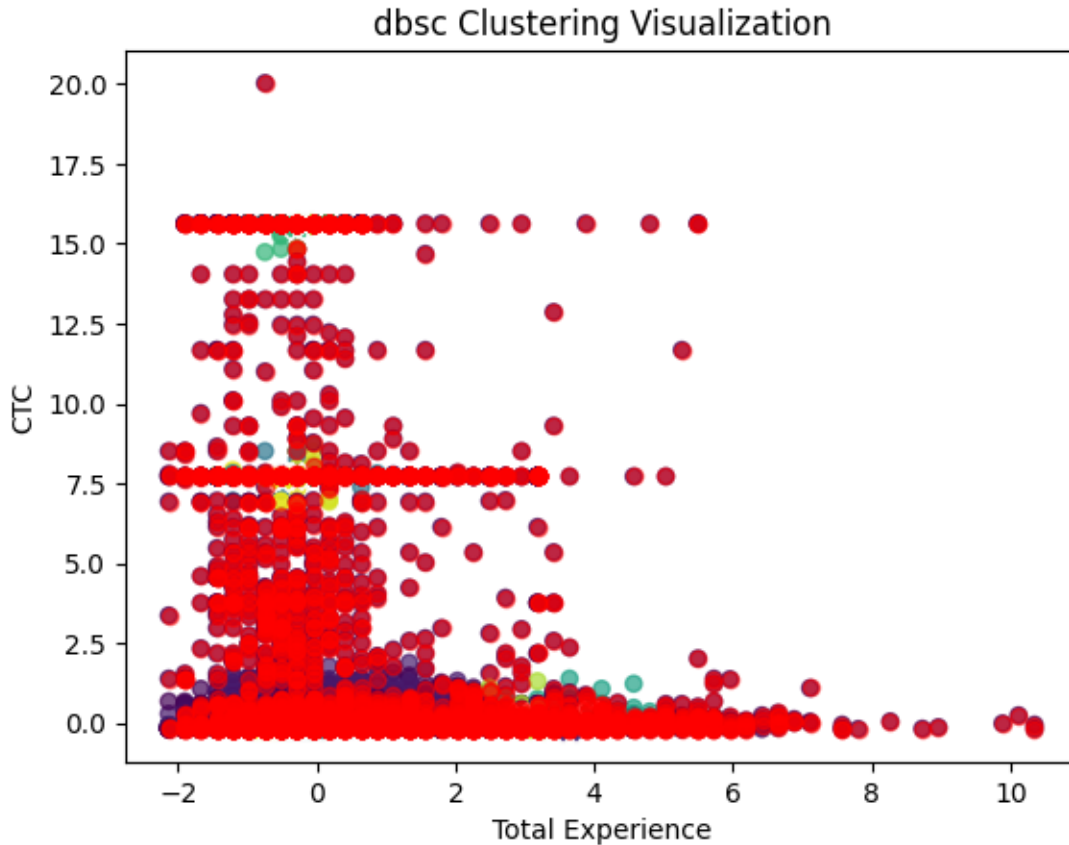
# Add legend for centroids and noise
# plt.legend(handles + centroid_handles, labels + centroid_labels +
['Noise'], loc='best')

# Add labels and title
plt.xlabel('Total Experience')
plt.ylabel('CTC')
plt.title('dbsc Clustering Visualization')
warnings.filterwarnings("ignore")

```

```
plt.show()

# Usage Example
plot_dbosc_clusters(dbosc, X_train_clusters)
```



visualising the mean feauture values of clusters

```
[31]: clustered_df = X_train_clusters.groupby('dbosc_label').mean()

# labels = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4', 'Cluster 5',
#           'Cluster 6', 'Cluster 7']

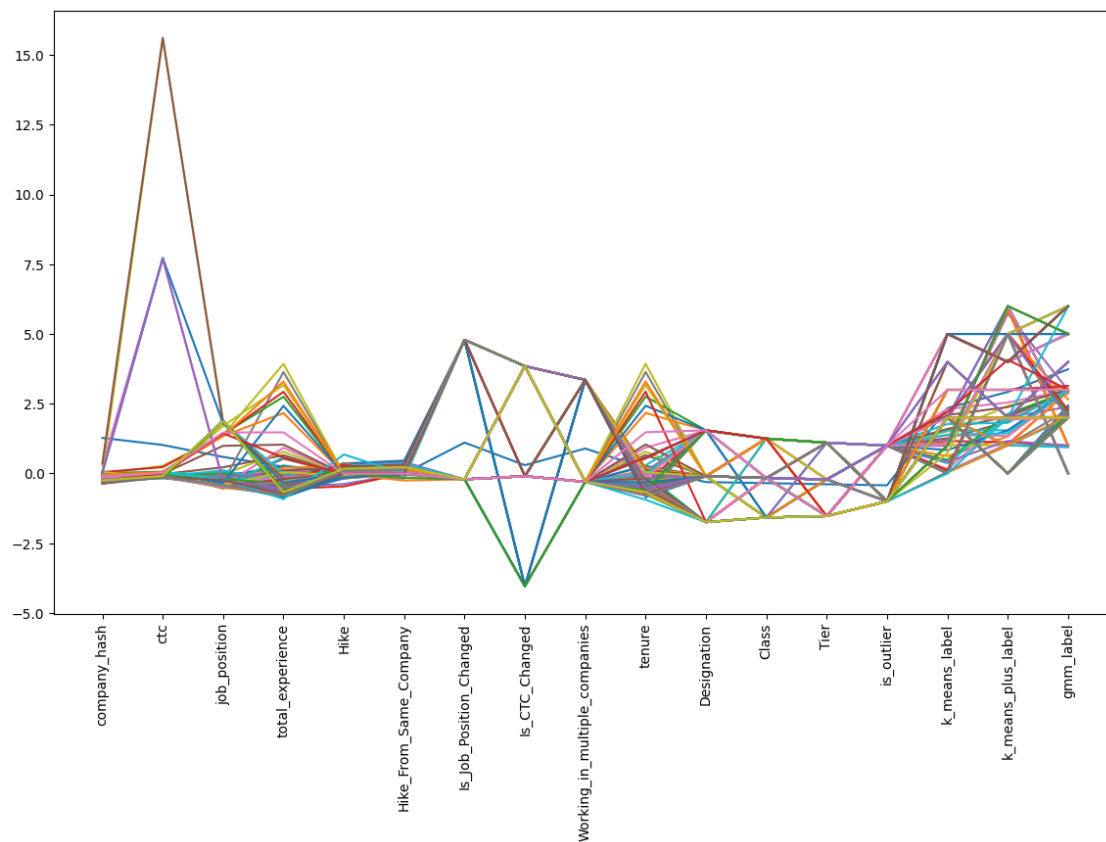
plt.figure(figsize=(14,8))
plt.plot(clustered_df.T)
plt.xticks(rotation=90)
# plt.legend(labels)
```

```
[31]: ([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16],
      [Text(0, 0, 'company_hash'),
       Text(1, 0, 'ctc'),
```

```

Text(2, 0, 'job_position'),
Text(3, 0, 'total_experience'),
Text(4, 0, 'Hike'),
Text(5, 0, 'Hike_From_Same_Company'),
Text(6, 0, 'Is_Job_Position_Changed'),
Text(7, 0, 'Is_CTC_Changed'),
Text(8, 0, 'Working_in_multiple_companies'),
Text(9, 0, 'tenure'),
Text(10, 0, 'Designation'),
Text(11, 0, 'Class'),
Text(12, 0, 'Tier'),
Text(13, 0, 'is_outlier'),
Text(14, 0, 'k_means_label'),
Text(15, 0, 'k_means_plus_label'),
Text(16, 0, 'gmm_label']]

```



dbsc_label UMAP Visualisation

```

[32]: import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import seaborn as sns

```

```

def create_umap(binary_train_umap, y_train_pred):
    # Create a figure
    fig, ax = plt.subplots(figsize=(6, 6))

    # Define a colormap for the training data based on unique cluster labels
    train_cmap = ListedColormap(sns.husl_palette(len(np.unique(y_train_pred))))

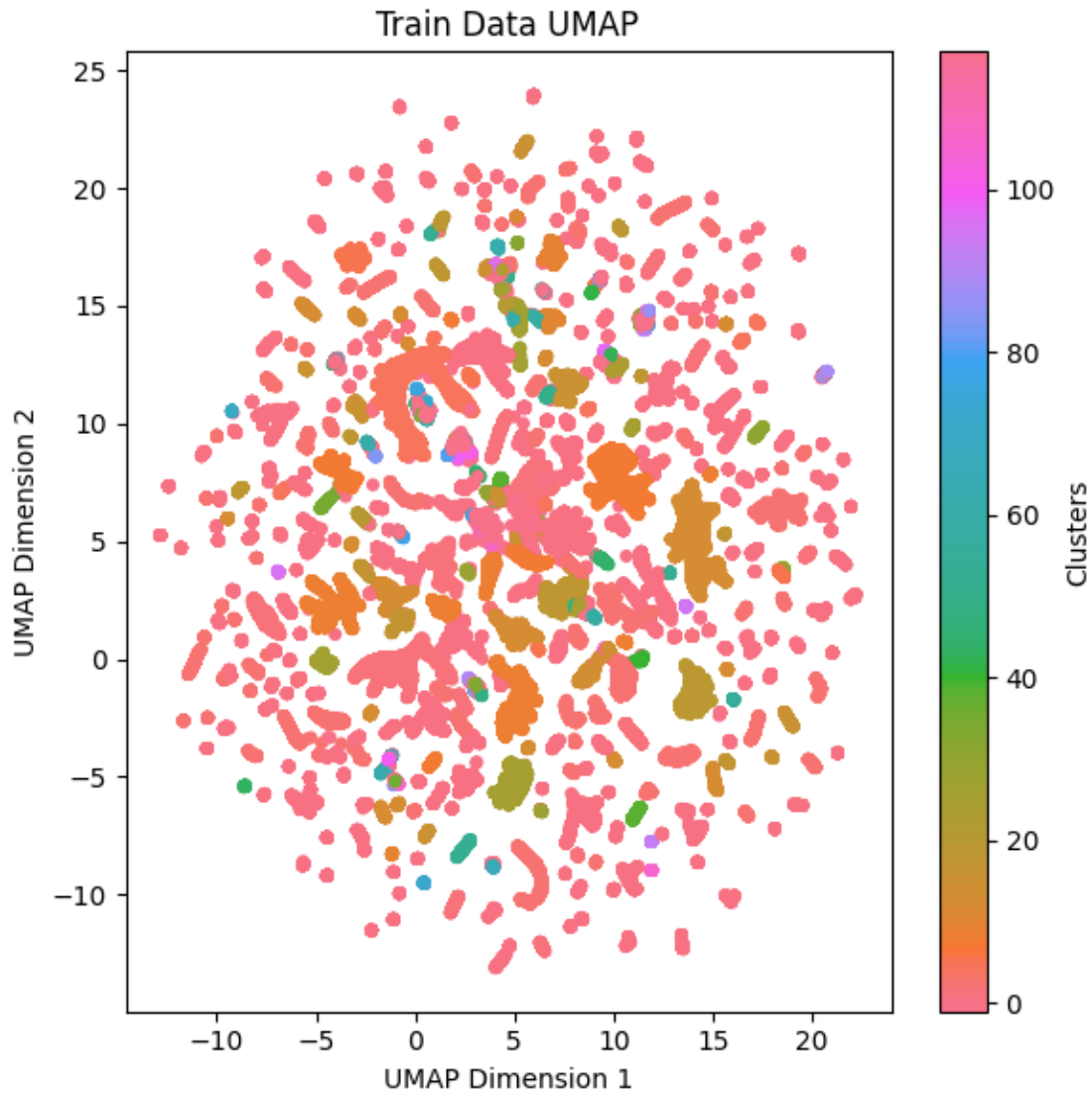
    # Plot the training data
    im_train = ax.scatter(binary_train_umap[:, 0],
                          binary_train_umap[:, 1],
                          s=25,
                          c=np.array(y_train_pred),
                          cmap=train_cmap,
                          edgecolor='none')
    ax.set_title('Train Data UMAP')
    ax.set_xlabel('UMAP Dimension 1')
    ax.set_ylabel('UMAP Dimension 2')

    # Add colorbar for the training data clusters
    cbar_train = fig.colorbar(im_train, ax=ax, label='Clusters')

    # Display the plot
    plt.tight_layout()
    plt.show()

# Usage Example (train data only)
create_umap(binary_train_umap, X_train_clusters["dbsc_label"])

```



6.8.2 3D plot

```
[53]: fig = px.scatter_3d(X_train_clusters, x='total_experience', y='ctc', z='Hike',
    ↪color='dbsc_label', width=600, height=600)
fig.update_traces(marker=dict(size=2), selector=dict(mode='markers'))
fig.show()
```

6.8.3 INSIGHTS

Dbscan is unable to provide proper differentiation between clusters.

Dbscan can be used to remove the outliers instead of using for clustering using density in this case.

7 INSIGHTS AND RECOMMENDATIONS

7.0.1 Key Insights from the Clustering Approaches:

1. **KMeans:**
 - It produced several clusters, with interpretations ranging from “**High Earners**” to “**Frequent Movers**”. However, there was an imbalance in the value counts across the clusters, which is not ideal for proper differentiation.
 - The “**Elbow Method**” suggested that 7 clusters was the optimal number, though the cluster imbalance could hinder meaningful segmentation.
2. **KMeans++:**
 - Showed better balance in the value counts compared to KMeans. The interpretation of the clusters remained similar, but the results were more stable.
 - “**Dynamic Movers**” and “**Frequent Movers**” were common names across both KMeans and KMeans++.
3. **Agglomerative Clustering:**
 - This method also produced several clusters, with a focus on “**High Earners**” and “**Frequent Movers**”, and some segmentation based on “**Veteran**” profiles and “**Outliers**”.
4. **Gaussian Mixture Model (GMM):**
 - GMM highlighted clusters like “**Frequent Movers**”, “**Parallel Jobs**”, and “**Free Lancers with Variable Income**”, with some overlap in interpretation with other models.
5. **DBSCAN:**
 - DBSCAN struggled to provide clear clustering. The density-based method worked better for removing outliers but did not differentiate the clusters well.

7.0.2 General Observations:

- The **optimal number of clusters** for most algorithms seems to hover around 7 (based on KMeans and KMeans++ results).
- **Cluster names** are centered around work patterns such as “**Frequent Movers**”, “**Veterans**”, and “**High Earners**”.
- Algorithms like **KMeans** and **Agglomerative Clustering** have good differentiation, while **DBSCAN** did not produce usable clusters and is better suited for outlier detection.

7.0.3 Recommendation:

- For actionable insights, focusing on **KMeans++** and **Agglomerative Clustering** might be more promising as they offer better balance and clustering quality compared to other methods.
- **DBSCAN** could be used solely for **outlier removal**, and not for primary clustering.